

---

# SH2SC-EDT and C2P-ARQ:

## A Self-Healing Hardware and Software Complex Utilizing the ChaCha20-Poly1305 Automatic Repeat reQuest Protocol

Oleksandr Dreval

Project's GitHub Repository

[github.com/OleksandrDreval/sh2sc-edt](https://github.com/OleksandrDreval/sh2sc-edt)

Project Documentation Site

[oleksandrdreval.github.io/sh2sc-edt](https://oleksandrdreval.github.io/sh2sc-edt)

E-mail

[drevaloleksandr@gmail.com](mailto:drevaloleksandr@gmail.com)

GitHub Profile

[github.com/OleksandrDreval](https://github.com/OleksandrDreval)

LinkedIn

[linkedin.com/in/oleksandr-dreval](https://linkedin.com/in/oleksandr-dreval)

*March 2026*

---

## Abstract

Commercial off-the-shelf (COTS) IoT communication modules operating over unshielded physical channels are vulnerable to active jamming and provide no integrated authenticated encryption at the transport layer, leaving session state irrecoverably lost upon link disruption. This paper introduces SH2SC-EDT (Self-Healing Hardware and Software Complex for Encrypted Data Transmission) and its transport engine, the C2P-ARQ (ChaCha-Poly Automatic Repeat reQuest) protocol. The architecture is founded on strictly deterministic, non-blocking finite state machines and employs ChaCha20-Poly1305 AEAD for per-packet authenticated encryption, with session nonces derived from a hardware TRNG seeded by multiple physical entropy sources. Protocol correctness is stressed via a stateful, time-dependent data stream whose variable inter-packet timing simultaneously exercises the Smart Dynamic Watchdog adaptive timeout and the Auto-Resume mechanism — properties that a static or homogeneous payload cannot validate. Hardware simulation on ATmega328P (16 MHz, 2 KB SRAM) in SimulIDE demonstrates that C2P-ARQ recovers desynchronised sessions after sustained jamming of arbitrary duration without data loss and without operator intervention, confirming the self-healing property at the application layer.

# Table of Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                  | <b>7</b>  |
| 1.1      | The Vulnerability Landscape of COTS RF Modules                       | 7         |
| 1.1.1    | Replay Attacks                                                       | 7         |
| 1.1.2    | Plaintext Injection                                                  | 7         |
| 1.1.3    | Session State Desynchronization                                      | 8         |
| 1.2      | The Jamming Threat and Synchronization Loss                          | 8         |
| 1.3      | Our Contribution: The SH2SC-EDT Architecture                         | 9         |
| 1.3.1    | Integrated Authenticated Encryption per Packet                       | 9         |
| 1.3.2    | Strictly Non-Blocking Finite State Machines                          | 10        |
| 1.3.3    | Self-Healing ARQ with Adaptive Watchdog                              | 10        |
| <b>2</b> | <b>System Architecture and Proof-of-Concept Design</b>               | <b>11</b> |
| 2.1      | The Proof-of-Concept Payload: Rationale for Melody Transmission      | 11        |
| 2.1.1    | Temporal Dependency                                                  | 11        |
| 2.1.2    | Stateful Data Stream                                                 | 12        |
| 2.1.3    | Audible Diagnostic                                                   | 12        |
| 2.1.4    | Downward Scalability                                                 | 12        |
| 2.2      | The “Fat Server / Thin Client” Paradigm                              | 13        |
| 2.3      | Deterministic Behavior via Finite State Machines (FSM)               | 14        |
| <b>3</b> | <b>Hardware Architecture and Simulation Environment</b>              | <b>16</b> |
| 3.1      | The SimulIDE Testbed: Topology and Physical Layer Abstraction        | 16        |
| 3.2      | Common Node Components                                               | 18        |
| 3.3      | Node Asymmetry: Interrupt-Driven TX and PWM-Driven RX                | 19        |
| 3.3.1    | Transmitter (Node A)                                                 | 19        |
| 3.3.2    | Receiver (Node B)                                                    | 20        |
| <b>4</b> | <b>True Random Number Generation (TRNG) &amp; Entropy Management</b> | <b>22</b> |
| 4.1      | Hardware Entropy Sources                                             | 23        |

|       |                                                                         |           |
|-------|-------------------------------------------------------------------------|-----------|
| 4.2   | The Cryptographic Entropy Blender                                       | 27        |
| 4.2.1 | Stage 1 — The Rotate-XOR Sponge                                         | 27        |
| 4.2.2 | Stage 2 — ChaCha20 Keystream Expansion                                  | 28        |
| 4.3   | Non-blocking CSPRNG Access at Runtime                                   | 30        |
| 5     | <b>The C2P-ARQ Protocol Specification</b>                               | <b>32</b> |
| 5.1   | Session Lifecycle: Handshake (SYN), Data Transfer (DAT), Teardown (FIN) | 33        |
| 5.1.1 | SYN Handshake                                                           | 33        |
| 5.1.2 | DAT Transfer Phase                                                      | 35        |
| 5.1.3 | FIN Teardown                                                            | 37        |
| 5.1.4 | Complete Session Sequence Diagram                                       | 37        |
| 5.2   | Wire Format: HelloPacket and DataPacket Structures                      | 38        |
| 5.2.1 | AAD: The Authenticated Plaintext Header                                 | 39        |
| 5.2.2 | HelloPacket — 15 Bytes on Wire                                          | 39        |
| 5.2.3 | DataPacket — 17 Bytes on Wire                                           | 40        |
| 5.3   | Acknowledgement Mechanism and Runaway Retry Prevention                  | 41        |
| 5.3.1 | The ARQ Feedback Loop                                                   | 41        |
| 5.3.2 | Timeout Condition and Retry Increment                                   | 42        |
| 5.3.3 | Runaway Retry Prevention and the Self-Healing Suspension                | 43        |
| 6     | <b>Security Model &amp; Cryptographic Core</b>                          | <b>46</b> |
| 6.1   | Authenticated Encryption with Associated Data (ChaCha20-Poly1305)       | 47        |
| 6.1.1 | Why Integrity Dominates Confidentiality for Command Channels            | 47        |
| 6.1.2 | The Role of Additional Authenticated Data — Frame Header Binding        | 47        |
| 6.1.3 | Encryption and Authentication Pipeline                                  | 48        |
| 6.1.4 | Formal Verification Predicate                                           | 49        |
| 6.2   | 16-bit Sequence Number and Nonce Management                             | 52        |
| 6.2.1 | The Nonce-Reuse Hazard                                                  | 52        |
| 6.2.2 | Per-Packet Nonce Derivation                                             | 35        |
| 6.2.3 | Injectivity of the Mapping                                              | 54        |



|       |                                                          |           |
|-------|----------------------------------------------------------|-----------|
| 6.2.4 | XOR versus Modular Addition                              | 55        |
| 6.3   | Key Zeroization and Secure Memory Management             | 55        |
| 6.3.1 | Data Remanence on ATmega328P SRAM                        | 55        |
| 6.3.2 | Forward Secrecy Through Deterministic Zeroization        | 56        |
| 6.3.3 | Zeroization Trigger Points                               | 57        |
| 6.3.4 | Entropy Seed Scrubbing                                   | 59        |
| 6.3.5 | Compiler Optimisation Note                               | 60        |
| 7     | <b>Fault Tolerance and Self-Healing Mechanisms</b>       | <b>61</b> |
| 7.1   | The RECONNECTING State and Payload State Preservation    | 62        |
| 7.1.1 | The Naive IoT Architecture                               | 62        |
| 7.1.2 | C2P-ARQ State Partition                                  | 62        |
| 7.1.3 | Self-Healing Invariant                                   | 62        |
| 7.1.4 | Non-Blocking RECONNECTING Polling                        | 64        |
| 7.2   | Smart Dynamic Watchdog: Payload-Aware Timeout Adaptation | 64        |
| 7.2.1 | The Receiver Deadlock Problem                            | 64        |
| 7.2.2 | Payload-Aware Adaptation                                 | 65        |
| 7.2.3 | Formal Timeout Condition                                 | 65        |
| 7.2.4 | Absence of Race Conditions Between TX and RX             | 67        |
| 7.3   | Stale Buffer Drain and Desynchronization Recovery        | 68        |
| 7.3.1 | The Stale-Byte Poisoning Problem                         | 68        |
| 7.3.2 | TX Active Drain — drainRxFifo()                          | 69        |
| 7.3.3 | RX Passive Drain — resetParser()                         | 69        |
| 7.3.4 | RX Passive Sync — Preamble Hunting                       | 70        |
| 7.3.5 | Probability of False Synchronization                     | 71        |
| 8     | <b>Adversary Model and Jamming Simulation</b>            | <b>73</b> |
| 8.1   | The Attacker Node Capabilities                           | 73        |
| 8.2   | Digital Noise Generator Architecture and XOR Injection   | 74        |
| 8.3   | Simulation Results: Resilience Under Prolonged Jamming   | 77        |

|          |                                                                             |            |
|----------|-----------------------------------------------------------------------------|------------|
| 8.3.1    | Scenario 1: System Initialization Under Active Jamming                      | 78         |
| 8.3.2    | Scenario 2: Baseline Payload Transmission (Clear Channel)                   | 81         |
| 8.3.3    | Scenario 3: Cryptographic Variance in Subsequent Packets (Clean Channel)    | 84         |
| 8.3.4    | Scenario 4: Active Jamming, Payload Corruption, and State Desynchronization | 87         |
| 8.3.5    | Scenario 5: Transmitter Timeout and Session Suspension                      | 92         |
| 8.3.6    | Scenario 6: Mutual Session Teardown and Dead Air                            | 94         |
| 8.3.7    | Scenario 7: Session Re-initialization Attempts Under Active Jamming         | 97         |
| 8.3.8    | Scenario 8: Successful Session Recovery and Application State Persistence   | 102        |
| 8.3.9    | Scenario 9: Legitimate Session Termination and Traffic Analysis Evasion     | 107        |
| <b>9</b> | <b>Conclusion and Future Work</b>                                           | <b>113</b> |
| 9.1      | Summary of Protocol Efficacy                                                | 113        |
| 9.2      | Migration to Actuator-Based IoT Commands                                    | 114        |
| 9.3      | Deep Sleep and Power Optimization                                           | 115        |

# 1 Introduction

The proliferation of low-cost wireless IoT devices has created infrastructure that is reliant on COTS radio modules never designed to operate in adversarial environments. These modules implement only lower-layer protocol functions, delegating session management, integrity verification, and confidentiality entirely to the application developer — a delegation that is routinely omitted in resource-constrained deployments. SH2SC-EDT addresses this structural gap by co-designing the physical transmission substrate with a dedicated session-layer protocol, C2P-ARQ, that integrates ChaCha20-Poly1305 authenticated encryption as a first-class primitive rather than an optional extension. [1, 4, 5]

## 1.1 The Vulnerability Landscape of COTS RF Modules

COTS sub-GHz transceivers (e.g., Si446x, CC1101, RFM69HW) implement the PHY layer — modulation, forward error correction, and RSSI measurement — and a rudimentary MAC layer providing address filtering and CRC-16 frame validation. Their hardware-defined protocol stack terminates at the link layer. No concept of a session, sequence-number binding, or cryptographic authentication exists below the application boundary; the transceiver is structurally blind to session state.

This architectural truncation creates three independently exploitable vulnerability classes.

### 1.1.1 Replay Attacks

Without per-session freshness tokens, any captured PHY frame constitutes a valid command. An adversary positioned within radio range can re-inject a previously observed packet and trigger the identical actuator response at will. CRC-16 provides no mechanism to distinguish a legitimate transmission from a captured retransmission, because both are structurally identical at the link layer. [3, 6, 7]

### 1.1.2 Plaintext Injection

Cleartext payloads permit a passive observer to recover the complete command encoding scheme after minimal traffic analysis. The attacker can subsequently construct and

it arbitrary commands without possessing any key material, because no cryptographic authentication gate exists below the application layer. The absence of confidentiality is thus not merely an information-disclosure risk but a direct enabler of active command injection. [1, 2, 4]

### 1.1.3 Session State Desynchronization

COTS modules expose no transport-layer state machine. A single noise burst sufficient to corrupt one frame leaves the transmitter and receiver with inconsistent sequence counters and no protocol-defined path to reconciliation. The only available recovery mechanism is a full power-cycle reset of one or both nodes, which is operationally unacceptable in autonomous or embedded deployments.

These vulnerabilities are not implementation defects in any particular product; they are a direct structural consequence of the OSI layer boundary at which COTS module firmware terminates. Addressing them requires an explicit transport-layer protocol that mandates AEAD per packet, manages session nonces, and provides deterministic desynchronization recovery — the precise function set that C2P-ARQ supplies. [1, 4, 5]

## 1.2 The Jamming Threat and Synchronization Loss

Active RF jamming is not a passive environmental phenomenon but a deliberate attack on the physical layer: an adversary transmits an interfering signal on the target frequency band, elevating the noise floor above the receiver sensitivity threshold. The immediate effect at the PHY layer is a bit-error rate increase that systematically exceeds the correction capacity of any employed FEC scheme, producing a continuous stream of CRC failures. Unlike multipath fading or thermal noise — which corrupt frames probabilistically and intermittently — a sustained jamming signal renders the channel deterministically unusable for its duration. Packet drop rates approach 100% under full-power jamming regardless of retransmission policy, because the corruption occurs prior to any ARQ layer visibility.

The critical consequence for stateful protocols is session state mismatch. When a DATA frame carrying sequence number  $N$  is corrupted in transit, the transmitter's FSM remains in `WAITING_ACK` (expecting a response to  $N$ ), while the receiver's FSM remains in its prior state (having never processed  $N$ ). If the corresponding ACK or NACK is subsequently lost in the noise, the transmitter cannot distinguish a silent channel from a

crashed receiver. Symmetrically, if a NACK is lost, the transmitter may advance its sequence counter while the receiver expects a retransmission of  $N$  — producing an irrecoverable sequence number divergence. Either scenario results in an FSM state mismatch from which neither endpoint can recover without explicit protocol support.

In COTS-based deployments lacking a defined session-recovery mechanism, this state mismatch escalates to an FSM deadlock: the transmitter holds an open session handle while awaiting an acknowledgement that will never arrive, and the receiver occupies session RAM while awaiting a data frame that will never be sent. The stateful data stream is permanently interrupted. The only exit from this deadlock is an out-of-band reset — a manual power cycle or watchdog expiry that discards all session state and forces a cold restart from the beginning of the transmission sequence. For any application requiring continuity across disruptions, this behaviour is a fundamental protocol deficiency, not a deployment anomaly.

## 1.3 Our Contribution: The SH2SC-EDT Architecture

This paper presents SH2SC-EDT, a hardware-software complex and its transport engine C2P-ARQ, designed to eliminate the three vulnerability classes identified in [Section 1.1](#) and the deadlock condition described in [Section 1.2](#). The architecture rests on three co-designed pillars that collectively transform an unprotected, stateless radio link into a fault-tolerant, cryptographically authenticated session channel.

### 1.3.1 Integrated Authenticated Encryption per Packet

C2P-ARQ mandates ChaCha20-Poly1305 AEAD for every DATA and FIN frame without exception. A per-packet nonce is derived deterministically from the session nonce and the 16-bit sequence number, ensuring that no two packets share an IV across all 65,536 possible sequence values. The 3-byte frame header — comprising the flag byte and the little-endian sequence number — is bound to the ciphertext as Additional Authenticated Data (AAD), so any single-bit corruption of the flags or sequence counter invalidates the Poly1305 tag. This construction renders replay attacks, plaintext injection, and silent data manipulation cryptographically infeasible without knowledge of the 256-bit pre-shared key compiled into firmware.

### 1.3.2 Strictly Non-Blocking Finite State Machines

All transmitter and receiver logic is implemented as deterministic FSMs with no `delay()` calls in the main execution loop. Every FSM transition is conditioned on `millis()` delta comparisons, making RAM occupancy and CPU time per loop iteration statically bounded and predictable. The transmitter FSM defines nine explicit states covering the full session lifecycle — `IDLE`, `RECONNECTING`, `SENDING_HELLO`, `WAITING_HELLO_ACK`, `SENDING`, `WAITING_ACK`, `WAIT_BETWEEN_NOTES`, `SENDING_FIN`, and `WAITING_FIN_ACK` — while the receiver implements a four-state byte-level parser, as shown in Figure 1.1, driven by each incoming UART byte. This design eliminates the class of concurrency bugs that arise from blocking I/O and guarantees deterministic behaviour under the real-time constraints of the SimulIDE simulation environment.

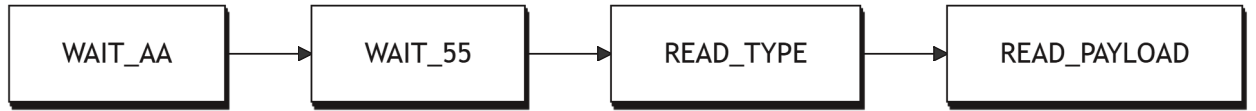


Figure 1.1: RX byte-level frame parser states (`processReceivedByte()`): each incoming UART byte advances the parser one step; `resetParser()` on MAC failure or timeout returns the parser to `WAIT_AA`.

### 1.3.3 Self-Healing ARQ with Adaptive Watchdog

C2P-ARQ incorporates an Auto-Resume mechanism that explicitly preserves payload progress state (`melodyIndex`) across session failures. Upon exhausting `MAX_RETRIES` retransmissions in any `WAITING_*` state, the transmitter invokes `suspendSession()` — zeroing the session nonce for forward secrecy, resetting the retry counter, and entering `RECONNECTING` — while leaving the transmission index intact. A fresh SYN handshake re-establishes the session, after which data transfer resumes from the exact packet that preceded the disruption. On the receiver side, the Smart Dynamic Watchdog computes a per-packet timeout as `note_duration_ms + NETWORK_GRACE_PERIOD_MS`, ensuring that session RAM is released promptly when a transmitter disappears without sending a FIN frame, regardless of the duration of the interrupted note. The entire architecture targets ATmega328P (16 MHz, 8-bit AVR, 2 KB SRAM) and is validated end-to-end in SimulIDE under hardware-injected noise produced by an XOR gate driven by a 1 kHz pulse generator.

## 2 System Architecture and Proof-of-Concept Design

SH2SC-EDT is architected around two co-equal constraints: node asymmetry, which confines stateful session logic exclusively to the transmitter, and strict determinism, which eliminates every blocking path from both firmware execution models. These two constraints are necessary but not sufficient for validation — a protocol that handles only static, homogeneous payloads may pass correctness checks under benign conditions while concealing latent failures in its timing and state-recovery logic. End-to-end validation therefore demands a payload that is inherently stateful, temporally heterogeneous, and human-verifiable without instrumentation.

### 2.1 The Proof-of-Concept Payload: Rationale for Melody Transmission

The proof-of-concept instantiation of C2P-ARQ transmits a monophonic musical score — specifically, the Super Mario Bros. theme — encoded as a sequence of (`note_index`, `duration_ms`) pairs stored in `PROGMEM` on the transmitter node. Each pair is independently encrypted, authenticated, and acknowledged before the next is sent. This choice of payload is not incidental; it is an engineering decision grounded in four distinct validation properties that a discrete or static payload cannot provide.

#### 2.1.1 Temporal Dependency

Note durations in a real musical score span at least two orders of magnitude — from sixteenth notes ( $\approx 100$  ms) to held notes exceeding 1000 ms. This heterogeneity is the only mechanism by which the Smart Dynamic Watchdog can be exercised under realistic conditions. The watchdog sets its session-release timeout  $T_{RX}$  dynamically after each authenticated packet according to the following formula:

$$T_{RX} = d_k + T_{\text{grace}}$$

where  $T_{RX}$  is the session-release timeout computed for each packet,  $d_k$  is the duration of the  $k$ -th note in milliseconds, and  $T_{\text{grace}}$  is a fixed network grace period absorbing the

worst-case retry storm. A payload of uniform duration would reduce this adaptive mechanism to a static timer, leaving its core adaptation logic entirely untested.

### 2.1.2 Stateful Data Stream

A musical score is an ordered sequence with a well-defined progress index. This property makes the Auto-Resume mechanism directly and unambiguously verifiable: after a session disruption at note index  $N$ , a correct implementation must resume at exactly  $N$  upon reconnection — not from index 0. An observer can confirm or refute this guarantee by ear in real time. A stateless payload — such as a single actuator toggle — provides no such checkpoint structure; the Auto-Resume mechanism would produce output indistinguishable from a cold restart.

### 2.1.3 Audible Diagnostic

The buzzer output on the receiver node constitutes an always-on, zero-instrumentation diagnostic channel. Any packet loss produces an audible gap; any duplicate delivery produces a repeated note; any timing deviation corrupts the rhythmic structure. These anomalies are immediately perceptible to a human observer without log analysis, oscilloscope attachment, or post-processing. This is the human-in-the-loop verification property: the entire C2P-ARQ session lifecycle — including retransmission, MAC rejection, and Auto-Resume — is audible as a seamless, uninterrupted melody when the protocol operates correctly.

### 2.1.4 Downward Scalability

The melody transmission scenario represents the hardest case within a class of streaming, time-sensitive payloads. The C2P-ARQ wire format is payload-agnostic: the (`note_index`, `duration_ms`) pair occupies the same four encrypted bytes that would carry any (`command_id`, `parameter`) pair in a production IoT deployment. If the protocol sustains correct, ordered, authenticated delivery under sustained jamming with a heterogeneous timing profile, its correctness under a simpler discrete-command workload follows directly. The migration from melody notes to actuator commands requires zero protocol-layer modifications.



## 2.2 The “Fat Server / Thin Client” Paradigm

C2P-ARQ deliberately concentrates the entire state burden of the session on the transmitter node. The transmitter — designated Node A — owns the melody array in `PROGMEM`, maintains the current `melodyIndex`, drives the full nine-state FSM lifecycle from `IDLE` through `WAITING_FIN_ACK`, executes `suspendSession()` on link failure, and manages `retryCount`, `seqNum`, all `millis()`-based timeout comparisons, and the reconnection interval. This asymmetric complexity allocation is not a consequence of implementation convenience but a deliberate protocol-level design choice: in any point-to-point ARQ system, exactly one endpoint must own the authoritative progress state. Placing that state on the transmitter confines the surface area of session logic to a single node and makes the receiver structurally stateless with respect to payload progress. [14, 15, 16]

The receiver — Node B — shoulders only the minimum work necessary to authenticate, decode, and execute each incoming command. Upon receiving a complete frame, the receiver derives the per-packet nonce, executes the full ChaCha20-Poly1305 decryption and Poly1305 tag verification pipeline, and conditionally plays the decoded note. No knowledge of the melody, no `melodyIndex` counter, and no session-recovery logic resides on the receiver. Its sole session-layer state is a 12-byte nonce buffer and a single boolean `s_sessionActive` flag. Error handling on the receiver is correspondingly minimal: a frame that fails tag verification is silently discarded after a `NACK_BYTE` is emitted on the feedback channel; the receiver does not log the failure, does not schedule a retransmission request, and does not alter its FSM state. All retransmission scheduling, retry counting, and timeout management are delegated entirely to the transmitter.

This asymmetric complexity distribution yields a concrete engineering advantage beyond conceptual elegance. In realistic IoT deployments, the receiver is frequently the more resource-constrained node — a battery-powered actuator, a relay controller, or a sensor hub with kilobytes of available RAM and no persistent storage. The Fat Server / Thin Client pattern ensures that the RAM footprint and per-iteration CPU cost of the receiver remain statically bounded and minimal regardless of session complexity, payload length, or network condition. The transmitter, which typically operates from mains power or a larger battery budget, absorbs the full computational and memory cost of the protocol. This allocation is stable under workload scaling: adding more melody notes, extending `MAX_RETRIES`, or increasing `RECONNECT_INTERVAL_MS` affects only the transmitter’s behaviour and leaves the receiver firmware entirely unchanged. [14, 15, 16]

## 2.3 Deterministic Behavior via Finite State Machines (FSM)

Blocking routines are a fundamental architectural incompatibility with transport-layer protocol implementations on single-core microcontrollers. A call to `delay()` suspends the CPU unconditionally for its entire duration, during which the processor cannot service incoming UART bytes, evaluate pending ACK timeouts, advance the TRNG entropy collection loop, or update the LCD display. On an ATmega328P operating at 9600 bps, a single `delay(50)` invocation occupies the same wall-clock interval as nearly five complete UART byte transmissions — sufficient for an arriving ACK to overflow the 64-byte hardware FIFO and be silently discarded. The result is CPU starvation: the protocol layer becomes deaf to channel events precisely during the intervals when it must be most responsive. Any correct C2P-ARQ implementation must therefore eliminate blocking routines from the main execution loop without exception. [17, 18]

C2P-ARQ resolves this constraint through exclusive reliance on non-blocking FSMs whose state transitions are conditioned on `millis()` delta comparisons. Each invocation of `loop()` executes at most one FSM transition, consumes a statically bounded number of instructions, and returns immediately — achieving a per-iteration latency measured in microseconds rather than milliseconds. Timeout conditions such as the ACK wait window (`ACK_TIMEOUT_MS = 50 ms`), the reconnection broadcast interval (`RECONNECT_INTERVAL_MS = 2000 ms`), the note playback duration, and the Smart Dynamic Watchdog limit are all implemented as comparisons of the form `(millis() - t_start) >= threshold`, evaluated once per loop iteration and advancing the FSM only when the condition holds. No thread synchronisation, no interrupt-driven state mutation, and no blocking I/O path exists anywhere in the protocol stack.

This design provides temporal determinism as a first-class property: at any point during execution, the precise state of the system is encoded in a single enum value with no hidden implicit state. The transmitter FSM defines nine states — `IDLE`, `RECONNECTING`, `SENDING_HELLO`, `WAITING_HELLO_ACK`, `SENDING`, `WAITING_ACK`, `WAIT_BETWEEN_NOTES`, `SENDING_FIN`, and `WAITING_FIN_ACK` — covering the complete session lifecycle from quiescence through authenticated teardown. The receiver implements a four-state byte-level parser, as shown in Figure 2.1, that advances one byte at a time and resets atomically to `WAIT_AA` on any MAC failure or parser timeout, guaranteeing that frame boundary corruption cannot propagate into subsequent frames. Because all session-recovery logic — `suspendSession()`, nonce erasure, `melodyIndex` preservation — executes within well-defined state transitions, the Auto-Resume mechanism is free of race conditions and memory leaks by construction: the system may remain in `RECONNECTING` indefinitely under sustained jamming without accumulating unbounded resource consumption. [19, 20]

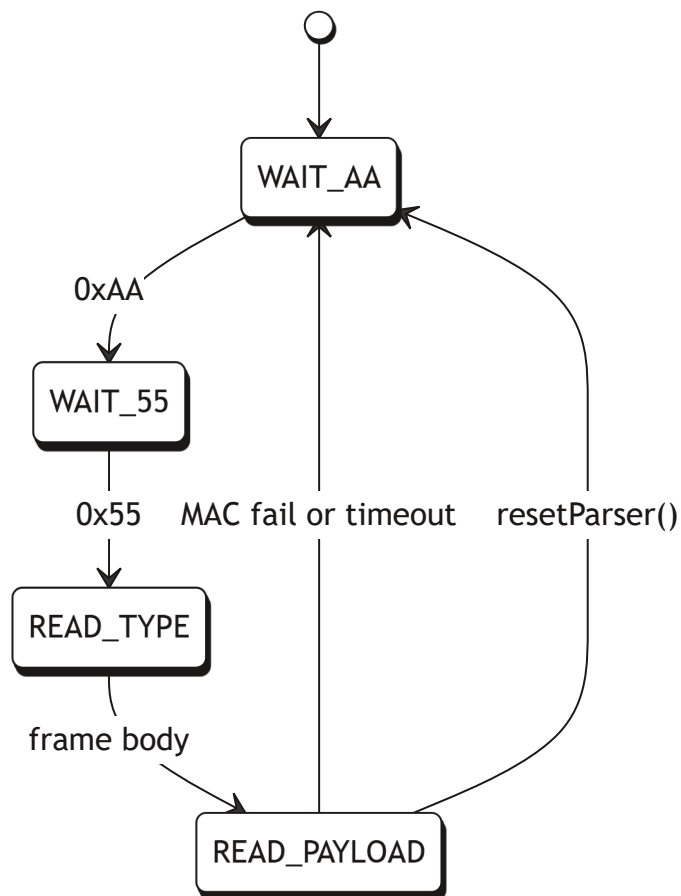


Figure 2.1: RX parser recovery path. The byte-level FSM advances through preamble admission and returns to **WAIT\_AA** on MAC failure, parser timeout, or an explicit **resetParser()** call.

## 3 Hardware Architecture and Simulation Environment

SH2SC-EDT was designed from inception to be validated within a fully reproducible simulation environment. Every physical component — the UART channel, the I2C bus, the piezo buzzer, the entropy ring oscillator, and the XOR noise injector — is modelled in SimulIDE, a real-time electronic circuit simulator, enabling rigorously repeatable fault-tolerance experiments without access to physical hardware. All timing constants, baud-rate settings, and MAC verification results presented in this paper were obtained directly from this simulation; no post-hoc calibration or parameter tuning was performed outside the simulation environment.

### 3.1 The SimulIDE Testbed: Topology and Physical Layer Abstraction

The SH2SC-EDT simulation topology comprises two microcontroller nodes and an Attacker sub-circuit, as shown in Figure 3.1. Node A (Transmitter, “The Conductor”) is an ATmega328P-based Arduino Nano responsible for the entire C2P-ARQ session lifecycle: it owns the `PROGMEM` melody table, maintains the authoritative `melodyIndex`, drives the nine-state transmitter FSM from `IDLE` through `WAITING_FIN_ACK`, and executes `suspendSession()` on link failure. Node B (Receiver, “The Synthesizer”) is a second Arduino Nano that holds only the 21-entry universal note-frequency dictionary and the 12-byte session nonce; it is intentionally stateless with respect to melody progress. The Attacker is a hardware-only sub-circuit — it is not a microcontroller, not a third node, and not a firmware entity. It consists of an XOR gate, a 1 kHz Pulse Generator with 15% duty cycle, an AND gate, and a manual Switch, wired in series on the UART data line between Node A and Node B.

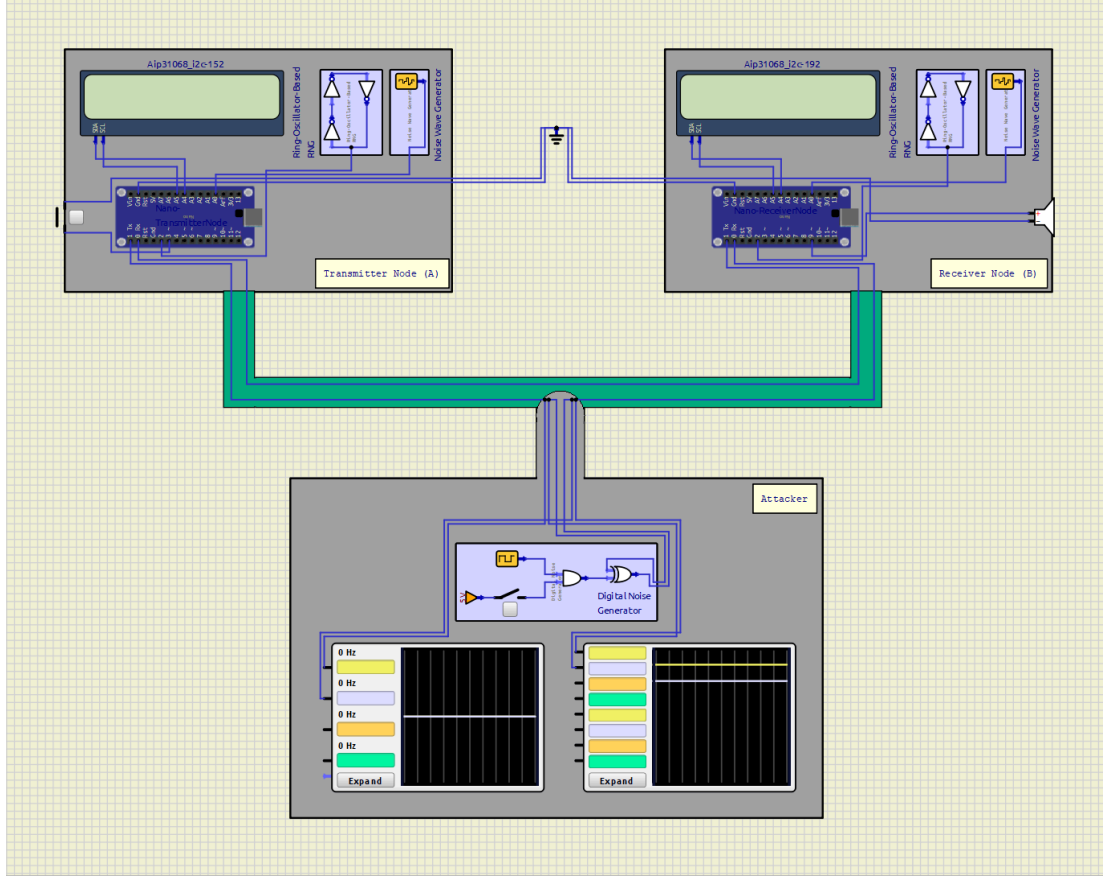


Figure 3.1: Full simulation topology in SimulIDE. Node A (Transmitter) is connected to Node B (Receiver) over a UART bus routed through the Attacker XOR noise injector. The clean feedback channel bypasses the injector entirely.



Figure 3.2: UART feedback channel direction in the SH2SC-EDT testbed. Node B TX reaches Node A RX only through the clean, unattacked feedback wire carrying ACK\_BYTE and NACK\_BYTE.

SimulIDE provides no RF propagation model. The physical layer is accordingly abstracted as a point-to-point UART digital bus operating at 9600 bps (8N1). This abstraction is not a limitation but a deliberate design choice: by removing RF-layer complexity — multipath, frequency-selective fading, antenna gain patterns — the experiment isolates the protocol layer as the sole variable under test. The UART bus carries two unidirectional channels, as shown in Figure 3.3. The data channel, routed from Node A TX to Node B RX, passes through the Attacker XOR gate: with the noise Switch open, the

XOR output equals the original signal; with the Switch closed, the Pulse Generator’s 1 kHz rectangular waveform is XORed onto the bus, flipping individual UART bits and producing realistic partial-frame corruption. At 9600 bps, a single UART bit spans approximately 104  $\mu$ s; a 150  $\mu$ s noise pulse (15% of a 1000  $\mu$ s period) is therefore wide enough to corrupt one to nine consecutive bits per burst without destroying every bit in every frame — producing the probabilistic packet-error distribution that the C2P-ARQ ARQ layer is designed to survive. The feedback channel, running from Node B TX to Node A RX, is a clean, unattacked direct wire. It carries only single-byte control frames: `ACK_BYTE = 0x06` on successful Poly1305 verification, and `NACK_BYTE = 0x15` on MAC failure.

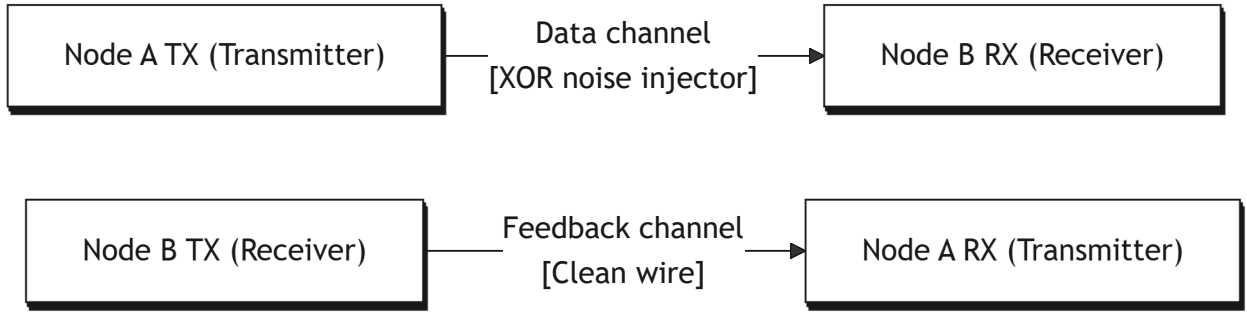


Figure 3.3: UART asymmetric channel topology in this section. The forward data channel passes through the XOR noise injector, while the reverse feedback channel remains a clean direct wire.

## 3.2 Common Node Components

Both legitimate nodes share an identical digital processing core. The computational substrate is an ATmega328P clocked at 16 MHz — an 8-bit AVR with 32 KB of Flash, 2 KB of SRAM, and 1 KB of EEPROM. The 2 KB SRAM budget is the dominant constraint of the entire firmware design: it prohibits dynamic allocation (`new`, `malloc()`), imposes a strict ceiling on stack depth, and mandates that all packet buffers, nonce material, and FSM state variables be declared statically at compile time. Every data structure in the C2P-ARQ stack — `HelloPacket`, `DataPacket`, `s_sessionNonce`, the 64-byte CSPRNG keystream cache — is sized at compile time and occupies a fixed, statically-known location in the SRAM map.

Both nodes are equipped with an Aip31068 16×2 I2C LCD display connected via the hardware I2C bus (pins A4/A5, address 0x3E). The display is not a cosmetic decoration but a critical out-of-band diagnostic instrument. Because the I2C writes are performed from within FSM state-transition code — rather than in a dedicated blocking display

loop — the LCD reflects the current protocol state in real time without introducing latency into the main execution path. On Node A, the top row shows `PKT:<seqNum> RTY:<retryCount>` and the bottom row shows the active FSM state label (`SENDING`, `WAITING_ACK`, `RECONNECTING`, etc.). On Node B, the top row displays the FSM state and the bottom row shows `SEQ:<n> OK` or `SEQ:<n> MAC!` after each validated or rejected frame. This continuous, human-readable state display allows a complete visual verification of the C2P-ARQ session lifecycle — handshake, data transfer, retransmission storms, self-healing reconnection, and authenticated teardown — without attaching a logic analyser or serial monitor. In the context of academic validation, the LCD constitutes a synchronised, unambiguous ground-truth channel for protocol-state observation.

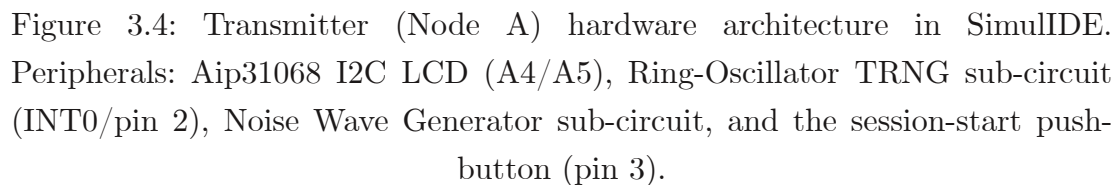
Both nodes additionally incorporate a hardware entropy sub-circuit for TRNG seeding. The architecture and entropy mixing strategy of this sub-circuit are examined in detail in [Section 4](#).

### 3.3 Node Asymmetry: Interrupt-Driven TX and PWM-Driven RX

Beyond the shared computing core, the two nodes differ in their peripheral complement, and these hardware differences are a direct physical expression of the Fat Server / Thin Client asymmetry described in [Section 2.2](#).

#### 3.3.1 Transmitter (Node A)

Node A is the session initiator and the sole source of protocol control. A momentary push-button connected to digital pin 3 (INT1) with `INPUT_PULLUP` conditioning triggers session establishment asynchronously via hardware interrupt. A `millis()`-based 50 ms debounce filter eliminates contact-bounce noise from the interrupt line, ensuring that a single physical button press generates exactly one FSM transition from `IDLE` to `SENDING_HELLO`. The transmitter carries no audio output peripheral; its entire peripheral surface consists of the I2C LCD, the entropy sub-circuit, and the interrupt-bearing button. The hardware is shown in Figure 3.4.



Node B is the execution endpoint and produces no protocol control signals beyond single-byte ACK/NACK frames. It carries no button and requires no interrupt-driven input. Instead, it is equipped with a piezoelectric speaker connected to PWM-capable pin 9 (Timer 1, OC1A), driven by the Arduino `tone()` / `noTone()` API — which configures the hardware timer to generate a square wave at the target frequency without software intervention, freeing the CPU to continue servicing incoming UART bytes during playback. Note playback is strictly non-blocking: `tone(pin, frequency)` is called once at note onset, the end-timestamp is recorded, and `noTone()` is called in the subsequent `loop()` iteration in which `(millis() - noteStartTime) >= durationMs` holds. No `delay()` is inserted anywhere in the playback sequence. The receiver hardware is shown in Figure 3.5.



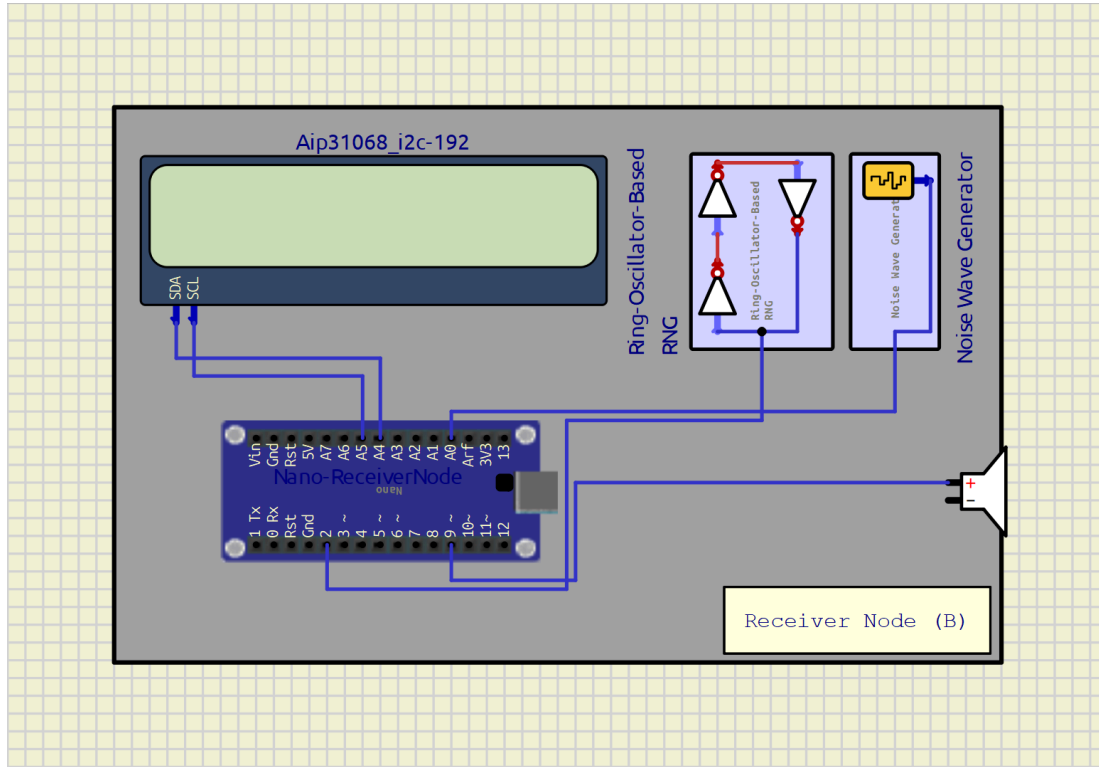


Figure 3.5: Receiver (Node B) hardware architecture in SimulIDE. Peripherals: Aip31068 I2C LCD (A4/A5), Ring-Oscillator TRNG sub-circuit (INT0/pin 2), Noise Wave Generator sub-circuit, and the piezoelectric speaker (PWM pin 9).

# 4 True Random Number Generation (TRNG) & Entropy Management

The cryptographic security of C2P-ARQ rests on a single, non-negotiable requirement: every session nonce must be drawn from a source of genuine physical randomness. This requirement is not a matter of implementation preference but a formal security constraint imposed by the AEAD construction itself. ChaCha20-Poly1305 is an IND-CPA-secure cipher under the assumption that no nonce is ever reused under the same key. Formally, for a 96-bit nonce space and a 256-bit key  $K$ , any two encryptions become insecure if the nonce is reused [21, 36, 37]:

$$E_{K,IV_1}(m_1), E_{K,IV_2}(m_2)$$

$$IV_1 = IV_2$$

In that case the XOR of the plaintexts becomes observable:

$$m_1 \oplus m_2$$

because ChaCha20's keystream is deterministic:

$$E_{K,N}(m) = m \oplus \text{ChaCha20}(K, N)$$

where the symbol

$$E_{K,N}(m)$$

denotes the encryption of message  $m$  under key  $K$  and nonce  $N$ , and  $\text{ChaCha20}(K, N)$  is the corresponding purely deterministic keystream block. A single nonce collision therefore catastrophically breaks both confidentiality and Poly1305 integrity — the MAC key, derived from the first block of the ChaCha20 keystream, is identical for both messages, enabling an attacker to forge arbitrary authenticated ciphertexts. This is not a theoretical edge case; any deterministic pseudo-random number generator (PRNG) with a predictable or reproducible state — including the Arduino `random()` function seeded from a fixed constant — makes nonce collisions across power cycles a certainty.

Software PRNGs are structurally insufficient as a standalone entropy source for this purpose. A linear-congruential generator or an xorshift PRNG produces a deterministic sequence parameterised entirely by its initial seed. On an embedded node that is power-cycled between sessions — a routine operational event — any seed derived from a compile-time constant, an uninitialized SRAM snapshot alone, or a fixed ADC reading will reproduce the same seed on each boot, and therefore the same first nonce. C2P-ARQ requires a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) whose seed incorporates genuine physical entropy — bit-level randomness sourced from unpredictable physical processes that are statistically independent of all prior state, all boot history, and all adversarial observation.

## 4.1 Hardware Entropy Sources

Physical randomness in SH2SC-EDT is harvested by `generateEntropyPool()`, a one-shot function invoked once during node initialisation (`setup()` on TX, `rx_setup()` on RX) that fills a 32-byte seed buffer by executing eight sequential word iterations. In each iteration, multiple physically independent sources are folded into a 32-bit accumulator via a rotate-XOR sponge; the resulting word is written as four little-endian bytes into the output buffer. The total blocking cost is therefore:

$$8 \times 2 \text{ ms} = 16 \text{ ms}$$

This is an acceptable one-time expenditure in the initialisation context, which carries no UART receive obligations. The transmitter node feeds seven sources into each word; the receiver node feeds six, the `micros()` human-timing term being absent because the receiver carries no button input. The hardware sub-circuit implementing Sources 1 and 5 is shown in Figure 4.1.

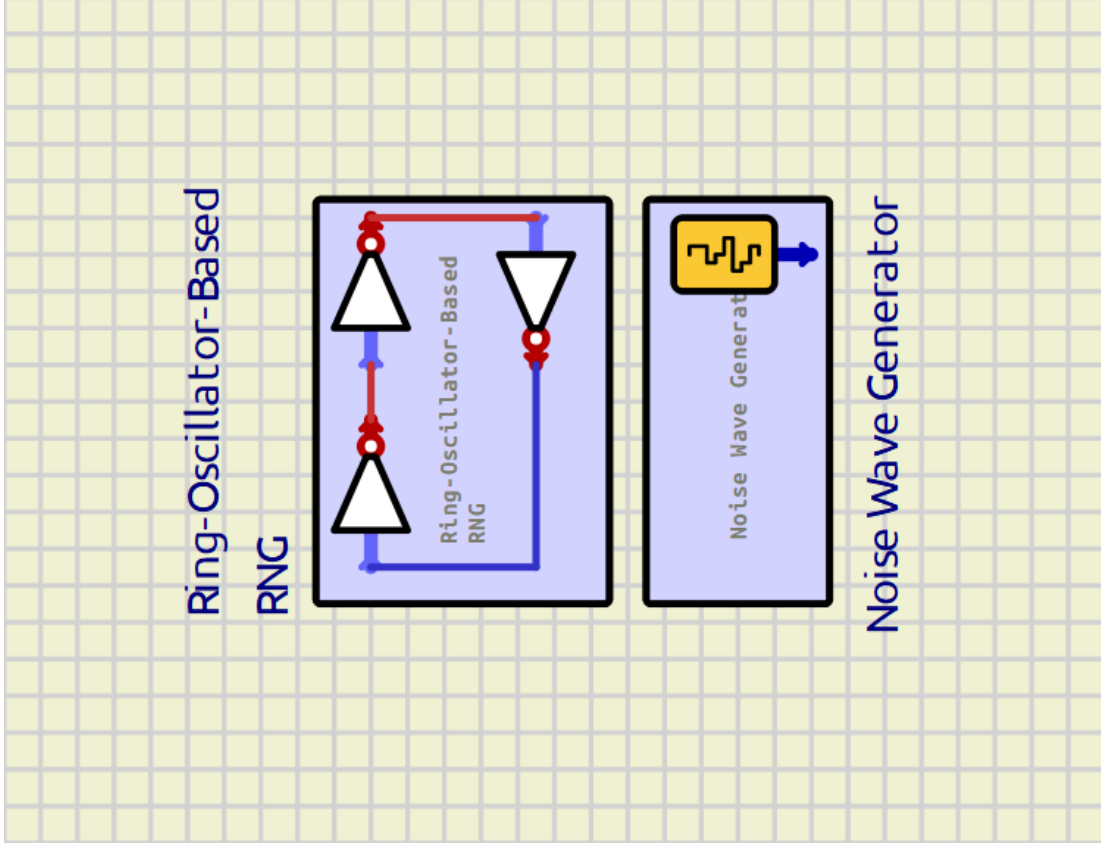


Figure 4.1: Hardware entropy sub-circuit featuring a Ring-Oscillator and Noise Wave Generator. The ring oscillator (three CMOS inverter stages in a feedback loop) drives INT0; the Noise Wave Generator injects frequency-modulated perturbations to augment phase jitter in the SimulIDE simulation context.

**Source 1 — ring-oscillator gate (INT0, pin 2, RISING).** At the start of each word iteration, `s_ringOscPulses` is zeroed and the ISR `onRingOscPulse()` is attached to INT0 on the RISING edge. The MCU then blocks for 2 ms via `delay(2)`, during which the hardware ring oscillator drives INT0; the ISR increments the pulse counter on every rising edge. At gate closure the interrupt is detached and the counter is folded into the pool. The instantaneous period of the  $k$ -th ring-oscillator cycle is expressed as:

$$T_k = T_0 + \Delta t_k$$

where  $T_0$  is the nominal, device-specific oscillation period and  $\Delta t_k$  is accumulated gate-stage phase jitter, distributed as a zero-mean Gaussian with variance  $\sigma_j^2$ :

$$\Delta t_k \sim \mathcal{N}(0, \sigma_j^2)$$

Because  $\Delta t_k$  is physical, the edge count per 2 ms window varies unpredictably across iterations. The Shannon entropy of the sampled count  $C$  satisfies [22, 23, 24]:

$$H(C) = - \sum_c P(C = c) \log_2 P(C = c)$$

Under nominal jitter conditions, empirical counts span 8–16 distinct values, yielding an estimated Shannon entropy per gate window in the range:

$$H(C) \approx 3\text{--}4$$

bits per gate window. In the SimulIDE simulation context the Noise Wave Generator sub-circuit injects frequency-modulated perturbations on pin 2 to compensate for the deterministic propagation delays of ideal gate models that would otherwise suppress physical jitter.

**Source 2 — uninitialised SRAM.** SRAM cells on the ATmega328P retain residual charge patterns from manufacturing variation and prior power cycles. The 64 bytes beginning at address `0x0100` are read without prior initialisation. Each of the 8 word iterations consumes a distinct, non-overlapping 8-byte slice — bytes at `0x0100 + wordIndex × 8` through `+7` — folding each byte individually through `mixEntropy()`. This source provides non-reproducible per-device entropy that depends on the physical cell-charge distribution of the specific silicon die. [25]

**Source 3 — on-die temperature sensor ADC.** The ATmega328P integrates a temperature-sensing diode accessible via ADC channel 8. Setting `ADMUX = 0xC8` (`_BV(REFS1) | _BV(REFS0) | _BV(MUX3)`) selects the internal 1.1 V bandgap reference and multiplexes the temperature diode output onto the ADC input. One conversion is discarded immediately after the reference switch to allow the sample-and-hold capacitor to settle. Eight successive conversions are then performed; the LSB of each `ADCL` reading is extracted and shifted into an 8-bit accumulator, which is folded into the pool. The temperature sensor readout is subject to quantisation noise and thermal fluctuations that are not synchronised to the CPU clock; the LSBs are non-deterministic under normal operating conditions, contributing approximately:

$$H_{\text{LSB}} \lesssim 1 \text{ bit per sample}$$

This corresponds to up to 8 bits per word in aggregate.

**Source 4 — TCNT1 timer capture.** The 16-bit Timer/Counter 1 runs free from MCU reset. Its value at each word-iteration boundary is determined by the cumulative execution time of all preceding sources in that iteration — ring-oscillator gate duration, ADC conversion latencies, SRAM access timing, and interrupt service overhead — none

of which are synchronised to any observable external signal. TCNT1 is sampled once per iteration and its 16-bit value folded into the pool, contributing phase diversity orthogonal to the other sources.

**Source 5 — A0 noise generator readout.** Pin A0 is connected in the SimulIDE schematic to the output of the Noise Wave Generator sub-circuit, which produces a frequency-modulated waveform whose instantaneous amplitude at the moment of each `analogRead(A0)` call is pseudo-random within the analogue input range. Eight `analogRead(A0)` calls are performed per word; the LSB of each 10-bit reading is extracted and shifted into an 8-bit accumulator, which is folded into the pool. On real ATmega328P hardware this source would harvest thermal noise or an analogue environmental signal connected to A0.

**Source 6 — `micros()` human-timing jitter (TX node only).** The moment `generateEntropyPool()` is invoked on the transmitter is determined by the user’s button press, which is unpredictable to sub-microsecond precision. `micros()` is sampled once per word iteration and folded into the pool. This source is structurally absent on the receiver node — the RX variant of `generateEntropyPool()` explicitly notes its omission — giving RX six sources per word versus seven on TX.

**Source 7 — `Arduino random()` (obfuscation layer).** A single call to the Arduino linear-congruential PRNG is folded into each word as a final stage. This source contributes no physical entropy — its sequence is deterministic from the PRNG state — but raises the preimage complexity for an adversary with partial knowledge of the physical sources, functioning as an obfuscation layer over the preceding six terms.

The 32-byte seed produced by `generateEntropyPool()` is passed immediately to `initCSPRNG()` and scrubbed from the stack with `memset()` to prevent residual key material exposure. The raw entropy budget is dominated by the ring-oscillator contribution. Under nominal jitter conditions, the per-window entropy is on the order of:

$$H(C) \approx 3\text{--}4$$

bits per 2 ms gate window. Across 8 independent iterations, this yields:

$$H_{\text{ring}} \approx 24\text{--}32$$

bits. Uninitialised SRAM power-on bias, temperature ADC LSBs, TCNT1 phase capture, the A0 noise generator readout, and — on the transmitter — `micros()` human-timing jitter contribute additional structurally orthogonal entropy that is difficult to quantify precisely in isolation. Under the conservative assumption that only the ring-oscillator source contributes measurably, the total raw entropy budget is estimated at:

$$H_{\text{raw}} \approx 24\text{--}32$$

bits across the 32-byte seed.

## 4.2 The Cryptographic Entropy Blender

The raw bit contributions harvested from the seven physical sources described in [Section 4.1](#) cannot feed directly into the ChaCha20 engine as a key. Two structural deficiencies preclude it. First, every source introduces independent statistical bias: ring-oscillator edge counts cluster around a device-specific mean, ADC LSBs exhibit hardware-dependent 0/1 imbalance, and the `random()` term is wholly deterministic. Second, folding an arbitrary number of input bits into a 32-bit accumulator cannot raise that word’s entropy above 32 bits — the actual budget is bounded by the dominant physical source alone. `initCSPRNG()` resolves both deficiencies through a two-stage blending pipeline executed immediately after `generateEntropyPool()` returns.

### 4.2.1 Stage 1 — The Rotate-XOR Sponge

The first stage condenses the per-source bit contributions into a compact 32-byte raw seed using a rotate-XOR sponge applied to each of the eight 32-bit accumulator words. After each source contribution, the accumulator is updated by the following primitive [28, 29, 30, 31]:

$$\text{pool}' = (\text{pool} \lll 1) \oplus \text{bits}$$

where `pool'` is the updated 32-bit accumulator,  $\lll 1$  denotes a left-rotation of the accumulator by one bit position (implemented as `((pool << 1u) | (pool >> 31u)) ^ bits` in C++), and `bits` is the contribution folded in from the current entropy source. Applied iteratively — once per source per word, across seven sources and eight word iterations — this primitive prevents a pathological sequence of identical `bits` values from cancelling to zero, an unavoidable defect of plain bitwise-XOR accumulation, and diffuses the statistical bias of each individual source across all 32 bit positions of the accumulator. The eight resulting words are concatenated to form the raw seed passed immediately to Stage 2:

$$K_{\text{hw}} = w_0 \| w_1 \| w_2 \| w_3 \| w_4 \| w_5 \| w_6 \| w_7$$

where each

$$w_i \in \{0, 1\}^{32}$$

is the final accumulator value after all source contributions for iteration  $i$  have been folded in, and  $\parallel$  denotes concatenation. After Stage 2 consumes this seed, the 32-byte buffer is scrubbed from the stack with `memset()` to prevent residual key material exposure.

### 4.2.2 Stage 2 — ChaCha20 Keystream Expansion

The second stage resolves the remaining deficiency:  $K_{\text{hw}}$  remains a non-uniform random variable whose effective entropy is orders of magnitude below the 256-bit security parameter that a full-strength ChaCha20 key nominally requires.

$$K_{\text{hw}} \in \{0, 1\}^{256}$$

Under conservative assumptions, this effective entropy is approximately:

$$H_{\text{raw}} \approx 24\text{--}32$$

bits.

Using this seed directly as cryptographic key material would allow an adversary to enumerate the key space far faster than  $2^{256}$  operations. `initCSPRNG()` addresses this through one-way cryptographic conditioning:  $K_{\text{hw}}$  is installed as the 256-bit ChaCha20 key via `s_chacha.setKey(hardwareSeed, 32)`, and a fixed 8-byte zero IV is set via `s_chacha.setIV(zeroIV, 8)` [26, 27, 32, 33]:

$$IV = 0^{64}$$

The Rhys Weatherley **ChaCha** library operates in the original djb-ChaCha configuration with a 64-bit nonce — distinct from the 12-byte IETF nonce used by the C2P-ARQ packet cipher. The zero IV is safe because  $K_{\text{hw}}$  is freshly derived from physical entropy on every power cycle; the resulting pair is therefore unique across sessions with overwhelming probability:

$$(K_{\text{hw}}, IV)$$

`s_fillKeystreamBlock()` then produces the first 64-byte keystream block by encrypting a zero-filled buffer in-place:

$$B_0 = \text{ChaCha20}(K_{\text{hw}}, 0^{64}, c_0)$$



where the all-zero plaintext XORed with the ChaCha20 keystream yields the keystream directly —  $\text{output} = 0x00\dots0 \text{ XOR } \text{keystream} = \text{keystream}$  — without requiring a second 64-byte allocation. The entire 64-byte block  $B_0$  is stored in `s_keystreamBuf` and constitutes the initial CSPRNG cache. Subsequent calls to `getSecureRandom32()` consume 4 bytes of `s_keystreamBuf` per invocation; when the 64 bytes are exhausted, `s_fillKeystreamBlock()` is called again, producing block  $B_n$ :

$$B_n = \text{ChaCha20}(K_{\text{hw}}, 0^{64}, c_0 + n)$$

where

$$c_0 + n$$

is ChaCha20’s internal 64-bit block counter, ensuring

$$B_n \neq B_m$$

for all

$$n \neq m$$

and yielding a non-repeating keystream for up to  $2^{64}$  blocks — capacity far beyond any plausible C2P-ARQ session lifetime.

## Cryptographic Properties of the Conditioning Step

### Bias Elimination

ChaCha20’s 20-round double-round ARX transformation satisfies the Strict Avalanche Criterion: complementing any single bit of  $K_{\text{hw}}$  changes every bit of  $B_0$  with probability  $\approx 0.5$ . The statistical non-uniformity of Stage 1 outputs — ring-oscillator count clustering, ADC LSB imbalance, and the wholly deterministic `random()` term — is thus mapped to a computationally uniform output distribution regardless of the input bias magnitude.

### Computational Indistinguishability

Under the standard pseudorandom function assumption for ChaCha20-20,  $B_n$  is computationally indistinguishable from a uniformly random 512-bit string for any probabilistic polynomial-time adversary. The effective security of `getSecureRandom32()` output is therefore  $\min(H_{\text{raw}}, 256)$  bits — bounded by the physical entropy budget entering Stage 1, not by the keystream expansion length.

## Forward Secrecy Across Power Cycles

Each power cycle derives a fresh  $K_{hw}$  via Stage 1; all prior CSPRNG outputs are computationally inaccessible from the new state. An adversary observing a current CSPRNG block  $B_n$  cannot recover any session nonce generated in a prior power cycle without inverting ChaCha20.

The cache refill completes in approximately 100  $\mu$ s at 16 MHz — one ChaCha20 block invocation over 64 zero bytes — and has no measurable impact on C2P-ARQ protocol timing.

## 4.3 Non-blocking CSPRNG Access at Runtime

The 16 ms blocking cost incurred by `generateEntropyPool()` — eight sequential 2 ms ring-oscillator gate windows with synchronous ADC and SRAM reads — is architecturally safe because of when it executes. Both `tx_setup()` and `rx_setup()` call `generateEntropyPool()` after `Serial.begin()` but before the first `loop()` invocation. At that point the UART hardware is initialised but no packet traffic can arrive: no C2P-ARQ session is open, no FSM state requires timeout supervision, and no UART byte is expected on the receive buffer. The 16 ms expenditure is therefore front-loaded entirely into the MCU’s initialisation phase and has no impact on any protocol timing guarantee.

After `initCSPRNG()` returns, the ring-oscillator ISR `onRingOscPulse()` is not re-attached and the ADC hardware is not placed in free-running mode. All physical entropy sources used during `generateEntropyPool()` are fully released: INT0 is unused, ADC channel 8 reverts to its saved `ADMUX` state, and A0 is available for any analogue use. There is no background entropy accumulation during active protocol operation. The CSPRNG from this point forward operates as a deterministic ChaCha20 counter-mode keystream seeded from the one-time boot harvest  $K_{hw}$ , as described in [Section 4.2](#).

The only entropy consumer during runtime is `sendHelloPacket()`, which calls `getSecureRandom32()` three times in succession to fill the 12-byte session nonce `s_sessionNonce`. This function is invoked exclusively from `TxState::SENDING_HELLO` — a brief transitional state that executes once per session SYN handshake and once per auto-reconnect attempt. The nonce derivation therefore occurs precisely when a new session key is needed and at no other time. The receiver node does not call `getSecureRandom32()` at any point during protocol operation; its CSPRNG is seeded at boot for potential future use but is not consumed by the C2P-ARQ receive path.

`getSecureRandom32()` is non-blocking under all runtime conditions. The normal execution path reads four bytes directly from `s_keystreamBuf` — an array access and a counter increment, completing in a handful of AVR clock cycles. A cache refill is triggered automatically when `s_keystreamUsed + 4 > 64`, requiring one ChaCha20 block invocation ( $\sim 100\ \mu\text{s}$  at 16 MHz). Because `sendHelloPacket()` draws only 12 bytes (three calls) per session and blocks of 64 bytes are generated on each refill, the refill cost is amortised to approximately  $4.7\ \mu\text{s}$  per `getSecureRandom32()` call — negligible against the 9600 bps UART frame time of 1.04 ms per byte.

The architectural coupling between the entropy domain and the protocol domain is therefore a strict one-way dependency: physical entropy flows from hardware sources at boot into  $K_{\text{hw}}$ , from  $K_{\text{hw}}$  into the ChaCha20 engine, and from the engine into session nonces on demand. No protocol event feeds back into the CSPRNG state; no FSM state inspects or modifies the entropy sub-system. This uni-directional structure provides a formal domain separation: a correctness proof or security argument about the C2P-ARQ transport layer may treat `getSecureRandom32()` as an ideal random oracle, because its outputs are computationally indistinguishable from uniformly random values under the ChaCha20 pseudorandom function assumption regardless of the calling FSM state.

Forward secrecy across sessions is maintained not by entropy replenishment but by nonce erasure. After every session close — whether via a successful FIN handshake or via `suspendSession()` — `s_sessionNonce` is overwritten with zeros (`memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`). A subsequent session uses the next block of the ChaCha20 keystream for its nonce, which is computationally independent of all prior blocks. An adversary who captures  $B_n$  cannot recover any nonce derived from:

$$B_0, \dots, B_{n-1}$$

without inverting ChaCha20, satisfying backward secrecy. An adversary who captures the current CSPRNG state cannot predict any future nonce if the ChaCha20 counter has advanced — a property guaranteed by the counter-mode construction.

# 5 The C2P-ARQ Protocol Specification

C2P-ARQ is a custom stateful session-layer protocol that simultaneously provides reliable ordered delivery over an unreliable, bit-corrupting physical channel; per-packet authenticated encryption that prevents replay, injection, and silent tampering; a three-phase session lifecycle that maintains consistent cryptographic state between transmitter and receiver; and automatic session recovery that resumes payload delivery after link failure without operator intervention. The entire protocol stack executes within the 2 KB SRAM budget of a 16 MHz ATmega328P microcontroller, a constraint that prohibits dynamic memory allocation, heap-resident session tables, and retransmit queues of unbounded depth. All protocol state is declared statically at compile time with sizes known to the compiler.

The underlying transport model is Stop-and-Wait ARQ: the transmitter sends exactly one frame and then blocks all progress in a `WAITING_*` FSM state until it receives an explicit one-byte acknowledgement or exhausts the per-state retry limit (`MAX_RETRIES` = 50, `ACK_TIMEOUT_MS` = 50 ms). This model is appropriate for the 9600 bps UART channel: at one byte per approximately 1.04 ms, the largest on-wire frame — 17 bytes including preamble — occupies roughly 18 ms of channel time, leaving the 50 ms ACK window with a comfortable round-trip margin. The worst-case retry storm duration is therefore bounded at [49, 50, 51, 52]:

$$50 \times 50 = 2500 \text{ ms}$$

This value is deliberately chosen to fall below the 3000 ms grace period of the receiver’s Smart Dynamic Watchdog ([Section 7.2](#)).

The asymmetric channel model is a defining structural property of C2P-ARQ, as shown in Figure 5.1. The forward data channel, routed from Node A TX to Node B RX, passes through the XOR noise injector and is subject to arbitrary bit-flip bursts; the feedback channel, running from Node B TX to Node A RX, is a direct, noiseless wire carrying only `ACK_BYTE` (0x06) and `NACK_BYTE` (0x15). Reliability decisions — whether to accept a frame, retransmit, or suspend the session — are therefore made on clean signal regardless of the noise level on the forward path. The subsections that follow specify the three-phase lifecycle ([Section 5.1](#)), the on-wire packet formats ([Section 5.2](#)), and the acknowledgement and retry-limit mechanism ([Section 5.3](#)).

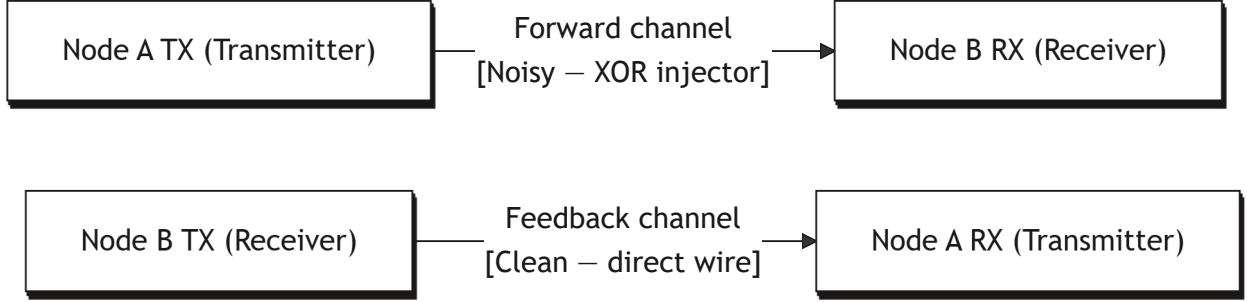


Figure 5.1: C2P-ARQ asymmetric UART channel topology in the SimulIDE testbed. The data channel is subject to XOR bit-flip corruption; the feedback channel carrying ACK\_BYTE and NACK\_BYTE is a direct, unattacked wire.

## 5.1 Session Lifecycle: Handshake (SYN), Data Transfer (DAT), Teardown (FIN)

The C2P-ARQ session lifecycle comprises three strictly sequenced phases. The SYN handshake establishes shared cryptographic state by delivering the CSPRNG-generated 96-bit session nonce from transmitter to receiver; this nonce determines every per-packet IV for the duration of the session and is therefore the sole state that both endpoints must agree on before data transfer can begin. The DAT transfer phase delivers the ordered payload sequence under Stop-and-Wait ARQ, with each frame independently authenticated by a Poly1305 MAC whose tag covers both the ciphertext and the 3-byte plaintext header. The FIN teardown authenticates the session boundary and triggers immediate, symmetric nonce erasure on both nodes, establishing forward secrecy for all ciphertext captured during the session.

### 5.1.1 SYN Handshake

The SYN phase synchronises cryptographic state between the two nodes. Its single deliverable is the 12-byte session nonce, generated on the transmitter and replicated on the receiver.

$$N_s \in \{0, 1\}^{96}$$

In `TxState::IDLE`, the transmitter polls digital pin 3 (`INPUT_PULLUP`) with a 50 ms `millis()` debounce window. On the first confirmed button-press edge, the FSM resets `melodyIndex = 0` and `seqNum = 0`, then transitions to `TxState::SENDING_HELLO`. In `SENDING_HELLO`, `sendHelloPacket()` calls `getSecureRandom32()` three times in succession

to fill `s_sessionNonce[0..11]` with three independent 32-bit CSPRNG words; it then calls `drainRxFifo()` — consuming and discarding all bytes currently waiting in the 64-byte hardware UART receive FIFO — before writing the complete 15-byte frame to `Serial`: two preamble bytes (0xAA, 0x55), the `FLAG_SYN` flags byte (0x01), and the twelve nonce bytes. Immediately after transmission the FSM transitions to `TxState::WAITING_HELLO_ACK` and records `ackWaitStart = millis()`.

The `drainRxFifo()` call before every transmission is a defensive protocol invariant: stale bytes accumulated in the hardware FIFO during the preceding idle or retry interval could otherwise be misinterpreted as a response to the freshly transmitted frame, triggering a false retry storm or, in the worst case, a spurious `ACK_BYTE` that advances the FSM without a genuine receiver response.

On the receiver, `processReceivedByte()` advances the four-state byte-level parser, illustrated in Figure 5.2, one byte at a time, using the preamble pair 0xAA 0x55 as the synchronisation gate. When the parser has accumulated all 13 HelloPacket body bytes into `rx_buffer` and `rx_buffer[0] & FLAG_SYN` holds, it dispatches `processHelloBody()`. This function copies `rx_buffer[1..12]` into `s_sessionNonce`, calls `s_cipher.setKey(MASTER_PSK, 32)` to pre-load the 256-bit PSK into the AEAD state, sets `s_sessionActive = true`, writes `ACK_BYTE` (0x06) to `Serial`, and initialises the Dynamic Smart Watchdog with `current_timeout_limit = 5000 ms` and `last_valid_packet_time = millis()`.

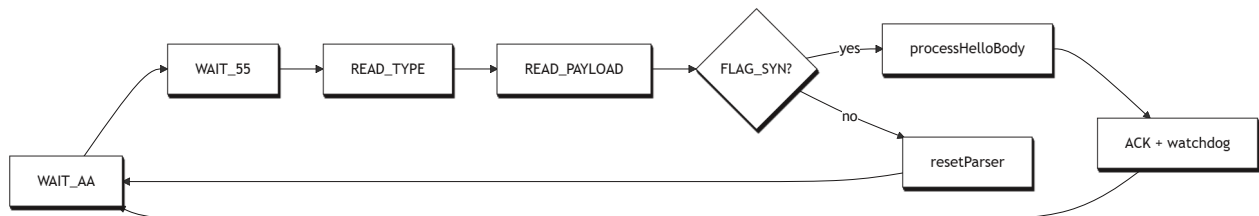


Figure 5.2: SYN handshake admission path for HelloPacket frames. Only `FLAG_SYN` proceeds to `processHelloBody()`, which installs the session nonce, emits `ACK_BYTE`, and arms the Dynamic Smart Watchdog.

On receiving `ACK_BYTE` in `TxState::WAITING_HELLO_ACK`, the transmitter transitions to `TxState::SENDING`. If no `ACK_BYTE` arrives within `ACK_TIMEOUT_MS = 50 ms`, the FSM returns to `TxState::SENDING_HELLO` and issues a new `sendHelloPacket()` call — which generates a fresh CSPRNG nonce on every invocation, ensuring that the nonce window advances monotonically regardless of retry count. After `MAX_RETRIES = 50` consecutive failures, `suspendSession()` is invoked and the FSM enters `TxState::RECONNECTING` with `melodyIndex` preserved.

The session nonce  $N_s$  is transmitted in cleartext because it is not secret: it functions as a public IV whose sole purpose is to guarantee semantic security — encrypting the same note index twice under the same session produces distinct ciphertext. The confidentiality of the session rests entirely on `MASTER_PSK`, which is compiled into both firmware images and never transmitted over any channel.

### 5.1.2 DAT Transfer Phase

The DAT phase implements Stop-and-Wait ARQ with full ChaCha20-Poly1305 AEAD per packet. Its central invariant is the `seqNum` binding: every packet that reaches the receiver carries a unique `seqNum`, and the per-packet nonce is derived deterministically from that `seqNum` — ensuring that the AEAD context is never reused across any two successfully acknowledged frames within a single session.

#### Per-Packet Nonce Derivation

For the packet with sequence number  $i$  (where  $i$  is the current `seqNum`), the per-packet IV is derived from a local copy of  $N_s$  with bytes at positions 10 and 11 XORed with the high and low bytes of  $i$ , respectively:

$$IV_i[j] = \begin{cases} N_s[j] & 0 \leq j \leq 9 \\ N_s[10] \oplus \lfloor i / 256 \rfloor & j = 10 \\ N_s[11] \oplus (i \bmod 256) & j = 11 \end{cases}$$

The original `s_sessionNonce` is never mutated; the XOR is applied exclusively to a stack-allocated copy `packetNonce[HELLO_NONCE_SIZE]`. The mapping is injective over the 16-bit domain:

$$i \mapsto IV_i$$

$$i \in [0, 65535]$$

If

$$i \neq i'$$

then:

$$((i \bmod 256) \neq (i' \bmod 256)) \vee (\lfloor i/256 \rfloor \neq \lfloor i'/256 \rfloor)$$

and therefore:

$$IV_i \neq IV_{i'}$$

All 65,536 sequence-number values therefore produce distinct per-packet nonces within a single session.

## AEAD Pipeline

In `TxState::SENDING`, `sendPacket()` executes the following pipeline: (1) `s_cipher.clear()`, `s_cipher.setKey(MASTER_PSK, 32)`, `s_cipher.setIV(packetNonce, 12)`; (2) `s_cipher.addAuthData({FLAG_DAT, seq_lo, seq_hi}, 3)`, which hashes the three-byte plaintext header into the Poly1305 accumulator without encrypting it; (3) `s_cipher.encrypt(plaintext, ciphertext, 4)`, encrypting the four-byte payload `{note_index, duration_ms}`; (4) `s_cipher.computeTag(fullMac, 16)`, of which only the first 8 bytes (`TRUNCATED_MAC_SIZE = 8`) are appended to the `DataPacket.mac` field and transmitted. The symmetric pipeline executes on the receiver inside `authenticateAndPlay()`, with `decrypt()` followed by `computeTag()` and a constant-time `memcmp` against the received 8-byte tag; `startNote()` is invoked only after `memcmp` returns zero. Binding the flag byte and both bytes of `seq_num` to the AAD means that any single-bit noise burst affecting either field produces a guaranteed MAC failure regardless of the ciphertext content. The full wire format of `DataPacket` is specified in [Section 5.2](#).

## Acknowledgement and seqNum Advancement

After each transmission the FSM waits in `TxState::WAITING_ACK`. On `ACK_BYTE`, the FSM transitions to `TxState::WAIT_BETWEEN_NOTES`, where it waits for `pendingNoteDuration` milliseconds before executing `melodyIndex++`; `seqNum++`. This is the only code path in the firmware where `seqNum` is incremented — it never advances on NACK receipt or ACK timeout. If `melodyIndex` has exhausted the melody table after the increment, the FSM transitions to `TxState::SENDING_FIN`; otherwise it returns to `TxState::SENDING` for the next note.

On NACK or ACK timeout, the FSM retransmits the snapshot (`pendingNoteIndex`, `pendingNoteDuration`) with the same `seqNum`. This idempotency is sound because the same pair  $(N_s, i)$  always produces identical ciphertext: the receiver, on receiving the retransmission, runs the same AEAD pipeline after a prior MAC failure and — if the retransmitted frame is uncorrupted — obtains a valid `memcmp` result and emits `ACK_BYTE`. After `MAX_RETRIES = 50` consecutive failures, `suspendSession()` is invoked, zeroing `s_sessionNonce`, preserving `melodyIndex`, and entering `TxState::RECONNECTING`.



### 5.1.3 FIN Teardown

When `TxState::WAIT_BETWEEN_NOTES` detects that all notes have been acknowledged, the FSM transitions to `TxState::SENDING_FIN`. The function `sendFinPacket()` constructs a `DataPacket` with `flags = FLAG_FIN (0x04)` and an all-zero plaintext payload `{0, 0}`, then executes the full AEAD pipeline with the current `seqNum` and the AAD `{FLAG_FIN, seq_lo, seq_hi}`. The resulting authenticated zero-payload frame constitutes a cryptographically signed session-close signal: any noise-induced `0x04` byte pattern in the data stream will fail MAC verification on the receiver and elicit NACK, leaving the session open for FIN retransmission.

On the receiver, `processFinPacket()` runs the identical AEAD pipeline. A structural requirement of the ChaCha20-Poly1305 construction is that `decrypt()` must be called before `computeTag()` even when the plaintext is discarded: the Poly1305 accumulator ingests ciphertext bytes as a side effect of `decrypt()`; omitting the call leaves the accumulator in an incorrect intermediate state and produces a false MAC mismatch for a legitimate FIN. The `processFinPacket()` implementation satisfies this requirement by decrypting the four zero bytes into a local `discardBuf[DATA_PAYLOAD_SIZE]` array before calling `computeTag()`.

On MAC success, the receiver (1) writes `ACK_BYTE` to `Serial`, (2) calls `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`, (3) sets `s_sessionActive = false`, and (4) calls `resetParser()`. On receiving `ACK_BYTE` in `TxState::WAITING_FIN_ACK`, the transmitter (1) calls `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)` symmetrically, (2) resets `melodyIndex = 0` and `seqNum = 0`, and (3) transitions to `TxState::IDLE`. After teardown, the only persistent secret material in the system is `MASTER_PSK` — compiled into both firmware images and never transmitted, logged, or displayed on any output channel. Neither node retains the session nonce, and no ciphertext block captured from the completed session can be reopened without knowledge of `MASTER_PSK`. The formal security and forward-secrecy properties of nonce erasure are analysed in [Section 6.3](#).

### 5.1.4 Complete Session Sequence Diagram

The full C2P-ARQ session lifecycle is shown in Figure 5.3. The SYN handshake (Phase 1) synchronises the session nonce; the DAT transfer phase (Phase 2) exhibits both the normal ACK path and a retransmission cycle triggered by a MAC failure; the FIN teardown (Phase 3) shows the authenticated close and symmetric nonce erasure. Note that `seqNum` advances exclusively in `TxState::WAIT_BETWEEN_NOTES` — the ACK-success path — and is never incremented on NACK receipt or ACK timeout.

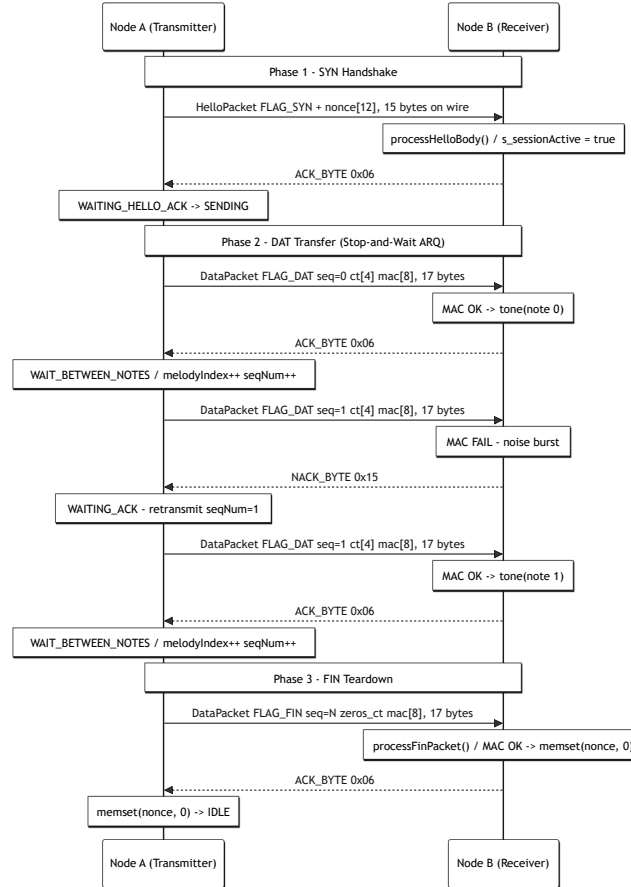


Figure 5.3: Complete C2P-ARQ session lifecycle sequence across the SYN handshake, Stop-and-Wait ARQ data transfer with retransmission, and FIN teardown with symmetric nonce erasure on both nodes.

## 5.2 Wire Format: HelloPacket and DataPacket Structures

All C2P-ARQ frames share a two-byte synchronisation preamble — `SYNC_BYTE_1 = 0xAA` followed by `SYNC_BYTE_2 = 0x55` — prepended to every packet on the forward data channel. The preamble is not a member field of either packet struct; the transmitter emits it via two explicit `Serial.write()` calls immediately before writing the packed struct body, and the receiver’s byte-level parser uses the `WAIT_AA` — `WAIT_55` state pair, as shown in Figure 5.2, to regain frame-boundary alignment after any noise-induced desynchronisation. The use of two distinct non-complementary bytes — rather than a single repeated byte — reduces the false-lock rate: the 16-bit pattern `0xAA55` cannot arise from a single bit inversion of either `0xAAAA` or `0x5555`, preventing isolated noise pulses from producing spurious frame-boundary locks.

Both packet types are declared with `#pragma pack(push, 1) / #pragma pack(pop)` in both firmware images. On AVR, the natural alignment of `uint8_t` and `uint16_t` fields already produces byte-aligned layout with no compiler-inserted padding, but the pragma is retained as an architecture-independent guarantee: the in-memory C struct layout is exactly the on-wire byte sequence, making the packed struct body directly castable to and from the hardware UART buffer without any explicit serialisation step.

### 5.2.1 AAD: The Authenticated Plaintext Header

The first three bytes of every `DataPacket` body — the `flags` byte (1 byte) and the little-endian `seq_num` (2 bytes) — are transmitted in cleartext and constitute the Additional Authenticated Data (AAD) of the ChaCha20-Poly1305 AEAD construction. The Poly1305 tag  $\tau$  is computed over the concatenation of the AAD, the ciphertext, their respective 16-byte-aligned padding blocks, and their encoded lengths — the standard IETF ChaCha20-Poly1305 construction:

$$\begin{aligned} \tau = & \text{Poly1305}_{K_{\text{MAC}}}(\text{AAD} \parallel \text{pad}_{16}(\text{AAD}) \parallel \text{ciphertext} \\ & \parallel \text{pad}_{16}(\text{ciphertext}) \parallel \text{len}_{64}(\text{AAD}) \parallel \text{len}_{64}(\text{ciphertext})) \end{aligned}$$

where  $K_{\text{MAC}}$  is the 32-byte one-time key derived from block 0 of the ChaCha20 keystream under the per-packet nonce  $IV_i$ . Because `flags` and `seq_num` are bound into the Poly1305 computation, any single-bit corruption of either field — whether from the noise injector or from an active adversary attempting frame reclassification or reordering — invalidates  $\tau$  with probability  $1 - 2^{-64}$ , causing the constant-time `memcmp` check to fail and triggering `NACK_BYTE` and retransmission. Neither field requires encryption: their integrity is enforced by the MAC rather than by confidentiality, and neither carries secret content.

The `HelloPacket` carries no AAD and uses no AEAD construction. Its `nonce` field is transmitted in cleartext because the session IV is a public parameter whose sole role is to ensure semantic security across sessions — adversarial knowledge of the nonce provides no advantage in recovering `MASTER_PSK`.

### 5.2.2 HelloPacket — 15 Bytes on Wire

The `HelloPacket` struct is a packed 13-byte body, preceded by the 2-byte preamble, for a total of 15 bytes on the wire. It contains two fields. The `flags` byte carries `FLAG_SYN = 0x01` and acts as the frame-type discriminator; the receiver matches it with the bitwise-AND predicate `rx_buffer[0] & FLAG_SYN` rather than strict equality, preserving tolerance

for future multi-role flag combinations without parser modification. The `nonce[HELLO_NONCE_SIZE]` array carries the 12-byte (96-bit) CSPRNG session IV generated on the transmitter by three successive `getSecureRandom32()` calls, one call per 32-bit word, filling the full 12-byte array. The complete byte-level wire layout is illustrated in Figure 5.4.

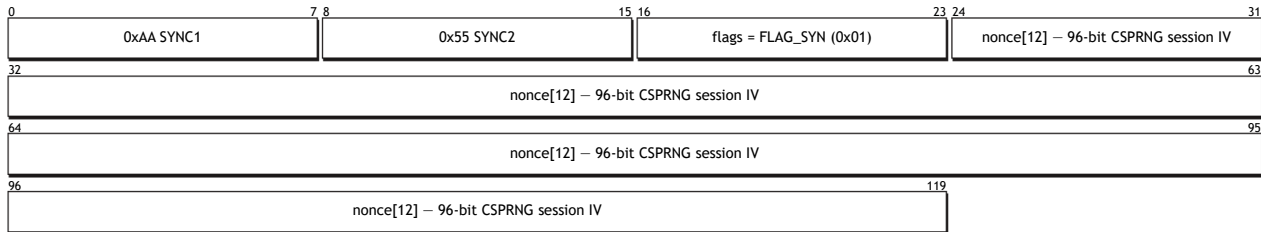


Figure 5.4: On-wire byte layout of HelloPacket in the C2P-ARQ frame format. The 2-byte preamble is emitted separately, followed by the 13-byte packed body carrying FLAG\_SYN and nonce[12].

The `nonce[12]` field is regenerated on every `sendHelloPacket()` invocation, including every reconnect attempt in `TxState::RECONNECTING`. Retransmitted SYN frames therefore carry a fresh nonce rather than replaying the prior one, advancing the CSPRNG output monotonically across all retry cycles and ensuring that `melodyIndex` preserved through `suspendSession()` is always paired with a freshly derived session key when the link is re-established.

### 5.2.3 DataPacket — 17 Bytes on Wire

The `DataPacket` struct is a packed 15-byte body, preceded by the 2-byte preamble, for a total of 17 bytes on the wire. It is used for both `FLAG_DAT = 0x02` (data transfer) and `FLAG_FIN = 0x04` (session teardown); the AEAD pipeline is identical in both cases. The struct contains four fields.

The `flags` field (1 byte) identifies the frame role and forms AAD byte 0. The `seq_num` field (2 bytes, `uint16_t`, little-endian) is the 16-bit packet sequence number and forms AAD bytes 1–2; it also serves as the input to the per-packet nonce derivation described in [Section 5.1](#). Together, `flags` and `seq_num` constitute the 3-byte AAD region that is cryptographically bound to the ciphertext by the Poly1305 tag. The `payload` field (4 bytes, `uint16_t[2]`) holds the ChaCha20 stream-cipher encryption of the `{note_index, duration_ms}` pair, both encrypted atomically in a single `s_cipher.encrypt()` call of length `DATA_PAYLOAD_SIZE = 4`. The `mac` field (8 bytes, `uint8_t[TRUNCATED_MAC_SIZE]`)

ing 8 bytes are computed internally by `s_cipher.computeTag()` but not transmitted, yielding a per-frame MAC forgery probability of  $2^{-64}$ . The complete byte-level wire layout is illustrated in Figure 5.5.

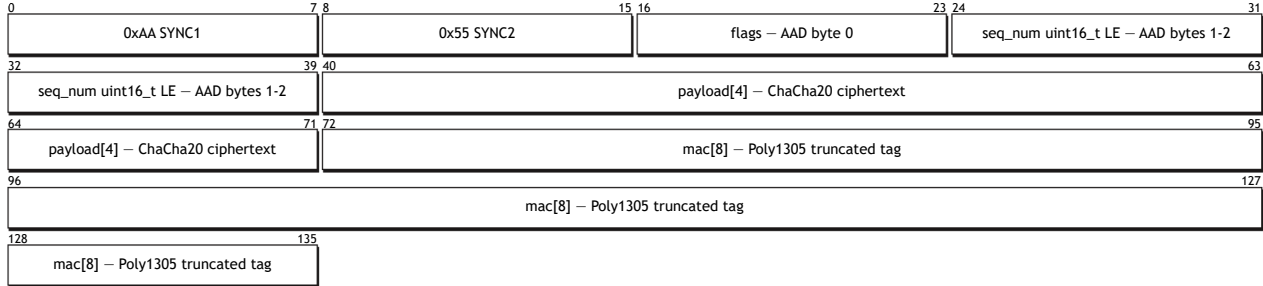


Figure 5.5: On-wire byte layout of DataPacket in the C2P-ARQ frame format. The 3-byte AAD region (flags + seq\_num) is authenticated by the Poly1305 tag but not encrypted, and the 4-byte payload is ChaCha20 ciphertext.

For `FLAG_FIN` frames, `payload` holds the ChaCha20 encryption of the zero plaintext `{0, 0}` — two `uint16_t` values equal to zero. On the receiver, `processFinPacket()` decrypts the four ciphertext bytes into a local `discardBuf[DATA_PAYLOAD_SIZE]` array; the decrypted values are discarded, but the `s_cipher.decrypt()` call is architecturally mandatory because the Poly1305 accumulator ingests ciphertext bytes as a side effect of decryption. Omitting this call before `s_cipher.computeTag()` leaves the accumulator in an incorrect intermediate state and produces a false MAC mismatch for a cryptographically valid FIN frame — a `discardBuf` pattern that any future port or extension of the receiver firmware must replicate exactly.

## 5.3 Acknowledgement Mechanism and Runaway Retry Prevention

### 5.3.1 The ARQ Feedback Loop

The reliability guarantee of C2P-ARQ is founded on a two-byte feedback vocabulary defined in `protocol.h`. The byte `ACK_BYTE = 0x06` signals that the receiver’s ChaCha20-Poly1305 MAC verification succeeded: the ciphertext arrived without corruption, the Poly1305 tag matched the independently computed expected value, and the decoded note has been enqueued for playback. The byte `NACK_BYTE = 0x15` signals the complementary outcome: the receiver computed a Poly1305 tag that did not match the received 8-byte

truncated MAC, and the payload was discarded in its entirety without acting on any field, including the `noteIndex` bound-check. The values `0x06` and `0x15` are separated by a Hamming distance of three bits, reducing the probability that a single noise-injected bit on the nominally clean feedback channel mutates one value into the other.

```

1 // TransmitterNode/protocol.h (shared C2P-ARQ constants on TX and RX)
2 const uint8_t ACK_BYTE = 0x06;
3 const uint8_t NACK_BYTE = 0x15;
4
5 const uint32_t ACK_TIMEOUT_MS = 50UL;
6 const uint8_t MAX_RETRIES = 50u;

```

Listing 1: ACK/NACK control bytes and ARQ retry limits

Here, `ACK_BYTE/NACK_BYTE` are the feedback control bytes, while `ACK_TIMEOUT_MS` and `MAX_RETRIES` define the Stop-and-Wait ARQ retry policy.

Within `TxState::WAITING_ACK`, `tx_loop()` invokes `Serial.available()` on every iteration. If a byte is present, it is consumed and tested against both control values. An `ACK_BYTE` receipt resets `retryCount` to zero, records `noteWaitStart = millis()`, and transitions the FSM to `TxState::WAIT_BETWEEN_NOTES`. Any byte that is neither `ACK_BYTE` nor `NACK_BYTE` is silently discarded; the transmitter remains in `WAITING_ACK` and the ACK timeout window continues to count down. A `NACK_BYTE` receipt signals MAC failure and mandates retransmission of the same packet with the same `seqNum` and the same pre-snapshotted payload (`pendingNoteIndex`, `pendingNoteDuration`). This invariant is architecturally non-negotiable: the per-packet nonce derivation on the receiver XORs `seqNum` into the base session nonce identically to the transmitter, so the receiver expects the same AEAD IV on the next retransmission attempt. Advancing `seqNum` on a NACK would produce an IV mismatch and guarantee a cascade of MAC failures on all subsequent packets. The sequence number advances only in `TxState::WAIT_BETWEEN_NOTES`, simultaneously with `melodyIndex++`, and at no other point in the FSM lifecycle.

### 5.3.2 Timeout Condition and Retry Increment

The zero-byte-received branch of `WAITING_ACK` implements a non-blocking ACK timeout using a `millis()` delta comparison:

$$(t_{\text{now}} - t_{\text{send}}) \geq \text{ACK\_TIMEOUT\_MS}$$

$$t_{\text{now}} = \text{millis}(), \quad t_{\text{send}} = \text{ackWaitStart}$$

with `ackWaitStart` being the `millis()` snapshot taken immediately after `formAndSendPacket()` returns. When the condition holds, the retry counter  $R$  is incremented by one, and — provided the runaway-prevention guard passes — the same packet is retransmitted and `ackWaitStart` is reset to restart the window.

Both the NACK path and the timeout path increment  $R$ , but they differ in one defensive detail. On the NACK path a `delay(10)` call is inserted before retransmission. This brief pause is a targeted defensive invariant: a NACK implies the receiver’s parser has just emitted a response byte on the feedback channel, which may be followed microseconds later by residual noise pulses still propagating through the UART line. The 10 ms drain window ensures that any such trailing bytes settle before the retransmitted packet arrives, preventing a stale byte from being misread as the ACK for the retransmission. On the timeout path no drain pause is inserted — no feedback byte was received, so the feedback FIFO is already empty and retransmission proceeds immediately, preserving as much of the 50 ms window as possible for the next response.

### 5.3.3 Runaway Retry Prevention and the Self-Healing Suspension

Without a hard upper bound on  $R$ , a sustained jamming event would trap the transmitter in `WAITING_ACK` indefinitely: every retransmission would be corrupted, every ACK would be lost, and neither endpoint could exit its current state through any channel event reachable under continuous interference. This constitutes a protocol-layer deadlock from which no graceful recovery is possible without explicit out-of-band support. C2P-ARQ eliminates this deadlock by enforcing the following precondition before every retransmission:

$$R < MAX\_RETRIES$$

When the guard fails:

$$R \geq MAX\_RETRIES$$

`suspendSession()` is called unconditionally and the retransmission path is not entered.

`suspendSession()` executes four operations in a precisely defined order. First, `memset(s_sessionNonce, 0x00, HELLO_NONCE_SIZE)` erases the 12-byte session nonce from SRAM. This is a forward-secrecy action: the old nonce is cryptographically invalidated, ensuring that knowledge of the post-suspension SRAM image cannot reconstruct any cip-



hertext from the interrupted session. Second, both `seqNum` and `retryCount` are reset to zero, clearing all ARQ accounting state. Third — and critically — `melodyIndex` is preserved without modification. This is the defining self-healing invariant of SH2SC-EDT: the transmitter retains the exact note-sequence position at which disruption occurred, and data transfer will resume from `melodyIndex` upon reconnection rather than restarting from index zero. Fourth, `currentState` is set to `TxState::RECONNECTING` and `lastReconnectAttempt` is pre-armed to `millis()`, scheduling the first automatic `HelloPacket` broadcast exactly `RECONNECT_INTERVAL_MS = 2000 ms` later. A blocking `delay(2000)` holds the “LINK LOST / RECONNECTING...” LCD message — the sole acceptable use of `delay()` outside of `setup()`, justified by its purely operator-visible purpose and its location in a state that carries no UART receive obligation.

The worst-case duration of the retry storm that exhausts `MAX_RETRIES` is bounded by:

$$\begin{aligned} T_{\text{storm}} &= \text{MAX\_RETRIES} \times \text{ACK\_TIMEOUT\_MS} \\ &= 50 \times 50 \text{ ms} = 2500 \text{ ms} \end{aligned}$$

This bound is not arbitrary: it is constrained by the receiver’s `NETWORK_GRACE_PERIOD_MS = 3000 ms` Smart Dynamic Watchdog period. After each authenticated `DataPacket`, the receiver sets its session-release timeout to `current_timeout_limit = duration_ms + NETWORK_GRACE_PERIOD_MS`. The following inequality guarantees that the receiver will not zero `s_sessionNonce` and reset to `WAIT_AA` before the transmitter has finished its complete retry cycle and entered `RECONNECTING`:

$$T_{\text{storm}} < \text{NETWORK\_GRACE\_PERIOD\_MS}$$

Explicitly:

$$2500 < 3000$$

Were the receiver to release its nonce buffer prematurely, the first `DataPacket` dispatched after a successful reconnect handshake would fail MAC verification — because the receiver would have already erased `s_sessionNonce` and could not derive the correct per-packet IV — forcing an additional SYN round-trip and defeating the seamlessness of Auto-Resume. The 500 ms margin eliminates this race condition by construction. The complete decision logic for `TxState::WAITING_ACK`, including all three event branches and the runaway-prevention gate, is illustrated in Figure 5.6.



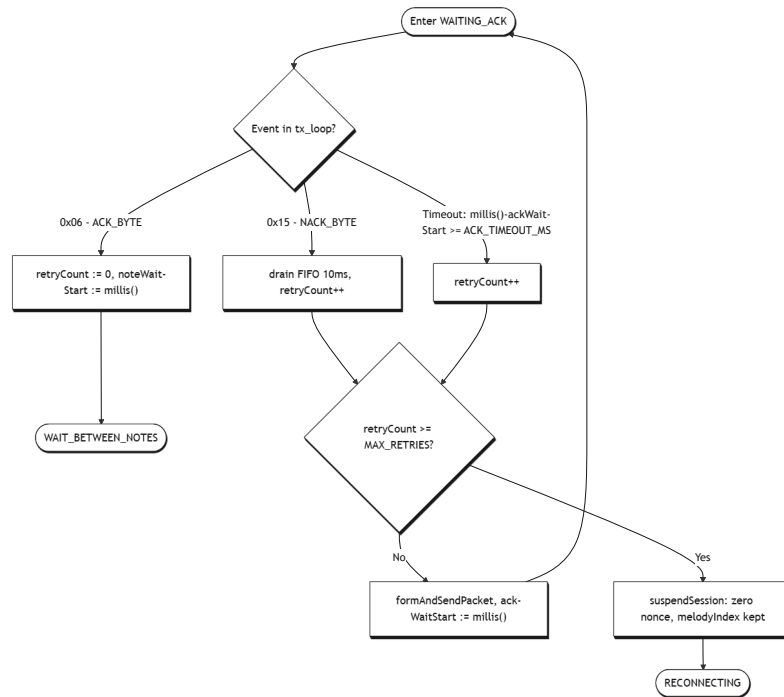


Figure 5.6: FSM decision tree for acknowledgement handling in the `WAITING_ACK` state. Both the `NACK` and `timeout` branches increment the `retryCount` and share the runaway-prevention gate; only the `NACK` branch inserts a 10 ms `FIFO drain` pause before retransmission. On gate failure, `suspendSession()` zeroes the session nonce for forward secrecy and transitions to `RECONNECTING` while preserving `melodyIndex` — the self-healing invariant.

## 6 Security Model & Cryptographic Core

The security of C2P-ARQ is analysed under the standard Dolev-Yao network adversary model [8]: the adversary occupies the communication channel between Node A and Node B as a complete man-in-the-middle and is granted unlimited capabilities over the forward data channel. Specifically, the adversary may observe every transmitted frame in its entirety, replay any previously captured frame at any future time, delay or reorder frames arbitrarily, drop any frame silently, and inject frames of any content at any byte position. These capabilities precisely model the threat surface presented by the SimulIDE XOR noise injector — a hardware switch that subjects every bit of the forward data channel to arbitrary bit-flip under operator control; the Dolev-Yao model is strictly stronger, as the adversary may craft targeted bit patterns rather than only XOR with a fixed waveform, making the SimulIDE experiment a conservative evaluation of a protocol designed to a higher threat standard. [9, 10]

Two bounding assumptions define the cryptographic trust boundary. First, standard-model cryptographic idealisation: ChaCha20 is modelled as a pseudorandom permutation computationally indistinguishable from a uniform random permutation; Poly1305 is modelled as a computationally secure one-time MAC; no polynomial-time adversary can break either primitive under the standard PRF/MAC assumptions. Second, physical memory isolation: the adversary cannot read SRAM contents directly from either node — `MASTER_PSK` and `s_sessionNonce` reside in firmware-controlled memory, are never streamed to any output channel, and are not exposed through any communication interface.

Under these assumptions the session-layer security of every C2P-ARQ session reduces to the secrecy of the single 256-bit `MASTER_PSK` compiled into both firmware images. The PSK is never transmitted, never printed to `Serial` or the Aip31068 LCD, and never derived from any observable parameter. It is the unique shared secret from which all per-session and per-packet cryptographic authority flows: an adversary without `MASTER_PSK` faces computationally infeasible forgery, decryption, and session-hijacking attacks against any frame produced by the AEAD pipeline. The three sections that follow formally establish the properties C2P-ARQ provides within this model: per-packet AEAD authentication ([Section 6.1](#)), nonce injectivity and sequence-number binding ([Section 6.2](#)), and forward-secrecy nonce erasure ([Section 6.3](#)). [11, 12, 13]

## 6.1 Authenticated Encryption with Associated Data (ChaCha20-Poly1305)

### 6.1.1 Why Integrity Dominates Confidentiality for Command Channels

An intuitive but incomplete design for an embedded command channel applies a stream cipher to the payload field alone, providing confidentiality without integrity. This construction is fundamentally insufficient against the active adversary: stream-cipher ciphertext is additively malleable. If the adversary flips bit  $k$  of the 4-byte ciphertext block in `DataPacket.payload`, bit  $k$  of the corresponding decrypted byte flips identically after decryption, regardless of the key. In the C2P-ARQ payload, `note_index` occupies bytes 0–1 and `duration_ms` occupies bytes 2–3 (both `uint16_t`, little-endian). A single ciphertext bit-flip in byte 0 always yields a structurally valid but different `note_index` after decryption — the receiver has no mechanism to detect the substitution. An adversary who observes one plaintext session recovers the complete payload field layout and can thereafter synthesise arbitrary command substitutions over the encrypted channel without knowledge of `MASTER_PSK`. This is the bit-flip injection variant of the plaintext-injection attack identified in [Section 1.1](#). [34, 35]

AEAD eliminates this attack class at the MAC gate. The Poly1305 tag  $\tau$  is a commitment to the exact byte sequence of the authenticated frame: any single-bit modification of the ciphertext or the plaintext header produces — with probability  $1 - 2^{-64}$  — a different expected tag on the receiver, causing the constant-time `memcmp` check to fail. The security invariant enforced by `authenticateAndPlay()` is absolute: `startNote()` executes if and only if `memcmp` returns zero. No field of a MAC-failed packet influences the receiver’s observable state, and no partial execution of the note-play path occurs on forgery attempt. Integrity is therefore not additive to confidentiality in C2P-ARQ — it is structurally prior: without the MAC gate, the adversary can inject commands via ciphertext malleability regardless of whether the payload is encrypted.

### 6.1.2 The Role of Additional Authenticated Data — Frame Header Binding

Ciphertext-only authentication would protect the encrypted payload but leave the 3-byte plaintext header — `flags` (1 byte) and `seq_num` (2 bytes) — unprotected. An adversary intercepting a `FLAG_DAT` frame could flip bit 1 of the `flags` byte, converting 0x02 into

0x04 (`FLAG_FIN`) and thus reclassifying a data-transfer frame as a session-teardown command. The unmodified ciphertext and 8-byte MAC would pass verification, because the tag was computed over the original `FLAG_DAT` header context. The receiver would execute `processFinPacket()`, erase `s_sessionNonce`, and terminate the session — a forced, unauthenticated session-close via a replayed partially modified frame.

C2P-ARQ prevents this through explicit AAD binding. In `sendPacket()`, the call `s_cipher.addAuthData(aadBuf, 3)` feeds the 3-byte local array `{flags, seq_lo, seq_hi}` into the Poly1305 accumulator before `encrypt()` processes the payload. The Poly1305 construction is irreversible: the accumulator mixes the AAD, the ciphertext, and their length encoding into a single 128-bit commitment. The symmetric call on the receiver, `s_cipher.addAuthData(rxAad, 3)`, feeds the same field triple from the received frame. Any inconsistency between the `flags` choice or `seq_num` bytes seen by the transmitter and those seen by the receiver — a one-bit change in any of the three AAD bytes — produces a different expected tag, mandating `NACK_BYTE`. Neither field can be altered without breaking the Poly1305 commitment.

### 6.1.3 Encryption and Authentication Pipeline

The complete per-packet AEAD pipeline — executed by `sendPacket()` on the transmitter and mirrored by `authenticateAndPlay()` on the receiver — comprises five sequential stages: (1) `s_cipher.clear()`, `s_cipher.setKey(MASTER_PSK, 32)`, `s_cipher.setIV(packetNonce, 12)` initialises a fresh ChaChaPoly instance per packet with the 256-bit PSK and the per-packet nonce  $IV_i$ , preventing any state carry-over between frames; (2) `s_cipher.addAuthData(aadBuf, 3)` feeds the 3-byte AAD into the Poly1305 accumulator; (3) `s_cipher.encrypt(plaintext, ciphertext, DATA_PAYLOAD_SIZE)` encrypts the 4-byte payload via the ChaCha20 counter-block keystream and simultaneously feeds the resulting ciphertext into the Poly1305 accumulator; (4) `s_cipher.computeTag(fullMac, AUTH_TAG_SIZE)` finalises the Poly1305 accumulator with the IETF length-encoding block and outputs the full 128-bit tag; (5) only the first `TRUNCATED_MAC_SIZE = 8` bytes of `fullMac` are appended to `DataPacket.mac` and placed on the wire. [36, 37, 38, 39, 40]

The resulting transmitter-side AEAD pipeline is summarized in Figure 6.1.

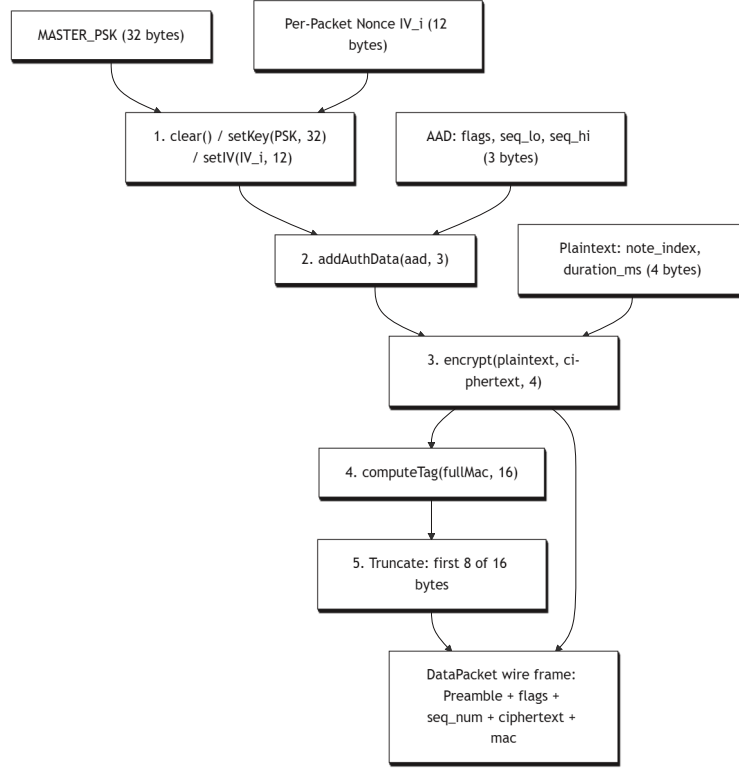


Figure 6.1: ChaCha20-Poly1305 AEAD TX pipeline executed by `sendPacket()` in C2P-ARQ. Each packet uses a fresh ChaChaPoly instance, authenticates the 3-byte plaintext header as AAD, encrypts the 4-byte payload, and transmits only the first 8 bytes of the Poly1305 tag.

### 6.1.4 Formal Verification Predicate

The receiver’s authentication gate is formalised as a deterministic binary predicate. Let  $K$  denote `MASTER_PSK`,  $IV_i$  the per-packet nonce derived for sequence number  $i$ ,  $A$  the 3-byte AAD vector `{flags, seq_lo, seq_hi}`,  $C$  the 4-byte received ciphertext in `DataPacket.payload`, and  $\tau_r$  the received 8-byte truncated MAC in `DataPacket.mac`. The AEAD decryption-and-verification function performed by `authenticateAndPlay()` returns either the 4-byte plaintext  $P$  or the rejection symbol  $\perp$ :

$$\text{Verify}(K, IV_i, A, C, \tau_r) \rightarrow \{P, \perp\}$$

Let

$$au_e \in \{0, 1\}^{128}$$

denote the full 16-byte expected tag produced by `s_cipher.computeTag(expectedMac, 16)` after executing the symmetric AEAD pipeline over

$$(K, IV_i, A, C)$$

Let  $\text{trunc}_8(\tau_e)$  denote the first `TRUNCATED_MAC_SIZE` = 8 bytes of  $\tau_e$ . The outcome is:

$$\text{Verify}(K, IV_i, A, C, \tau_r) = \begin{cases} P & \text{if } \text{trunc}_8(\tau_e) = \tau_r \\ \perp & \text{otherwise} \end{cases}$$

where the equality is evaluated by the constant-time `memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE)` to prevent timing side-channel leakage. The function returns  $P = \{\text{note\_index}, \text{duration\_ms}\}$  in the success branch; the failure branch emits `NACK_BYTE` on the feedback channel and calls `resetParser()`. `startNote()` is invoked exclusively on the branch

$$P \neq \perp$$

— the security invariant documented explicitly in `receiver.h`. Upon success, the `note_index` field of  $P$  is additionally checked against the bound `noteIndex < NOTE_DICT_SIZE` before indexing `universal_notes[]`; a valid MAC does not guarantee an in-range note index, and an out-of-bounds read on ATmega328P SRAM would access adjacent protocol state with undefined consequences.

```

1 // ReceiverNode/receiver.h
2 const uint8_t NOTE_DICT_SIZE = 21;
3
4 void authenticateAndPlay(const DataPacket* pkt);
```

Listing 2: Receiver-side dictionary bound and handler declaration

```

1  // ReceiverNode/ReceiverNode.ino (authenticateAndPlay)
2  uint8_t expectedMac[16];
3  s_cipher.computeTag(expectedMac, 16u);
4
5  if (memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE) != 0) {
6      resetParser();
7      Serial.write(NACK_BYTE);
8      return;
9  }
10
11 Serial.write(ACK_BYTE);
12 const uint16_t noteIndex = plaintext[0];
13 const uint16_t durationMs = plaintext[1];
14
15 if (noteIndex == REST_INDEX) {
16     stopNote();
17 } else if (noteIndex < NOTE_DICT_SIZE) {
18     startNote(universal_notes[noteIndex], durationMs);
19 }

```

Listing 3: MAC verification gate and safe note playback path

The forgery probability for the truncated-tag scheme follows from the Poly1305 security proof. For a uniformly random one-time MAC key [41, 42]

$$r \in \{0, 1\}^{128}$$

derived from ChaCha20 block 0 under  $(K, IV_i)$ , the probability that a computationally unbounded adversary produces a forgery  $\tau'$  satisfying:

$$\text{trunc}_8(\tau') = \text{trunc}_8(\tau_e)$$

without knowledge of  $K$  is at most  $2^{-64}$  per frame, independently of the number of prior valid MAC-ciphertext observations. At 9600 bps the data channel supports at most:

$$\left\lfloor \frac{9600}{17 \times 8} \right\rfloor = 70$$

DataPacket frames per second. At this rate a brute-force forged-MAC campaign exhausts the  $2^{64}$  tag space in approximately:

$$\frac{2^{64}}{70} \approx 2.6 \times 10^{17} \text{ s}$$

roughly 8 billion years at the maximum channel rate.

## 6.2 16-bit Sequence Number and Nonce Management

### 6.2.1 The Nonce-Reuse Hazard

ChaCha20 is a stream cipher whose security rests on the strict uniqueness of the  $(K, IV)$  pair across all encryptions performed under key  $K$ . If two distinct plaintexts  $P_1$  and  $P_2$  are encrypted under the same  $(K, IV)$ , the XOR of the resulting ciphertexts directly exposes the XOR of the plaintexts [43, 44, 45]:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

because the deterministic ChaCha20 keystream cancels identically. In C2P-ARQ terms, the four-byte payload carries `note_index` and `duration_ms`; if both values are known or guessable for one message — for example, if a rest note `REST_INDEX` = 255 with a known duration was previously observed — the second plaintext is immediately recoverable. Moreover, an IV collision exposes the Poly1305 one-time MAC key  $r$ , which is derived as the first 32 bytes of ChaCha20 block 0 under  $(K, IV)$ : with  $r$  known, an adversary can forge a valid Poly1305 tag for any chosen ciphertext and inject it into the session. IV uniqueness is therefore not a security preference but a protocol correctness prerequisite.

### 6.2.2 Per-Packet Nonce Derivation

C2P-ARQ guarantees IV uniqueness within a session through an explicit injective mapping from the 16-bit sequence number  $i$  to the 96-bit per-packet nonce  $IV_i$ . The derivation is implemented identically on TX and RX, operating on a stack-local copy of the session nonce so that `s_sessionNonce` is never mutated:



```

1 // TransmitterNode/TransmitterNode.ino (sendPacket)
2 uint8_t packetNonce[HELLO_NONCE_SIZE];
3 memcpy(packetNonce, s_sessionNonce, HELLO_NONCE_SIZE);
4 packetNonce[10] ^= static_cast<uint8_t>((seqNumber >> 8u) & 0xFFu);
5 packetNonce[11] ^= static_cast<uint8_t>(seqNumber          & 0xFFu);

```

Listing 4: TX per-packet nonce derivation from 16-bit sequence

```

1 // ReceiverNode/ReceiverNode.ino (authenticateAndPlay)
2 uint8_t packetNonce[HELLO_NONCE_SIZE];
3 memcpy(packetNonce, s_sessionNonce, HELLO_NONCE_SIZE);
4 packetNonce[10] ^= static_cast<uint8_t>((seqNum >> 8u) & 0xFFu);
5 packetNonce[11] ^= static_cast<uint8_t>(seqNum          & 0xFFu);

```

Listing 5: RX mirrored per-packet nonce derivation

$$IV_i[k] = \begin{cases} S[k] & k \in \{0, 1, \dots, 9\} \\ S[10] \oplus \lfloor i/256 \rfloor & k = 10 \\ S[11] \oplus (i \bmod 256) & k = 11 \end{cases}$$

where

$$S \in \{0, 1\}^{96}$$

is the 12-byte session nonce stored in `s_sessionNonce` and established during the SYN handshake;  $i$  is the 16-bit packet sequence number (`uint16_t seqNum`) with:

$$i \in \{0, 1, \dots, 65535\}$$

$$\lfloor i/256 \rfloor$$

is the high byte of  $i$ , computed as `(seqNum >> 8u) & 0xFFu`; and

$$i \bmod 256$$

is the low byte, computed as `seqNum & 0xFFu`. Bytes 0–9 of the nonce are left unmodified; only bytes 10 and 11 are perturbed. The immutability of `s_sessionNonce` is enforced by the code comment present in both firmware files: `// NEVER XOR directly into`

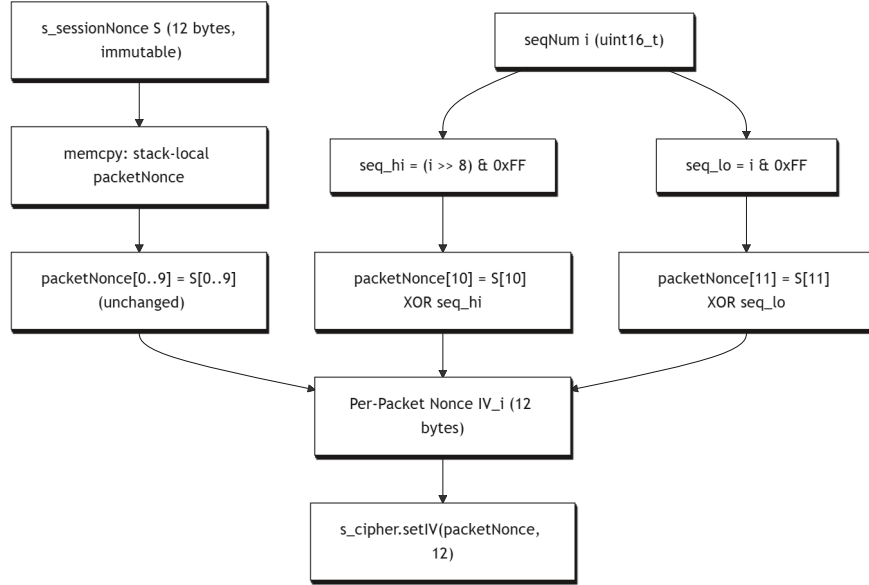


Figure 6.2: Per-packet nonce derivation in `sendPacket()` and `authenticateAndPlay()`. The base session nonce is copied to a stack-local buffer, and only bytes 10 and 11 are perturbed by XOR with the high and low bytes of the 16-bit sequence number.

### 6.2.3 Injectivity of the Mapping

The mapping

$$f : \{0, \dots, 65535\} \rightarrow \{0, 1\}^{96}, i \mapsto IV_i$$

is injective. Suppose

$$IV_i = IV_j \quad \text{for some } i \neq j$$

Since bytes 0–9 of both nonces equal

$$S[0..9]$$

identically, equality of  $IV_i$  and  $IV_j$  reduces to the two conditions:

$$\begin{aligned} S[10] \oplus i_{\text{hi}} &= S[10] \oplus j_{\text{hi}} \\ S[11] \oplus i_{\text{lo}} &= S[11] \oplus j_{\text{lo}} \end{aligned}$$

Applying the XOR cancellation law to each equation yields:

$$i_{\text{hi}} = j_{\text{hi}}, \quad i_{\text{lo}} = j_{\text{lo}}$$

hence:

$$i = j$$

— a contradiction. Therefore all 65,536 possible sequence numbers produce a distinct 96-bit nonce, and within any single session no IV repetition can occur regardless of message content or session duration. The 16-bit range of  $i$  comfortably exceeds the practical upper bound imposed by the ATmega328P Flash budget: the Super Mario Bros. melody stored in `PROGMEM` comprises 80 notes, far below the 65,536-packet ceiling.

## 6.2.4 XOR versus Modular Addition

The derivation employs bitwise XOR rather than modular addition to perturb bytes 10 and 11. Both operations are injective over the 8-bit unsigned range and would therefore produce a correct per-packet IV. The choice of XOR is motivated by AVR architecture constraints: on the ATmega328P, `EOR Rd, Rr` is a single one-cycle instruction with no carry output, and its result in byte 10 is statistically independent of its result in byte 11. Modular addition in C++ on an 8-bit AVR involves implicit integer promotion to `int`, requiring explicit masking back to 8 bits to prevent unsigned-wraparound artifacts, while XOR is carry-free by definition and requires no masking. The derivation is architecturally consistent with the rotate-XOR sponge employed in the entropy blender described in [Section 4.2](#), maintaining uniform use of the same primitive across all bit-manipulation operations in the C2P-ARQ stack.

## 6.3 Key Zeroization and Secure Memory Management

### 6.3.1 Data Remanence on ATmega328P SRAM

Unlike Flash or EEPROM, SRAM is a volatile medium that retains its contents as long as supply voltage is maintained and loses them on power removal. This power-dependency does not, however, eliminate all physical extraction paths. An adversary with physical access to a live node can extract SRAM contents via the debugWIRE/ISP interface by halting the CPU between two instructions, by attaching a probe to the I2C or UART bus and triggering a soft reset with a subsequent memory-read command, or — in a cold-boot variant — by exploiting the residual charge retention that CMOS SRAM cells ex-

ibit for tens to hundreds of milliseconds below the minimum operating voltage. For a session nonce [58]

$$S \in \{0, 1\}^{96}$$

resident in SRAM, any of these paths yields the complete per-packet IV derivation capability described in [Section 6.2](#): from  $S$  and the publicly observable sequence numbers  $i$ , all  $IV_i$  are reconstructible, and all session ciphertexts become decryptable against the static MASTER\_PSK compiled into Flash.

### 6.3.2 Forward Secrecy Through Deterministic Zeroization

C2P-ARQ addresses the data remanence threat through a mandatory zeroization policy: at every session boundary — clean or unclean — the 12-byte session nonce is immediately overwritten with zero bytes. Let  $t_{\text{close}}$  denote the moment at which the terminal FSM transition fires and the first `memset` call executes. The operation is:

$$S \leftarrow \mathbf{0}^{96} \quad \text{at} \quad t = t_{\text{close}}$$

All  $n$  packets transmitted during the session were encrypted under nonces

$$IV_0, IV_1, \dots, IV_{n-1}$$

derived from  $S$  per the mapping of [Section 6.2](#). Setting

$$S = \mathbf{0}^{96}$$

renders every  $IV_i$  irrecoverable from SRAM state: an adversary who obtains a RAM dump at any

$$t > t_{\text{close}}$$

observes only

$$S_{\text{dump}} = \mathbf{0}^{96}$$

Recovering any  $IV_i$  requires enumerating the original  $S$  over the full 96-bit nonce space. Even in the worst-case physical-access threat model in which MASTER\_PSK has been extracted from Flash, the adversary's per-session decryption advantage is bounded by:

$$\Pr[\mathcal{A}(\mathbf{0}^{96}, \text{MASTER\_PSK}) \rightarrow P_i] \leq 2^{-96}$$

where  $\mathcal{A}$  is any probabilistic polynomial-time adversary and  $P_i$  is any plaintext transmitted during the closed session. A brute-force nonce search at  $10^9$  guesses per second exhausts the full nonce space:

$$2^{96} \approx 7.9 \times 10^{28}$$

in approximately:

$$\frac{2^{96}}{10^9} \approx 7.9 \times 10^{19} \text{ s}$$

— more than 180 times the age of the observable universe. This establishes forward secrecy at the session layer: current SRAM state yields no decryption capability over ciphertexts from any closed session. [46, 47, 48]

### 6.3.3 Zeroization Trigger Points

The complete lifecycle of `s_sessionNonce` in SRAM — from its initial undefined value through active use and deterministic erasure — is illustrated in Figure 6.3. The C2P-ARQ implementation defines four erasure call sites, exhaustively covering all session termination paths.

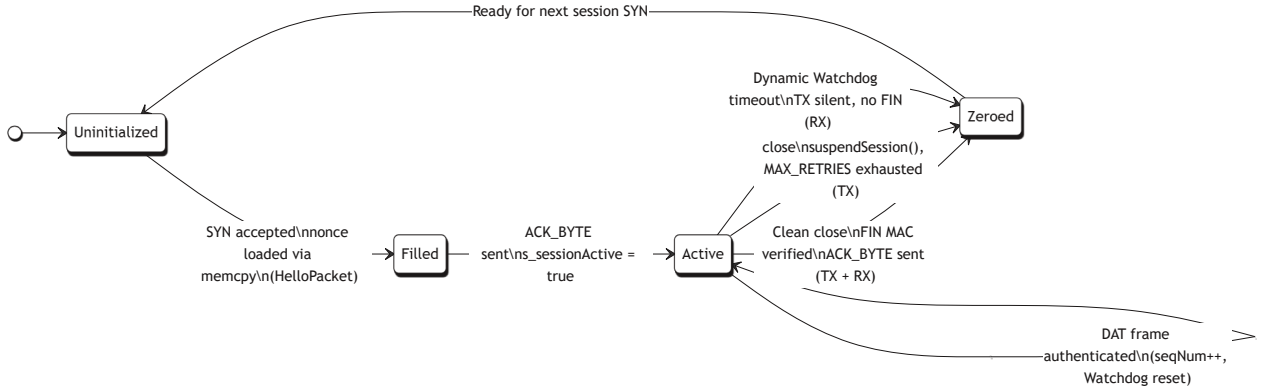


Figure 6.3: `s_sessionNonce` memory lifecycle in C2P-ARQ. Three independent triggers drive the transition to the Zeroed state, ensuring that key material is erased under both protocol-level termination and unilateral link failure.

#### TX Node — Clean Close

WAITING\_FIN\_ACK success branch (`TransmitterNode.ino`). Upon receiving `ACK_BYTE` confirming the `FLAG_FIN` packet, the FSM executes `memset(s_sessionNonce, 0,`

HELLO\_NONCE\_SIZE) before resetting `melodyIndex`, `seqNum`, and `retryCount` and transitioning to `TxState::IDLE`:

```

1  // TransmitterNode/TransmitterNode.ino (WAITING_FIN_ACK -> clean close)
2  if (response == ACK_BYTE) {
3      memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4      melodyIndex = 0;
5      seqNum      = 0;
6      retryCount  = 0;
7      currentState = TxState::IDLE;
8      updateTxDisplay(currentState, seqNum, retryCount);
9  }
```

Listing 6: Clean session close on FIN acknowledgment (TX)

## TX Node — Unclean Close

`suspendSession()` (`TransmitterNode.ino`). When `retryCount` reaches `MAX_RETRIES` in any `WAITING_*` FSM state, `suspendSession()` executes `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)` as its first action — before preserving `melodyIndex`, before resetting counters, and before entering `TxState::RECONNECTING`:

```

1  // TransmitterNode/TransmitterNode.ino (suspendSession)
2  void suspendSession() {
3      memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4      seqNum      = 0;
5      retryCount  = 0;
6      currentState = TxState::RECONNECTING;
7      lastReconnectAttempt = millis();
8  }
```

Listing 7: Unclean suspend path with reconnect transition (TX)

## RX Node — Clean Close

`processFinPacket()` (`ReceiverNode.ino`). Upon successful Poly1305 MAC verification of the `FLAG_FIN` packet, the function executes `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)` and clears `s_sessionActive` before calling `resetParser()`:

```

1 // ReceiverNode/ReceiverNode.ino (processFinPacket)
2 if (memcmp(expectedMac, pkt->mac, TRUNCATED_MAC_SIZE) != 0) {
3     Serial.write(NACK_BYTE);
4     resetParser();
5     return;
6 }
7
8 Serial.write(ACK_BYTE);
9 memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
10 s_sessionActive = false;
11 resetParser();

```

Listing 8: FIN verification and secure teardown on RX

## RX Node — Dynamic Watchdog Timeout

`rx_loop()` (`ReceiverNode.ino`). When no authenticated `DataPacket` arrives within `current_timeout_limit` milliseconds after `last_valid_packet_time` — the condition detecting that the transmitter has disappeared without sending `FLAG_FIN` — the watchdog handler executes `memset(s_sessionNonce, 0, HELLO_NONCE_SIZE)`, clears `s_sessionActive`, and resets the parser:

```

1 // ReceiverNode/ReceiverNode.ino (Dynamic Watchdog timeout)
2 if (s_sessionActive &&
3     (millis() - last_valid_packet_time) > current_timeout_limit) {
4     memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
5     s_sessionActive = false;
6     resetParser();
7 }

```

Listing 9: Dynamic watchdog-triggered session cleanup on RX

### 6.3.4 Entropy Seed Scrubbing

Both firmware variants apply a fifth `memset` call — not to `s_sessionNonce` but to the 32-byte raw entropy seed produced by `generateEntropyPool()`. In `tx_setup()` (line 472) and `rx_setup()` (line 640), the seed lives in a block-scoped stack buffer:

```
1  {  
2      uint8_t entropySeed[32];  
3      generateEntropyPool(entropySeed);  
4      initCSPRNG(entropySeed);  
5      memset(entropySeed, 0, sizeof(entropySeed));  
6  }
```

Listing 10: Entropy seed initialization and stack scrubbing

Without this scrub, the raw TRNG output — carrying up to:

$$H_{\text{raw}} \approx 24\text{--}32$$

bits of physical entropy prior to ChaCha20 conditioning — would remain accessible at a fixed stack address for the remainder of the program’s execution, providing a partial foothold for an adversary who can observe SRAM after initialisation.

### 6.3.5 Compiler Optimisation Note

The C standard permits a compiler to elide `memset()` calls that target buffers not subsequently accessed — the “dead store” elimination optimisation. Under `avr-gcc` at `-Os` (the default Arduino optimisation level), this elimination does not occur in practice for the SRAM-addressed writes examined here; however, production deployments requiring a formal machine-level guarantee should replace the plain `memset()` calls with `memset_s()` (C11 Annex K) or an explicit `volatile uint8_t *` cast, both of which are defined to prevent dead-store elimination across all conforming optimisation levels. [59]



## 7 Fault Tolerance and Self-Healing Mechanisms

The cryptographic mechanisms analysed in [Section 6](#) — AEAD authentication, per-packet nonce uniqueness, and deterministic key erasure — constitute a defence against an adversary whose goal is to *manipulate* the payload: forge notes, replay old frames, or reconstruct plaintext from captured ciphertext. Against such an adversary, C2P-ARQ provides strong security guarantees bounded by the computational hardness of compromising ChaCha20-Poly1305.

A physically active attacker, however, need not engage with the cryptographic layer at all. By injecting sufficient noise to corrupt every transmitted bit, the attacker transforms the channel into one on which no valid frame can reach the receiver. The Poly1305 MAC correctly rejects every corrupted frame; yet this correct behaviour does nothing to advance the melody. Each rejected frame consumes one unit of the retry budget. When `MAX_RETRIES` = 50 retries at `ACK_TIMEOUT_MS` = 50 ms each are exhausted — a sustained jamming interval of:

$$50 \times 50 = 2,500 \text{ ms}$$

— the session terminates. Security has been maintained perfectly; availability has been destroyed.

This dichotomy motivates the fault-tolerance mechanisms described in this section. Rather than treating link interruption as a fatal error requiring operator intervention, C2P-ARQ defines an autonomous recovery automaton that re-establishes the session and resumes transmission from the exact note at which it was disrupted. This property — termed Self-Healing — is the defining differentiator of SH2SC-EDT from conventional embedded communication implementations.

## 7.1 The RECONNECTING State and Payload State Preservation

### 7.1.1 The Naive IoT Architecture

The prevalent response to communication failure in resource-constrained embedded systems is a hardware watchdog reset: if the application does not service the watchdog within a configured interval, the MCU is rebooted. While a reboot guarantees that the firmware returns to a known initial state, it simultaneously destroys all application-layer progress. In the context of SH2SC-EDT, this would reset `melodyIndex` to zero and restart the melody from  $P_0$  on every link interruption, regardless of how many notes had already been successfully delivered.

### 7.1.2 C2P-ARQ State Partition

SH2SC-EDT avoids this regression by partitioning the transmitter’s runtime state into two conceptually distinct categories:

- Cryptographic state — state that *must* be reset after each session to maintain security: `s_sessionNonce` (12 bytes), `seqNum`, and `retryCount`. Retaining a stale nonce across a session boundary would allow the same IV to be reused for new plaintext, violating nonce uniqueness and enabling ciphertext distinguishability attacks.
- Payload state — state that encodes application-layer progress and *must not* be reset when a session is disrupted: `melodyIndex` (2 bytes). Resetting this field serves no security purpose and destroys the liveness property that motivates the Self-Healing design.

`suspendSession()` implements this partition precisely. The function zeroes `s_sessionNonce` via `memset()`, resets `seqNum = 0` and `retryCount = 0`, and transitions the FSM to `TxState::RECONNECTING` — leaving `melodyIndex` untouched. The co-located source comment reads: “*melodyIndex intentionally NOT reset — resume from point of failure.*” [60]

### 7.1.3 Self-Healing Invariant

Let

$$\mathcal{M} = \langle P_0, P_1, \dots, P_{M-1} \rangle$$

be the melody of  $M$  notes. A *disruption event* at index  $k$  is defined as the execution of `suspendSession()` while `melodyIndex` =  $k$ . Define  $k_j$  as the value of `melodyIndex` at the moment the  $j$ -th disruption occurs. The C2P-ARQ Self-Healing invariant states that the subsequent session  $\mathcal{S}_{j+1}$  begins transmission from  $P_{k_j}$ :

$$\text{melodyIndex}[\mathcal{S}_{j+1}] = k_j \quad \forall j \in \{1, \dots, n\}$$

For any sequence of  $n \geq 0$  disruption events, the note delivered in the first DataPacket of  $\mathcal{S}_{j+1}$  is therefore  $P_{k_j}$ , not  $P_0$ . The complete melody  $\mathcal{M}$  is eventually delivered without gaps provided that at least one session for each remaining suffix

$$\langle P_{k_j}, \dots, P_{M-1} \rangle$$

completes without disruption.

The Self-Healing automaton from the TX perspective is shown in Figure 7.1.

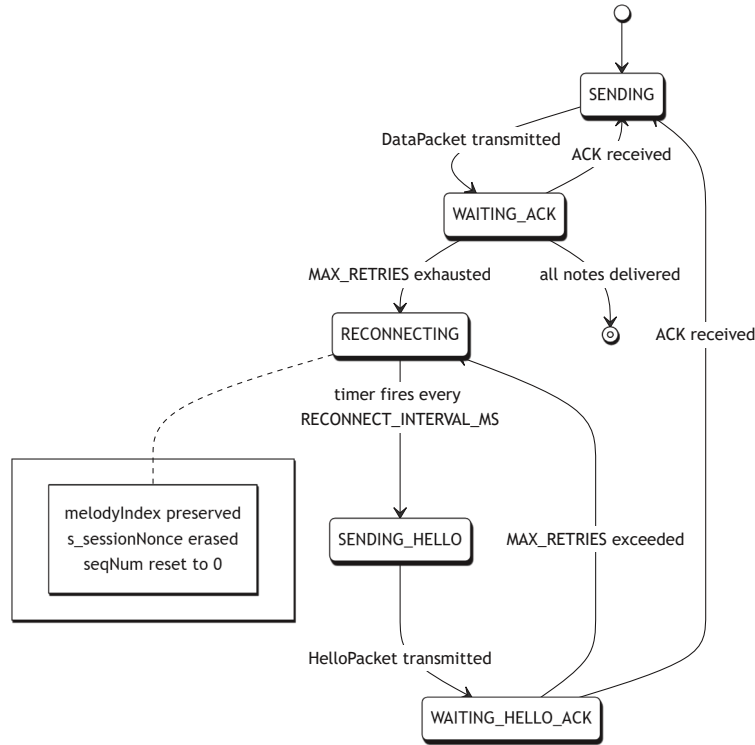


Figure 7.1: Self-Healing automaton on the TX node. On `MAX_RETRIES` exhaustion, `suspendSession()` erases cryptographic state and preserves `melodyIndex`; the non-blocking timer re-attempts the handshake every `RECONNECT_INTERVAL_MS`.

### 7.1.4 Non-Blocking RECONNECTING Polling

After entering `TxState::RECONNECTING`, the transmitter does not busy-wait or block. Every invocation of `tx_loop()` evaluates a single `millis()`-based condition:

```

1  if ((millis() - lastReconnectAttempt) >= RECONNECT_INTERVAL_MS) {
2      lastReconnectAttempt = millis();
3      currentState = TxState::SENDING_HELLO;
4      updateTxDisplay(currentState, seqNum, retryCount);
5  }
```

Listing 11: Non-blocking reconnect timer check in TX loop

When the condition holds, the FSM transitions to `TxState::SENDING_HELLO`, which issues a fresh `HelloPacket` with a new CSPRNG-generated nonce in the following `tx_loop()` invocation. If the receiver remains unreachable, `WAITING_HELLO_ACK` exhausts its own `MAX_RETRIES` budget and re-invokes `suspendSession()`, which re-enters `RECONNECTING` and restarts the 2000 ms timer. This creates a persistent auto-reconnect loop that terminates only when the receiver responds with `ACK_BYTE`. At that point, the `WAITING_HELLO_ACK → SENDING` transition carries the preserved `melodyIndex =  $k_j$`  into the new session — no operator intervention required.

An operator may override the Self-Healing resumption by pressing the TX button while in `RECONNECTING`: this explicitly resets `melodyIndex = 0` and forces a melody restart from  $P_0$ .

## 7.2 Smart Dynamic Watchdog: Payload-Aware Timeout Adaptation

### 7.2.1 The Receiver Deadlock Problem

The `FLAG_FIN` teardown path described in [Section 5](#) achieves clean session closure only under the assumption that the transmitter remains capable of sending a final authenticated packet. In practice this assumption is violated by two failure modes: a hard power-cut at the transmitter, and the scenario analysed in [Section 7.1](#) where the transmitter exhausts `MAX_RETRIES` and enters `RECONNECTING` without issuing a `FIN`. In both cases the re-

is explicitly released. Without an active release mechanism, the next session can never be established — the system deadlocks.

A static session-release timeout does not resolve this deadlock cleanly. A short static value — smaller than the longest note duration in the melody — fires before the transmitter has had time to transmit the next packet, producing a spurious session reset during normal note playback. A long static value — chosen conservatively to exceed the longest possible note — forces the receiver to hold session RAM for the full static interval after the transmitter disappears, delaying reconnection by the full watchdog period unnecessarily. Neither choice can simultaneously avoid false positives on long notes and minimise reconnection delay after a genuine link failure.

### 7.2.2 Payload-Aware Adaptation

C2P-ARQ resolves this dichotomy by exploiting a property of the payload that is already available on the receiver at no extra protocol cost: the duration field `duration_ms` carried in every authenticated `DataPacket`. Because this field is extracted from the Poly1305-verified plaintext — and is therefore guaranteed authentic — the receiver can compute a precise lower bound on the silence interval that must elapse before the next packet is expected. The watchdog timeout for packet  $k$  is not a compile-time constant but is computed dynamically inside `authenticateAndPlay()` immediately after the note starts:

```
1  current_timeout_limit = static_cast<uint32_t>(durationMs)
2                          + NETWORK_GRACE_PERIOD_MS;
3  last_valid_packet_time = millis();
```

Listing 12: Adaptive watchdog timeout update after valid packet

Prior to the first `DataPacket` of the session — during the interval after `processHelloBody()` completes and before `authenticateAndPlay()` is called for the first time — the watchdog is armed with a conservative default of `current_timeout_limit = 5000 ms`. This default covers the case where the channel is jammed immediately after the SYN handshake, before any note duration has been revealed.

### 7.2.3 Formal Timeout Condition

Let  $t_{\text{last}}^{(k)}$  denote the `millis()` timestamp recorded by the receiver immediately after authenticating packet  $k$ , and let  $d_k$  denote the note duration extracted from that packet's

plaintext. The Dynamic Smart Watchdog fires — zeroing `s_sessionNonce`, clearing `s_sessionActive`, and resetting the byte-parser — when and only when:

$$t_{\text{now}} - t_{\text{last}}^{(k)} > d_k + T_{\text{grace}}$$

where  $t_{\text{now}}$  is the value of `millis()` at the current `rx_loop()` invocation,

$$d_k \in \mathbb{N}_{>0}$$

is the duration in milliseconds of the  $k$ -th authenticated note, and  $T_{\text{grace}} = 3000 \text{ ms}$  is the fixed `NETWORK_GRACE_PERIOD_MS` constant compiled into both firmware images. The right-hand side is the minimum time that must elapse between the last authenticated packet and the watchdog trigger:  $d_k$  milliseconds for the note itself to play, plus  $T_{\text{grace}}$  milliseconds of additional tolerance absorbing the transmitter's worst-case retry storm and round-trip frame time.

The evaluation occurs in `rx_loop()` on every loop iteration, guarded by the `s_sessionActive` predicate to eliminate dead computation when no session is open:

```

1  if (s_sessionActive &&
2      (millis() - last_valid_packet_time) > current_timeout_limit) {
3      memset(s_sessionNonce, 0, HELLO_NONCE_SIZE);
4      s_sessionActive = false;
5      resetParser();
6  }
```

Listing 13: RX watchdog expiration condition and parser reset

The complete watchdog logic — from session arming through conditional erasure — is illustrated in Figure 7.2.

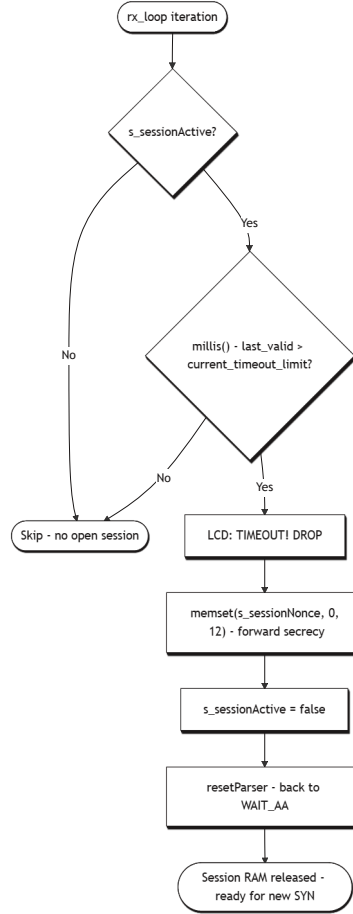


Figure 7.2: Smart Dynamic Watchdog evaluation path in `rx_loop()`. The condition `millis() - last_valid_packet_time > current_timeout_limit` is evaluated on every loop iteration; the erasure branch executes at most once per session.

### 7.2.4 Absence of Race Conditions Between TX and RX

A critical invariant must hold: the receiver’s watchdog must never fire while the transmitter is still in the middle of a legitimate retry storm — that is, the receiver must not zero `s_sessionNonce` before the transmitter has finished its `MAX_RETRIES × ACK_TIMEOUT_MS` retry cycle and entered `TxState::RECONNECTING`. If this race condition could occur, the first `HelloPacket` of the reconnect handshake would be processed by a receiver that has already released its session nonce, which is the correct outcome. However, a `DataPacket` retransmission arriving at the receiver after `s_sessionNonce` has been zeroed would fail MAC verification and produce a NACK — escalating an already-degraded link into an unrecoverable desynchronization until the next SYN handshake completes.

The protocol eliminates this race by construction. The worst-case TX retry storm duration is bounded at 2500 ms, as derived earlier in [Section 5.3.3](#).

The grace period satisfies:

$$T_{\text{grace}} = 3000 \text{ ms} > T_{\text{storm}} = 2500 \text{ ms}$$

providing a 500 ms safety margin. Since the watchdog timeout is

$$d_k + T_{\text{grace}}$$

and

$$d_k \geq 0$$

the receiver is guaranteed to hold its session nonce for at least  $T_{\text{grace}} = 3000 \text{ ms}$  beyond the last authenticated packet — longer than the transmitter can remain in any `WAITING_*` state before entering `RECONNECTING`. The source comment at `NETWORK_GRACE_PERIOD_MS` reflects this precisely: *“Absorbs the worst-case TX retry storm:  $\text{MAX\_RETRIES} \times \text{ACK\_TIMEOUT\_MS} = 50 \times 50 \text{ ms} = 2500 \text{ ms}$ . Value 3000 ms provides 500 ms margin above that maximum.”* The two nodes thus converge to their respective idle states — `TxState::RECONNECTING` and the byte-parser reset to `WAIT_AA` — in the correct temporal order, making the subsequent SYN handshake unconditionally successful.

## 7.3 Stale Buffer Drain and Desynchronization Recovery

### 7.3.1 The Stale-Byte Poisoning Problem

The ATmega328P UART peripheral is backed by a 64-byte hardware ring buffer managed by the Arduino `HardwareSerial` driver. During a noise burst — or during any interval in which the processor is occupied with cryptographic computation and cannot service the receive buffer — incoming bytes accumulate in this FIFO without being consumed. When the burst ends, these bytes remain resident in the buffer and will be delivered to the next `Serial.read()` call as though they had just arrived. This property is the source of a subtle failure mode: if a retransmission storm produces a sequence of NACK bytes that are still resident in the hardware FIFO when the next packet is transmitted, the transmitter will read them as the response to the new packet rather than the prior one — triggering false retransmissions or, in the worst case, advancing the FSM on a NACK that was never intended as a reply to the current sequence number.



The problem is exacerbated by asymmetry between the TX and RX UART buffers. The Arduino `Serial.flush()` call flushes only the *transmit* buffer — it blocks until all pending outgoing bytes have been shifted out of the TX shift register. It has no effect on the *receive* buffer. There is no Arduino API call that atomically drains incoming bytes from the hardware FIFO; the only correct mechanism is a manual drain loop that reads and discards all pending bytes before the transmission of a new packet. [53, 54, 55]

### 7.3.2 TX Active Drain — `drainRxFifo()`

C2P-ARQ addresses this on the transmitter side through a dedicated inline function called before every packet transmission:

```

1  static inline void drainRxFifo() {
2      while (Serial.available() > 0) {
3          Serial.read();
4      }
5  }
```

Listing 14: TX helper for draining stale UART RX bytes

`drainRxFifo()` reads and discards all bytes currently available in the receive buffer in a tight loop, consuming at most 64 iterations (the hardware FIFO depth). It is called as the first operation in each of the three transmit functions — `sendHelloPacket()`, `sendPacket()`, and `sendFinPacket()` — before `Serial.write()` is invoked. The source comment documents this precisely: *“Prevents stale NACK bytes — accumulated from noise bursts before the session started — from being misread as responses to the packet that is about to be transmitted. Called at the start of every send function to guarantee that the first byte read after a send is always a fresh response.”*

The consequence is a strong invariant: the first byte the transmitter reads from the feedback channel after issuing any packet is guaranteed to be a response to *that* packet, not a residue of any prior noise burst or retransmission cycle.

### 7.3.3 RX Passive Drain — `resetParser()`

On the receiver side, FIFO poisoning arises in the inverse direction: after a MAC failure or a parser timeout, the hardware FIFO may still contain trailing bytes from the corrupted frame or from subsequent noise pulses that arrived while the CPU was executing

eivedByte() on the next loop() iteration and may produce a spurious preamble match or corrupt the accumulation of the next legitimate frame. resetParser() eliminates this path with an identical drain loop followed by a hard FSM reset:

```

1  static void resetParser() {
2      while (Serial.available() > 0) {
3          Serial.read();
4      }
5      rx_index      = 0;
6      parseState    = WAIT_AA;
7      currentState = RxState::WAITING_SYNC_1;
8  }

```

Listing 15: RX parser reset with FIFO drain and state re-arm

The docstring states the design intent explicitly: “Called on MAC failure or parser timeout to prevent a noise burst that corrupted one frame from also corrupting the preamble detection of the next frame. Reads and discards all bytes in the 64-byte hardware FIFO (Serial.flush() only flushes TX; manual drain is required for RX).” After resetParser() returns, the byte-parser FSM is in a clean, fully-defined initial state: rx\_index = 0, parseState = WAIT\_AA, currentState = WAITING\_SYNC\_1.

### 7.3.4 RX Passive Sync — Preamble Hunting

Once the FIFO is drained and the parser is in WAIT\_AA, the receiver begins passively scanning the channel byte by byte, discarding everything until a valid 0xAA 0x55 preamble is identified. processReceivedByte() implements this as two sequential FSM states:

- WAIT\_AA: every incoming byte is examined. Only SYNC\_BYTE\_1 = 0xAA advances the parser to WAIT\_55; every other byte is silently discarded and the parser remains in WAIT\_AA.
- WAIT\_55: the second byte is examined. SYNC\_BYTE\_2 = 0x55 advances to READ\_TYPE (preamble confirmed). A second 0xAA keeps the parser in WAIT\_55 — this handles the overlapping case 0xAA 0xAA 0x55 correctly. Any other byte resets to WAIT\_AA — the partial preamble 0xAA X is discarded and scanning resumes.

No frame body is accumulated until both preamble bytes have been matched in sequence. The gate is unconditional: FLAG\_SYN, FLAG\_DAT, and FLAG\_FIN frames are all subject to the

The combined TX-drain and RX-preamble-hunt desynchronization recovery process is illustrated in Figure 7.3.

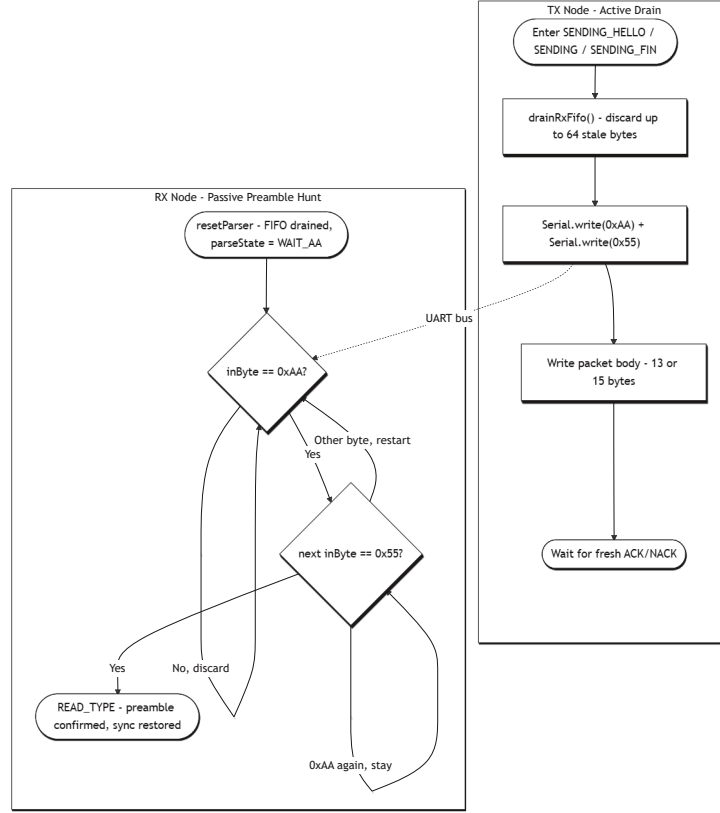


Figure 7.3: Bilateral desynchronization recovery after a noise burst. The TX node drains its receive FIFO before transmitting; the RX node discards arbitrary garbage bytes until the 0xAA 0x55 preamble is matched.

### 7.3.5 Probability of False Synchronization

The two-byte preamble 0xAA 0x55 is chosen to minimise the probability that random channel noise produces a spurious preamble match. Let  $b_i$  denote the value of an arbitrary byte drawn uniformly at random from

$$\{0, \dots, 255\}$$

The probability that two consecutive noise bytes constitute a valid preamble is [56, 57]:

$$P(b_i = \mathbf{0xAA} \wedge b_{i+1} = \mathbf{0x55}) = \frac{1}{256} \times \frac{1}{256} = 2^{-16} \approx 1.52 \times 10^{-5}$$

where  $b_i$  and  $b_{i+1}$  are two consecutive independently uniformly distributed random bytes,  $\mathbf{0xAA} = 170_{10}$  is SYNC\_BYTE\_1, and  $\mathbf{0x55} = 85_{10}$  is SYNC\_BYTE\_2. The probability

$2^{-16} \approx 1.52 \times 10^{-5}$  represents the expected rate of spurious preamble matches per two-byte window under the worst-case assumption that the noise source is a uniformly random byte generator — the maximum-entropy adversary for byte-value statistics. In practice, a 1 kHz XOR pulse-generator noise source produces bit-flips rather than uniformly random character-valued bytes, leading to correlated corruptions that cluster around the neighbourhood of the original data value. The false preamble rate under this physically realistic noise model is strictly lower than the theoretical  $2^{-16}$  bound.

After a false preamble match, the parser advances to `READ_TYPE`. A false `READ_TYPE` byte that does not match any recognised flag bit (`FLAG_SYN` = 0x01, `FLAG_DAT` = 0x02, `FLAG_FIN` = 0x04) immediately resets the parser to `WAIT_AA`: the source code at the unrecognised-flag branch reads “*No recognised flag bit — channel noise, restart preamble scan.*” For a false body to survive to the dispatch stage, the false `READ_TYPE` byte must have at least one of bits 0, 1, or 2 set — a probability of **7/256** — and the accumulated body bytes must produce a Poly1305 MAC that passes `memcmp` against the truncated 8-byte tag, the probability of which is  $2^{-64}$  under any computationally bounded adversary. The compound false-positive probability is therefore negligible under all realistic operating conditions.

# 8 Adversary Model and Jamming Simulation

[Section 6](#) and [Section 7](#) establish two necessary pillars of correctness: cryptographic security under the AEAD construction and deterministic recovery logic under bounded retransmission. These results guarantee confidentiality, integrity, and protocol-level state repair properties under the C2P-ARQ design assumptions. However, they are not by themselves sufficient to prove operational robustness in the presence of sustained physical-layer disruption.

Liveness and fault tolerance must be demonstrated empirically under adversarial channel conditions, where frame delivery is repeatedly impeded by active interference rather than by benign random noise. The objective of this section is therefore to subject C2P-ARQ to the strongest physical stress test available in the SimulIDE environment and evaluate whether end-to-end session progress is preserved without operator intervention.

## 8.1 The Attacker Node Capabilities

In the SH2SC-EDT testbed, the Attacker is not a programmable microcontroller and does not execute firmware logic. It is implemented as a dedicated hardware sub-circuit placed in-line with the UART data path, acting as a physical Man-in-the-Middle between the transmitter and receiver. Consequently, it has no protocol-layer visibility: it does not parse packet structures, does not compute or verify authentication tags, and does not generate C2P-ARQ-compliant frames.

Its passive capabilities are provided by SimulIDE instrumentation. Through the oscilloscope and logic analyzer, the adversary can observe the bit-level waveform, measure temporal characteristics of transmissions, and infer traffic morphology (burst structure, idle gaps, and retransmission cadence). Because payload-bearing frames are protected by ChaCha20-Poly1305, such observation exposes timing and channel-occupancy metadata only, not authenticated plaintext semantics.

Its active capability is physical channel interference: the adversary can seize the transmission line and inject disruptive signal activity that corrupts in-flight symbols and invalidates received frames. This model captures hardware-level jamming pressure on the UART link and provides the adversarial basis for evaluating protocol liveness and fault

tolerance under prolonged disruption. The internal construction of the jamming instrument is intentionally deferred to [Section 8.2](#).

The Attacker sub-circuit used in the SH2SC-EDT testbed is shown in Figure 8.1.

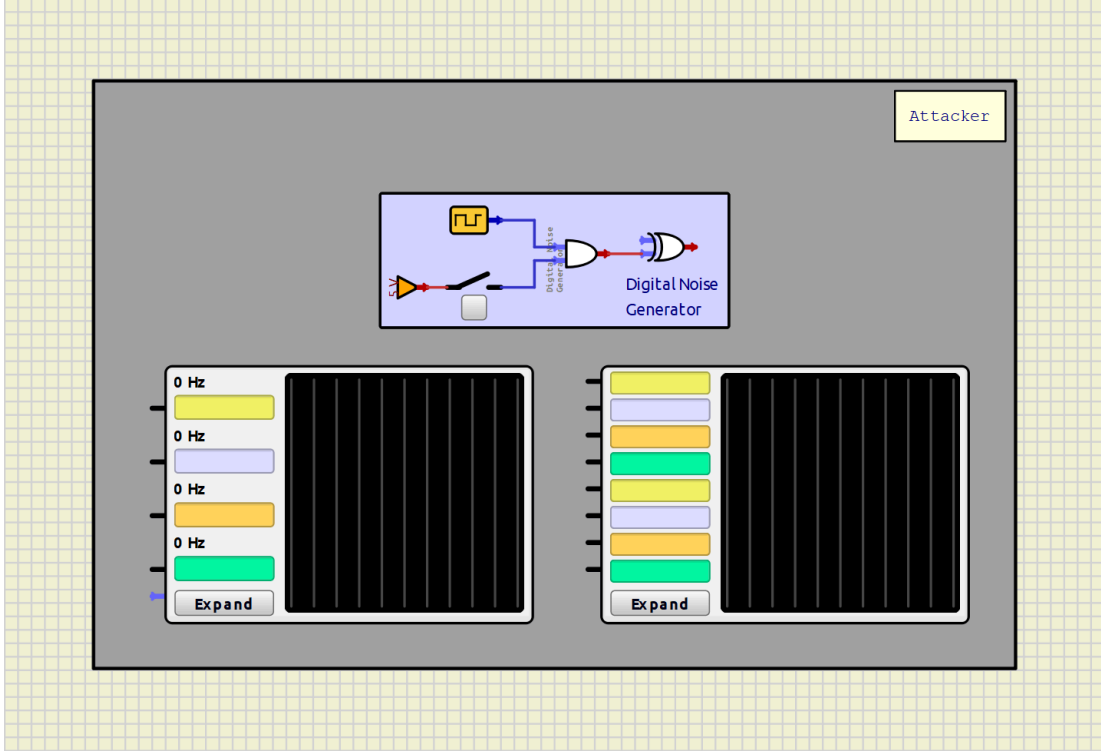


Figure 8.1: The Attacker sub-circuit overview in SimulIDE, equipped with an oscilloscope and logic analyzer for passive monitoring, and a digital noise generator for active channel disruption.

## 8.2 Digital Noise Generator Architecture and XOR Injection

The Digital Noise Generator is implemented in SimulIDE as a compact hardware logic chain whose sole purpose is to superimpose controlled binary interference onto the forward UART channel. Its topology consists of a 5 V manual enable path, a 2000 Hz pulse source, an AND gate that either permits or suppresses the disturbance stream, and an XOR gate that injects the resulting jamming waveform into the clean serial output of Node A before the signal reaches Node B.

The logic-chain architecture of the Digital Noise Generator is illustrated in Figure 8.2.

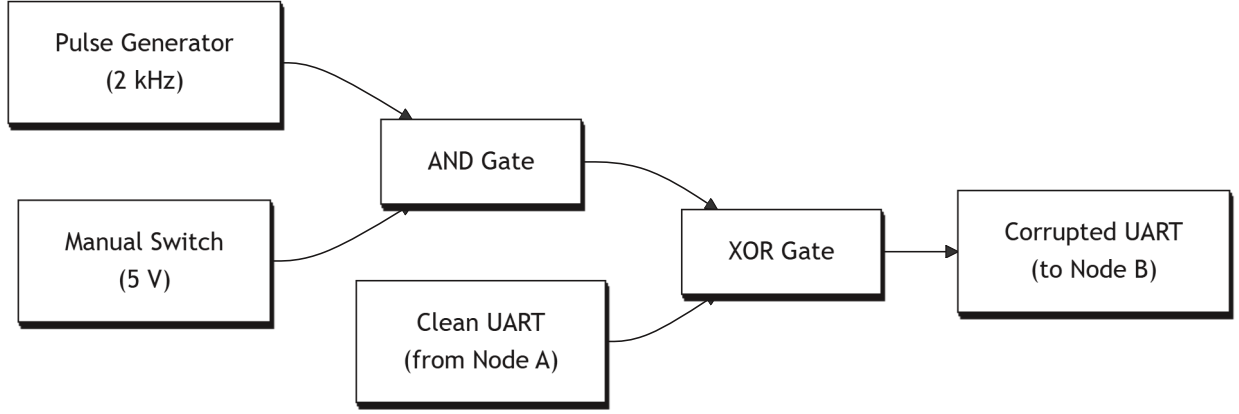


Figure 8.2: Logical flow of digital noise injection in the SH2SC-EDT SimulIDE testbed. A 2000 Hz pulse source is conditionally enabled by a manual 5 V switch through an AND gate and then injected into the clean UART stream through an XOR stage, yielding a corrupted serial signal at the receiver input.

At the logic level, the AND stage operates as an enable gate. Let

$$P(t) \in \{0, 1\}$$

denote the 2000 Hz pulse train and let

$$S \in \{0, 1\}$$

denote the state of the manual switch, where  $S = 0$  corresponds to the open state and  $S = 1$  to the closed state. The disturbance signal delivered to the XOR input is then

$$N(t) = P(t) \wedge S$$

so that, when the switch is open,

$$N(t) = P(t) \wedge 0 = 0$$

for all  $t$ , and the noise path is electrically blocked. In this quiescent configuration, the XOR stage receives a zero-valued disturbance input and therefore behaves transparently with respect to the serial line.

The corruption mechanism is governed by the Boolean algebra of XOR. If  $V_{\text{uart}}(t)$  denotes the instantaneous logic level on the clean UART line, then the output delivered toward Node B is

$$V_{\text{out}}(t) = V_{\text{uart}}(t) \oplus N(t)$$

which yields the two operative identities

$$V_{\text{uart}} \oplus 0 = V_{\text{uart}}, \quad V_{\text{uart}} \oplus 1 = \neg V_{\text{uart}}$$

These relations formalize the physical behaviour of the injector. With the switch open, the AND output is forced low and the UART signal passes through the XOR stage without alteration. With the switch closed, the AND output reproduces the 2000 Hz pulse train, and every interval in which that train is logically high forces the XOR stage to invert the instantaneous level presently carried by the UART conductor. The channel is therefore not overwritten with an independent waveform; it is conditionally polarity-inverted in real time.

The destructive effect follows from the timing relationship between the disturbance source and the asynchronous serial link. At 9600 bps, one UART bit occupies

$$T_{\text{bit}} = \frac{1}{9600} \approx 104 \mu s$$

whereas a 2000 Hz disturbance source has period

$$T_{\text{jam}} = \frac{1}{2000} = 500 \mu s$$

which is substantially shorter than the duration of a complete 10-bit UART character (approximately **1.04 ms** including start and stop bits). Consequently, once the injector is enabled, polarity inversions occur on a sub-frame timescale and intersect multiple symbol intervals during a packet transmission. Any such inversion that strikes the authenticated header, ciphertext, or truncated Poly1305 tag changes the received bit pattern and causes the C2P-ARQ integrity check to fail deterministically. The Digital Noise Generator therefore serves as a physically grounded bit-flip instrument: it does not emulate packet loss at the software level, but instead corrupts the electrical representation of the frame in transit.

The internal gate-level architecture of the Digital Noise Generator is shown in Figure 8.3.



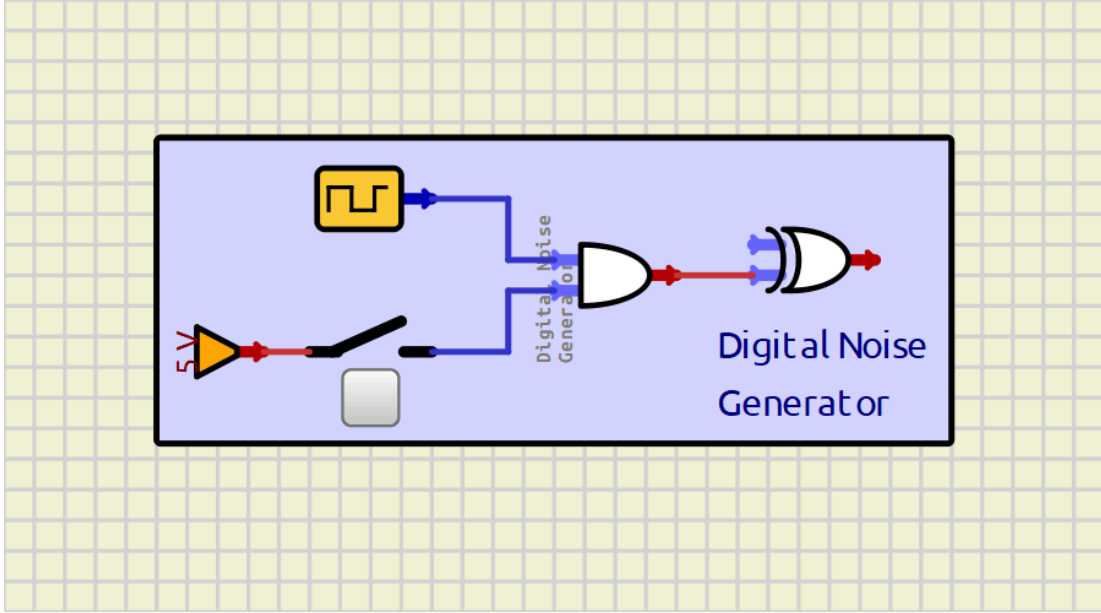


Figure 8.3: Internal architecture of the Digital Noise Generator in SimulIDE. A 2000 Hz clock signal is gated by a manual switch via an AND gate, and injected into the UART data line via an XOR gate to induce deterministic bit-flips.

### 8.3 Simulation Results: Resilience Under Prolonged Jamming

This subsection reports the empirical behaviour of SH2SC-EDT under sustained physical-layer interference, using a chronological scenario method rather than isolated snapshots. The timeline is traced from cold boot, through handshake attempts and protocol desynchronization under active jamming, to eventual recovery via the C2P-ARQ self-healing path. The objective is to validate liveness and fault-tolerance properties in an adversarial environment where cryptographic correctness alone is insufficient as evidence of operational continuity.

Two synchronized instruments are used throughout all scenarios. The oscilloscope provides the physical-layer view, exposing voltage-level interference and real-time line inversions. The logic analyzer provides the data-link/protocol view, revealing whether observable edge patterns can be interpreted as valid framed traffic by the receiver parser. The combined methodology establishes a direct causal chain between electrical corruption and protocol-level failure or recovery.

### 8.3.1 Scenario 1: System Initialization Under Active Jamming

At time  $t = 0$ , both nodes are powered on while the attacker channel is already active. Thus, the first protocol opening phase is attempted under immediate disturbance, rather than under a clean baseline. On the transmitter side (Node A), the C2P-ARQ opening path remains confined to the handshake loop (SENDING\_HELLO/WAITING\_HELLO\_ACK), and, upon repeated ACK timeout or rejection, re-enters RECONNECTING according to the retry budget ( $\text{MAX\_RETRIES} = 50$ ) and reconnect timer ( $\text{RECONNECT\_INTERVAL\_MS} = 2000\text{ms}$ ). No authenticated session is established, and no forward progress to data delivery occurs.

On the receiver side (Node B), no valid preamble is accumulated at parser level (WAIT\_AA/WAIT\_55), while the displayed RX FSM remains in synchronization-wait conditions (WAITING\_SYNC\_1). In other words, the channel carries transitions, but not frame boundaries acceptable to the SYNC\_BYTE\_1/SYNC\_BYTE\_2 gate. This is the expected boot-time failure mode under continuous jamming: parser activity without protocol admission.

The physical mechanism is captured by the XOR injection model

$$S_{\text{rx}}(t) = S_{\text{tx}}(t) \oplus N(t)$$

where  $S_{\text{tx}}(t)$  is the clean UART waveform from Node A,  $N(t)$  is the attacker disturbance process, and  $S_{\text{rx}}(t)$  is the waveform observed at Node B input. Whenever

$$N(t) = 1$$

the instantaneous UART level is inverted; therefore, even transient high intervals in  $N(t)$  can invalidate the start/data/stop structure of asynchronous serial symbols before any protocol-layer validation is reached.

Figure 8.4 shows the complete bench during the boot phase under attack. The transmitter has not obtained a valid acknowledgement path for session opening, and the receiver remains synchronization-bound. This global view confirms that the experiment starts in the strongest adversarial condition: there is no clean-channel warm-up interval preceding protocol activity.

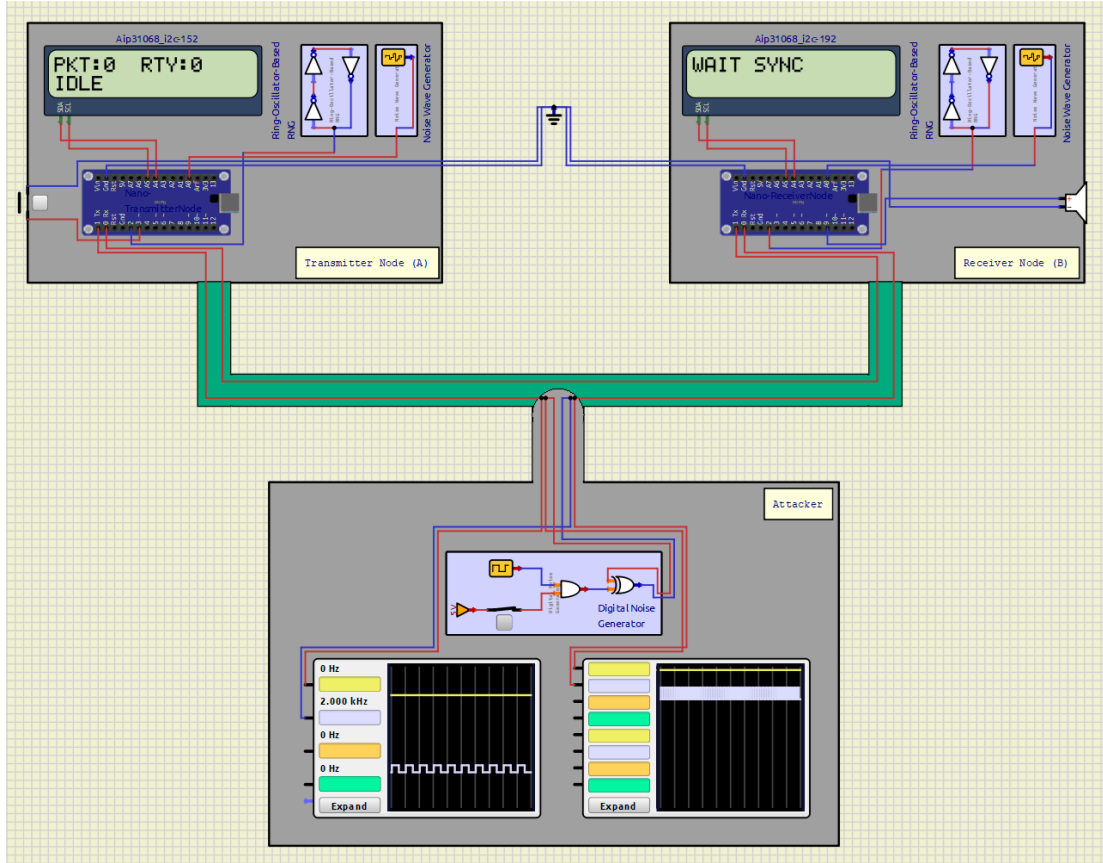


Figure 8.4: Full SH2SC-EDT boot-time topology under active jamming. The attacker sub-circuit is enabled from startup, and both endpoints begin operation with a disturbed forward channel.

Figure 8.5 provides the physical-layer evidence. The oscilloscope trace shows persistent disturbance energy on the monitored line, with occasional transmitter-driven assertions embedded in the same interval. The key observation is not mere amplitude variation but temporal contamination: symbol windows that should remain electrically stable over UART bit periods are repeatedly crossed by interference events, making clean byte reconstruction statistically implausible at startup.

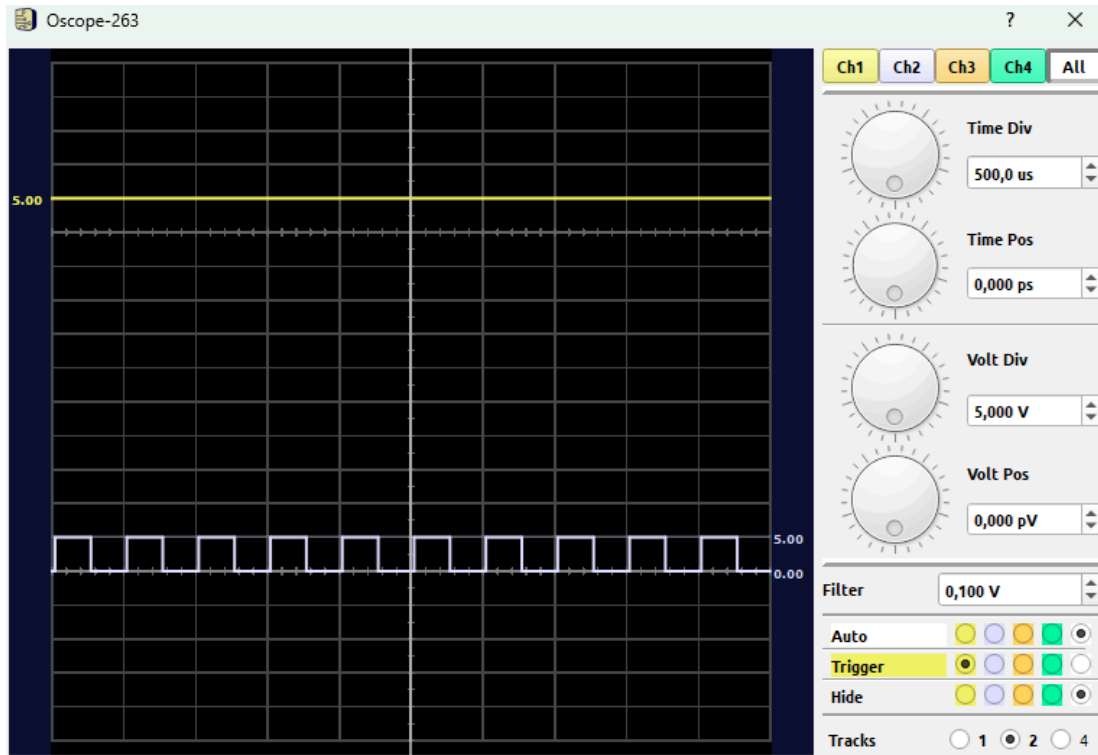


Figure 8.5: Oscilloscope evidence of continuous interference at boot. The disturbed line remains dominated by attacker-driven activity while legitimate transmitter assertions are intermittently superposed.

Figure 8.6 provides the corresponding protocol-side consequence. The logic analyzer interprets the disturbed waveform as rapid, non-semantic transitions that do not stabilize into a valid frame grammar. As a result, the receiver cannot lock on the preamble gate (0xAA 0x55), and the parser cannot enter a sustained payload acquisition path. The combined evidence of Figure 8.5 and Figure 8.6 demonstrates a strict physical-to-protocol causality: voltage-level inversions induced by active jamming are sufficient to prevent session bootstrap at the data-link boundary.

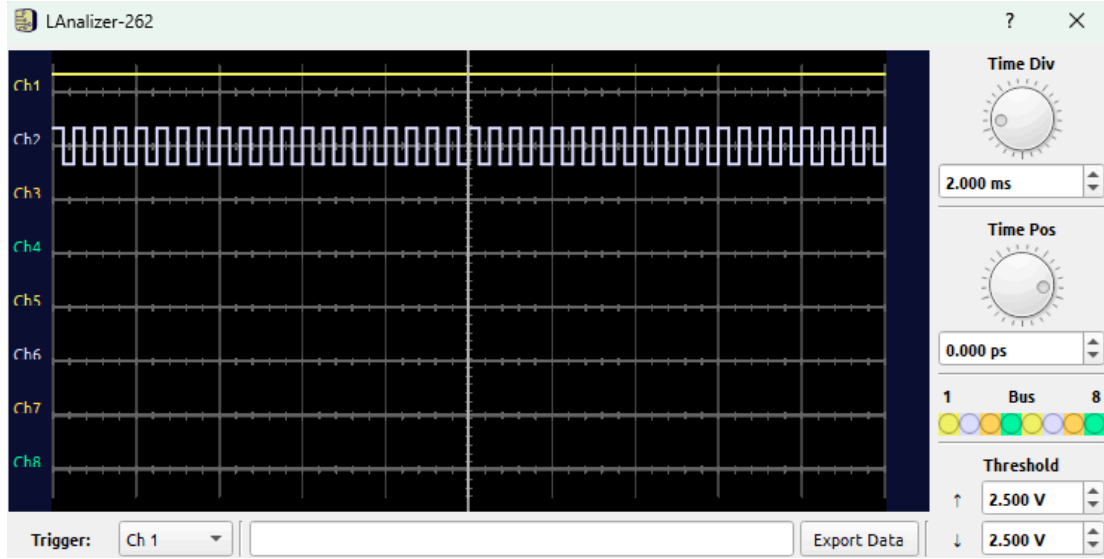


Figure 8.6: Logic-analyzer view of the same boot interval. Rapid non-semantic state transitions dominate the captured stream, preventing reliable preamble acquisition and packet framing.

### 8.3.2 Scenario 2: Baseline Payload Transmission (Clear Channel)

After the boot-time desynchronization observed in Scenario 1, the attacker is temporarily disabled by opening the noise-injector switch. In this configuration, the Digital Noise Generator is OFF and the disturbance process is quiescent:

$$N(t) = 0$$

The purpose of this scenario is to establish a baseline of how a legitimate, encrypted C2P-ARQ DATA delivery manifests at both the physical layer and the protocol layer when the channel is clean and the endpoints are already synchronized.

In the baseline state, Node A (TX) transmits an encrypted data packet containing an application payload, and Node B (RX) receives it successfully, performs authenticated decryption, executes the decoded command, and responds with an acknowledgement. Figure 8.7 captures the global topology of this condition.

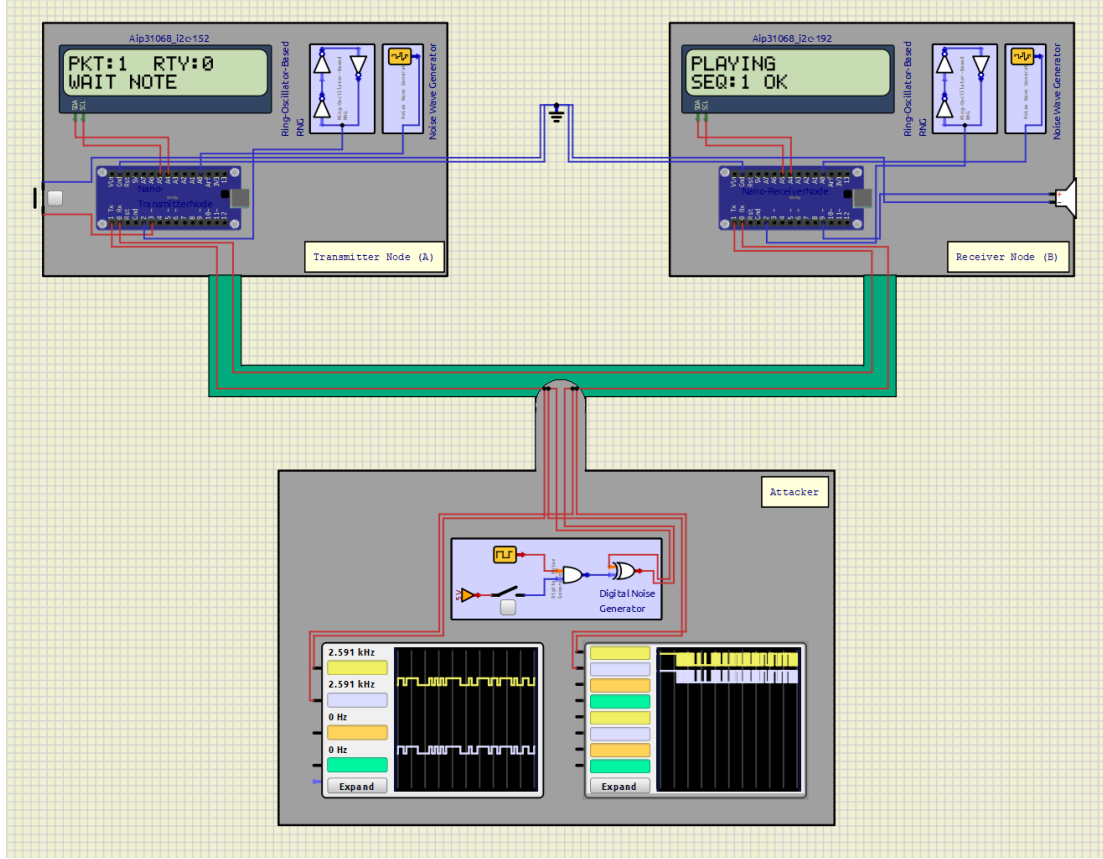


Figure 8.7: SH2SC-EDT topology during baseline payload transmission. The attacker switch is open, the noise path is electrically disabled, and Node A and Node B communicate without interference.

The topology view makes the clean-channel assumption explicit. With the switch open, the injected-noise input to the XOR stage is held low; the forward UART conductor therefore carries the transmitter waveform without attacker-induced inversions. This isolates protocol behavior from physical-layer corruption and provides a reference trace against which subsequent attack-enabled scenarios can be compared.

The physical-layer manifestation of this baseline is shown by the oscilloscope capture in Figure 8.8. Under the XOR injection model used throughout this section, the received waveform is

$$S_{\text{rx}}(t) = S_{\text{tx}}(t) \oplus N(t) = S_{\text{tx}}(t) \oplus 0 = S_{\text{tx}}(t)$$

so the channel is electrically transparent. The oscilloscope trace exhibits clean, sharp square-wave transitions between 0 V and 5 V, and stable symbol intervals with no bit-level polarity inversions superposed onto the UART stream.

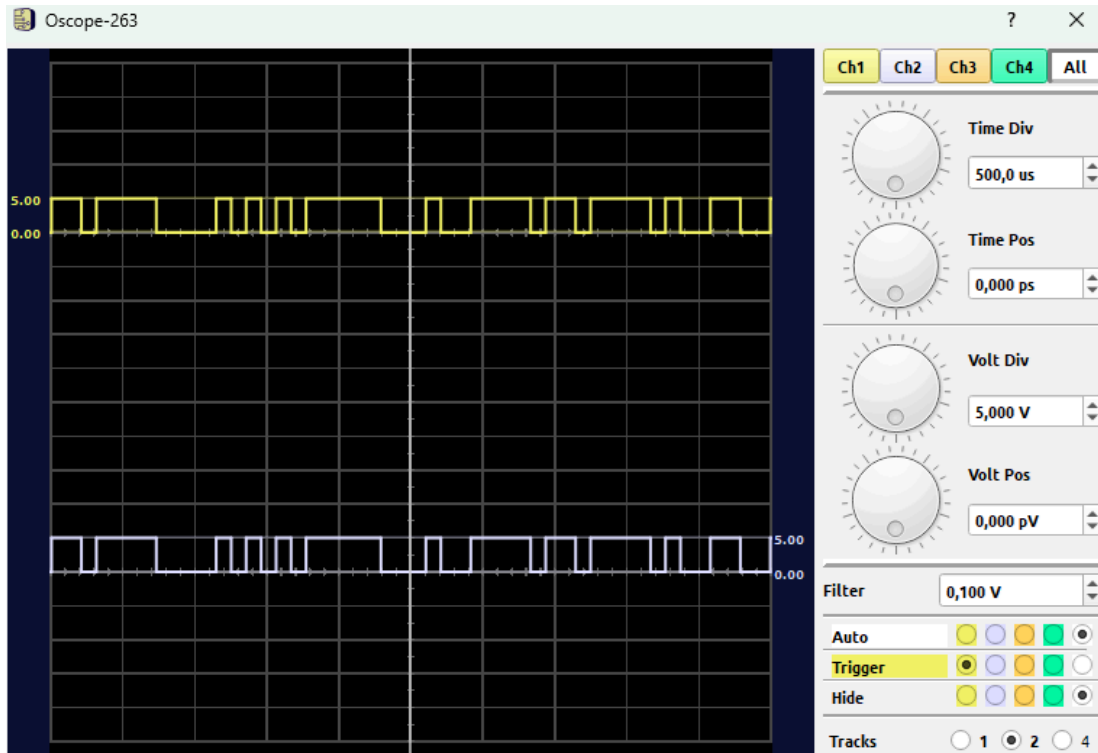


Figure 8.8: Oscilloscope evidence of baseline operation with the noise path disabled. The XOR injector passes the UART signal unchanged, producing a clean square-wave waveform without attacker-driven inversions.

At the protocol layer, this physical cleanliness corresponds to a successful framed exchange: Node A emits a complete DATA packet, and Node B responds with an ACK after receiving and executing the decrypted payload. The message flow is summarized in Figure 8.9.

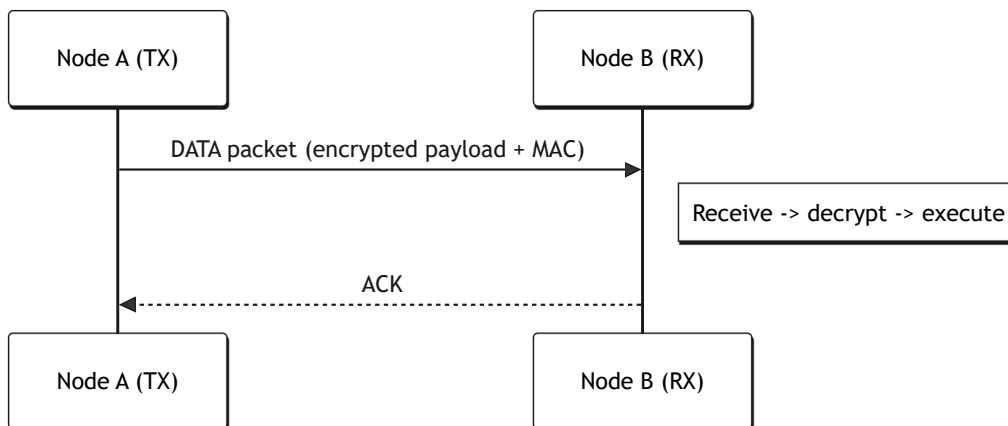


Figure 8.9: Successful C2P-ARQ DATA-to-ACK exchange in a clean channel. The receiver acknowledges only after it has received and executed the authenticated decrypted payload.

The Figure 8.10 provides the corresponding logic-analyzer view. The analyzer captures a continuous, well-framed data transmission in which a static preamble is followed by the packet body containing an encrypted payload and its MAC tag. This trace establishes the visual baseline signature of a valid C2P-ARQ frame in the absence of jamming: stable preamble acquisition, uninterrupted byte flow across the packet body, and a coherent packet structure that can be parsed and validated end-to-end by the receiver.

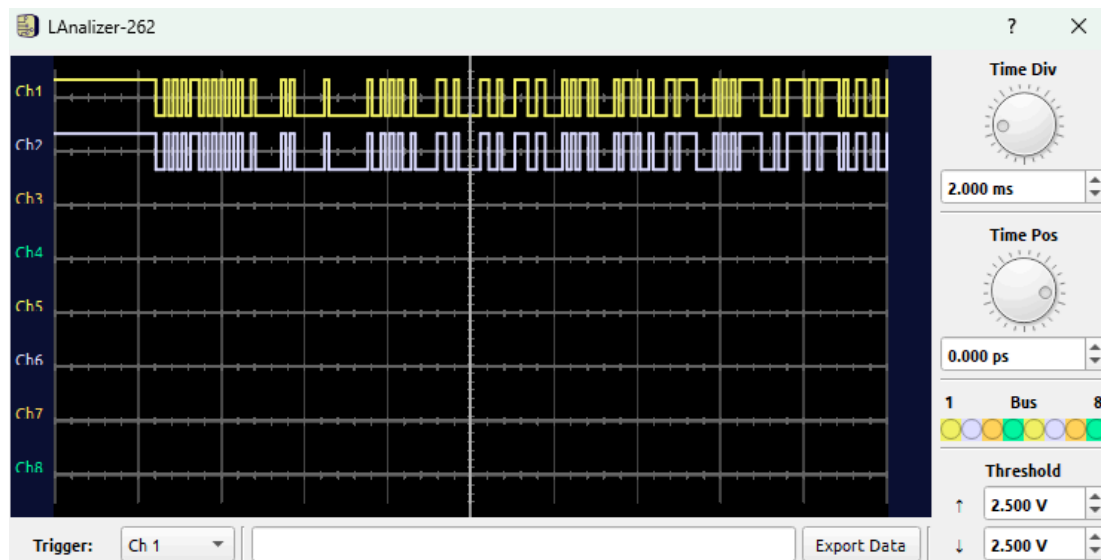


Figure 8.10: Logic-analyzer baseline of a valid C2P-ARQ data frame in a clear channel. A static preamble is followed by the encrypted payload and MAC tag, enabling seamless frame capture and validation.

### 8.3.3 Scenario 3: Cryptographic Variance in Subsequent Packets (Clean Channel)

With the Digital Noise Generator still disabled, the transmitter proceeds to emit the next sequential DATA packet over the same clean forward channel. Node A and Node B remain synchronized, and the exchange completes successfully, confirming that any observed differences in the captured packet waveform are not attributable to physical-layer interference but to protocol-level cryptographic variance.

The Figure 8.11 provides the global topology for this second clean-channel delivery. The attacker switch remains open, and the node LCDs confirm forward progress to the subsequent packet: Node A reports an incremented packet index with zero retries (PKT:2, RTY:0) while Node B confirms authenticated reception (SEQ:2 OK) during payload execution.



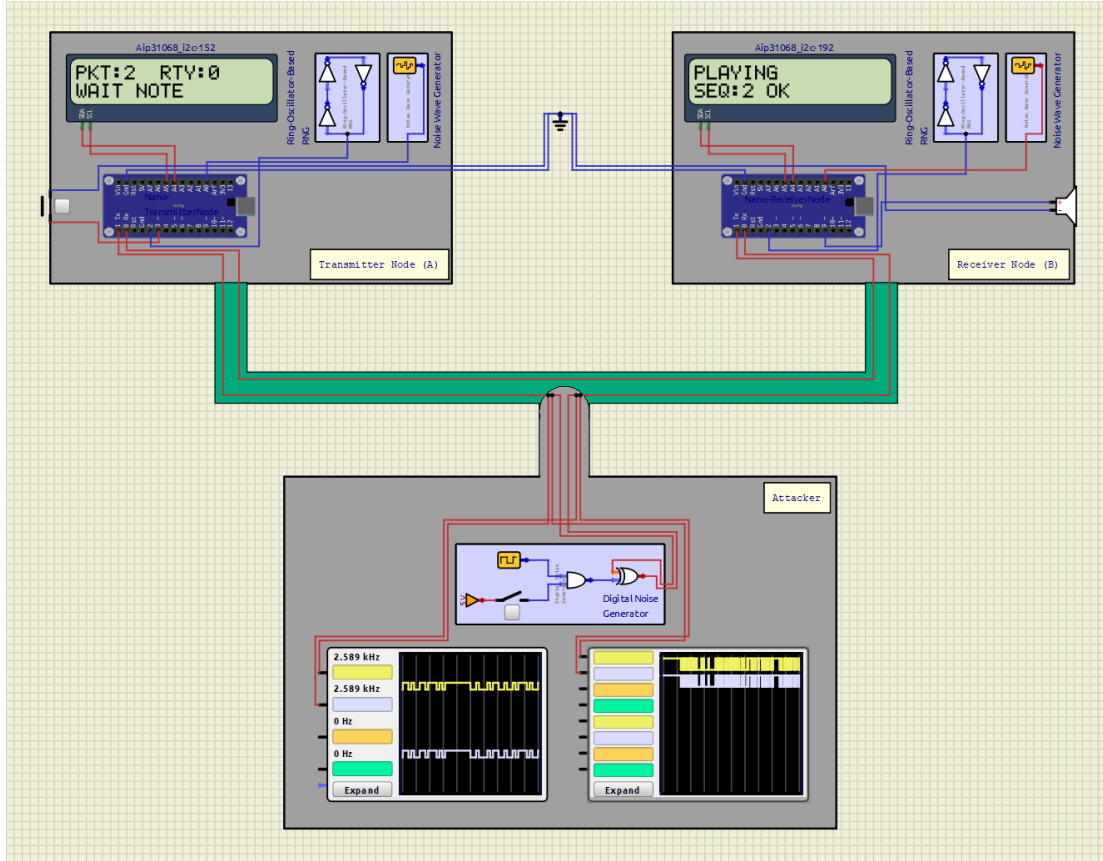


Figure 8.11: SH2SC-EDT topology during the next sequential clean-channel DATA delivery. The attacker remains disabled, and the LCD state confirms incremented packet progression with no retransmissions.

The physical-layer condition remains stable in Figure 8.12. The oscilloscope capture shows a clean UART waveform consistent with the baseline in Figure 8.8: sharp transitions and stable logic levels without disturbance-driven polarity inversions. This observation constrains the interpretation of subsequent protocol traces: the channel is electrically reliable, so any divergence in the data pattern observed at the logic analyzer must arise from the packet contents rather than from noise-induced corruption.

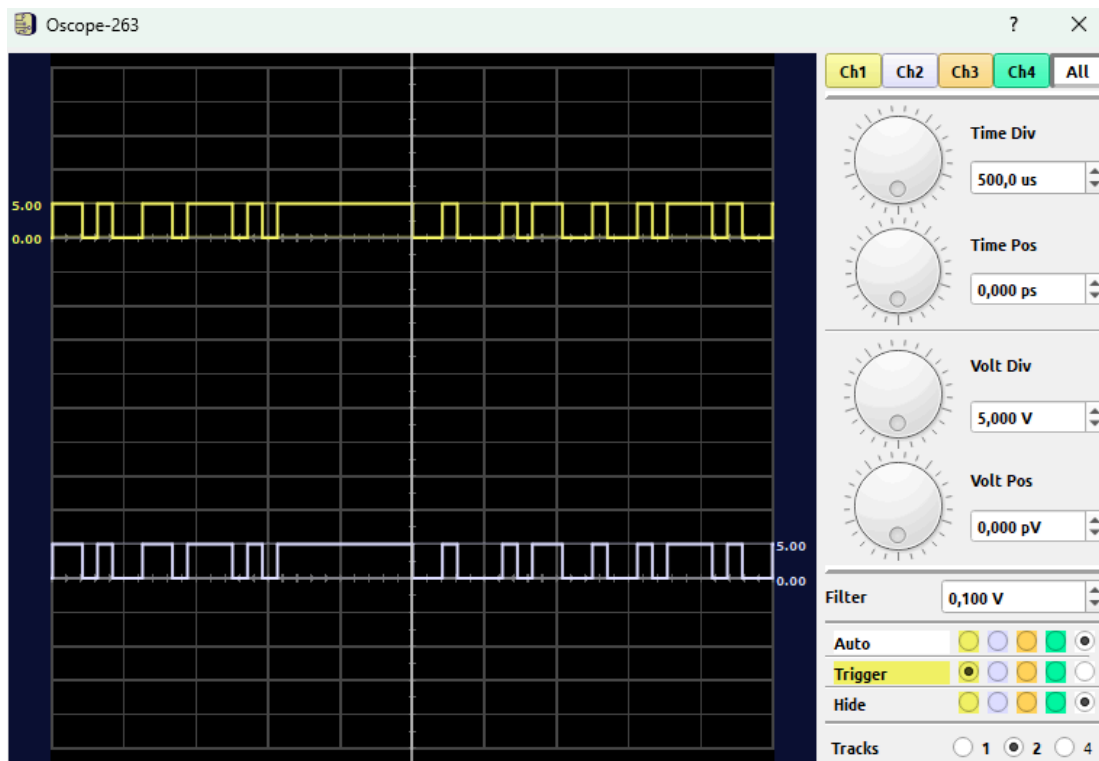


Figure 8.12: Oscilloscope evidence of continued clear-channel operation during the next DATA packet. The physical layer remains clean and stable, isolating packet-level differences to protocol behavior rather than interference.

The cryptographic effect is exposed most clearly by the logic-analyzer capture in Figure 8.13. When compared directly against the prior clean-channel packet (Figure 8.10), the preamble remains visually identical, as expected: this synchronization pattern is intentionally static to support deterministic frame acquisition at the receiver. In contrast, the post-preamble region exhibits a markedly different high/low sequence with no visually apparent correlation to the previous packet's payload/MAC segment.

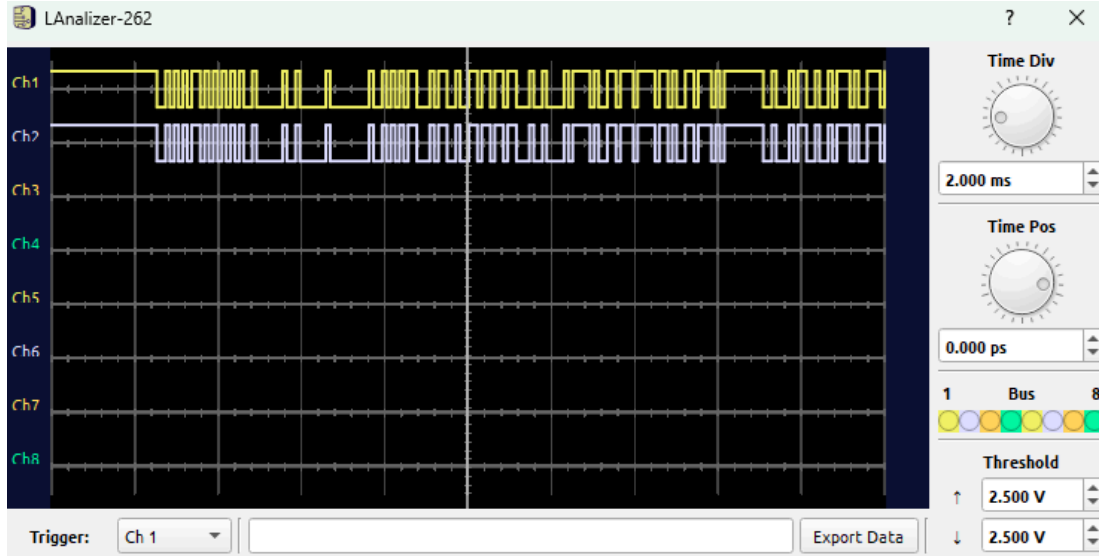


Figure 8.13: Logic-analyzer capture of the next sequential DATA frame under clear-channel conditions. The preamble matches the prior packet, while the ciphertext and MAC region exhibits no visible correlation, consistent with ChaCha20-Poly1305 semantic security under a nonce change.

This lack of observable structure is the expected manifestation of semantic security at the data-link layer. Even if the underlying application command size is unchanged between adjacent packets, the protocol’s incremented sequence number induces a distinct per-packet nonce, yielding a new ChaCha20 keystream and, consequently, a different ciphertext for the encrypted payload. Because the Poly1305 tag is computed over the authenticated header and the ciphertext, the MAC bytes likewise change across packets. The resulting post-preamble waveform therefore behaves as a one-time, packet-unique pattern to an external observer, preventing reliable pattern matching across sequential transmissions and undermining any attempt at correlation-based traffic inference on the forward channel.

### 8.3.4 Scenario 4: Active Jamming, Payload Corruption, and State Desynchronization

In this scenario, the attacker re-activates the Digital Noise Generator while payload packets are in flight. The forward channel is therefore driven by a non-quiescent disturbance process,

$$N(t) \neq 0$$

and the receiver input becomes an XOR-composed waveform rather than a direct representation of the transmitter line. The experiment exposes a critical failure mode in embedded communication under interference: the transmitter and receiver can enter divergent operational states even when both endpoints continue executing correctly according to their local logic.

The Figure 8.14 captures the global topology during this attack interval. Node A has advanced to packet (#145) and is observed in a `WAIT ACK` state with two retries already consumed (`RTY: 2`). Node B, however, is still executing the previously authenticated payload corresponding to packet (#144), indicated by `PLAYING SEQ:144 OK`. This simultaneous observation establishes state desynchronization: the transmitter is attempting to deliver the next stop-and-wait `DATA` frame, while the receiver remains engaged in application-layer execution of the prior command.

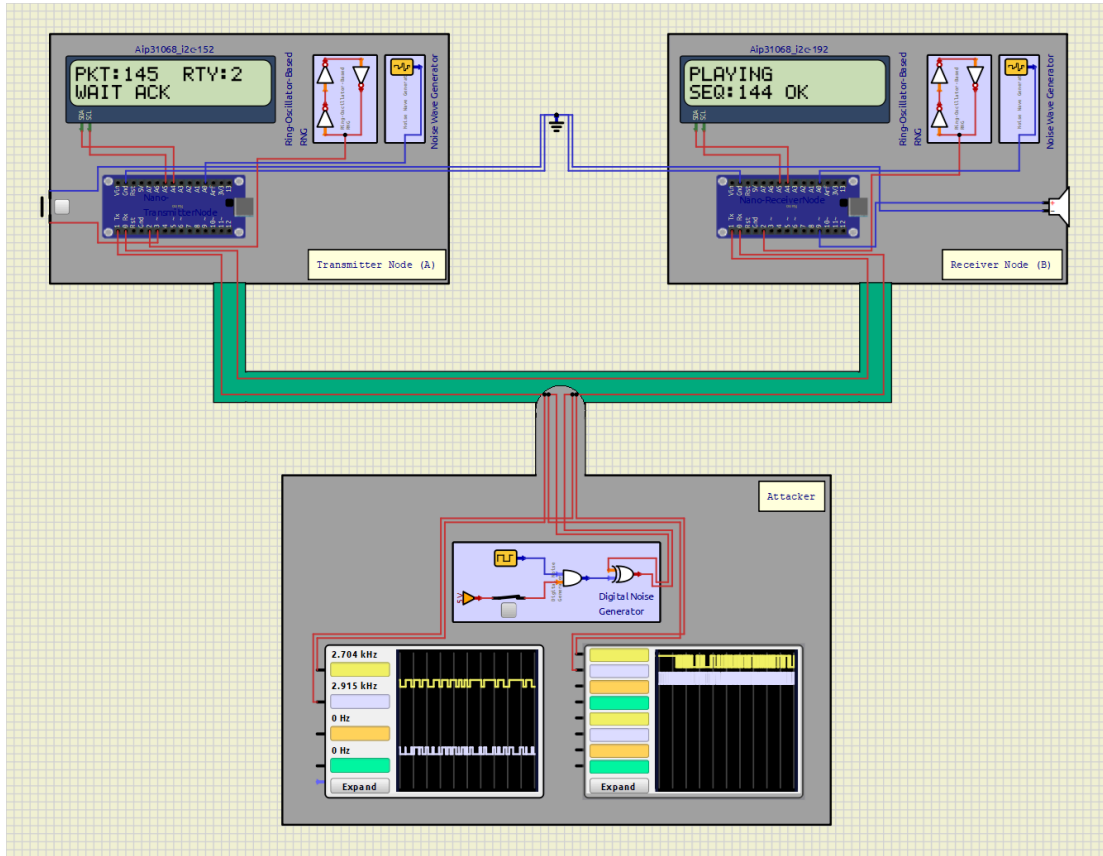


Figure 8.14: SH2SC-EDT topology under active jamming during payload delivery. Node A is stalled in `WAIT ACK` for `PKT:145` with `RTY:2`, while Node B is still in `PLAYING` state for `SEQ:144 OK`, evidencing protocol-state divergence under interference.

The physical-layer mechanism driving this divergence is shown in Figure 8.15. With the attacker enabled, the receiver-side waveform satisfies

$$S_{\text{rx}}(t) = S_{\text{tx}}(t) \oplus N(t)$$

so any interval in which

$$N(t) = 1$$

produces a polarity inversion of the instantaneous UART level. The oscilloscope trace exhibits a severely perturbed serial signal relative to the clean baseline: transitions occur within symbol windows that would otherwise remain stable, and the bit-level structure required for reliable asynchronous reconstruction is repeatedly violated. Under these conditions, correctly receiving a full preamble, header, ciphertext, and MAC as a single coherent frame becomes unlikely even if the receiver were exclusively dedicated to parsing.

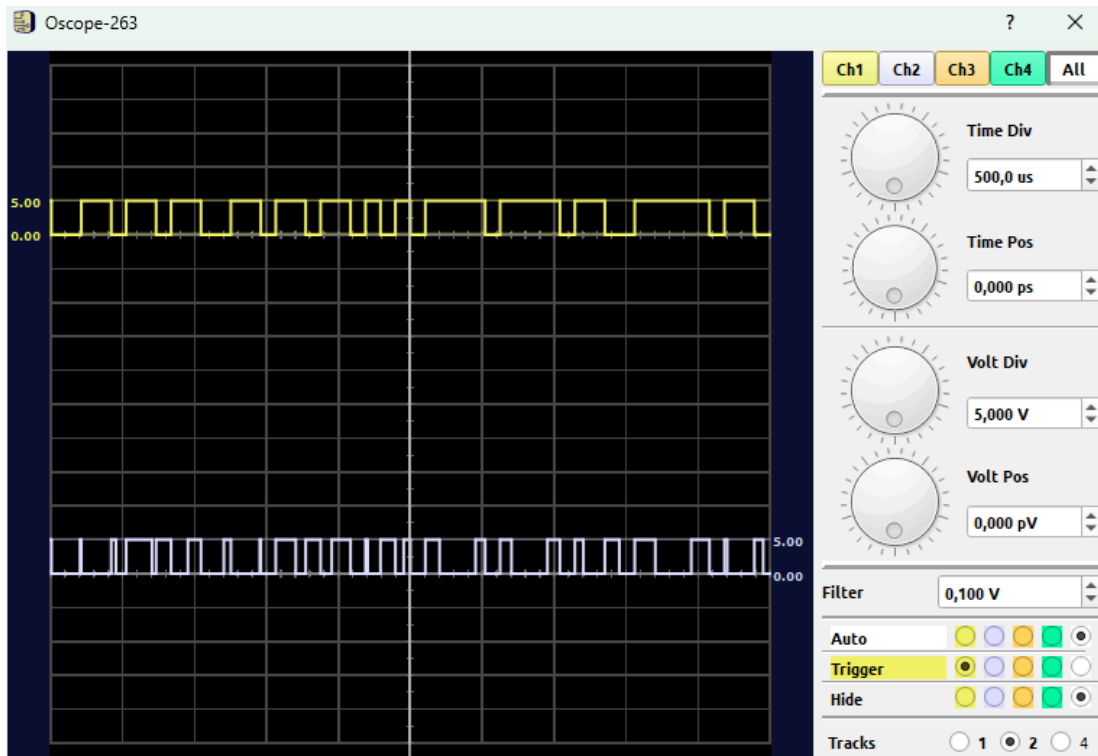


Figure 8.15: Oscilloscope evidence of physical-layer corruption under active jamming. XOR-injected disturbance introduces bit-level inversions into the forward UART waveform during DATA transmission.

The protocol-layer manifestation is captured by the logic analyzer in Figure 8.16. In contrast to the clean baselines (Figure 8.10 and Figure 8.13), the jammed trace does not exhibit a stable, interpretable packet body following a consistent preamble. Instead, the

captured high/low sequence is dominated by non-semantic transitions that erase the structural invariants required for framing and authenticated processing. This observation provides the key security-relevant conclusion of the scenario: the failure is not an AEAD authentication rejection of an otherwise well-formed ciphertext, but a prior destruction of packet integrity at the data-link boundary such that a valid frame may never be presented to the cryptographic verification stage.

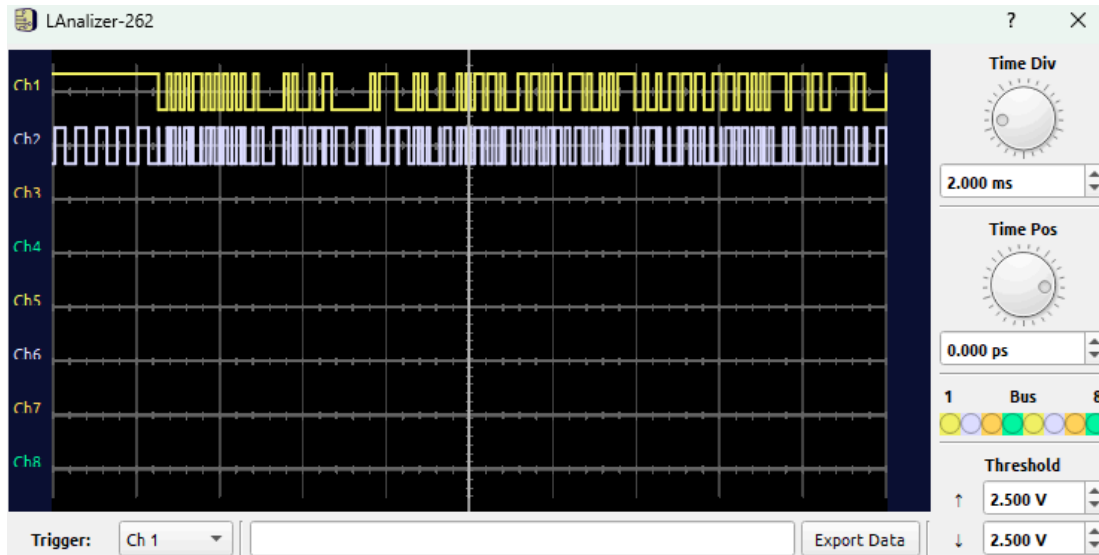


Figure 8.16: Logic-analyzer view of the jammed forward channel. The packet’s structural integrity is destroyed relative to the clean baseline, preventing stable framing and end-to-end validation.

The ARQ and application-layer consequences are visible directly on the node displays. Figure 8.17 shows Node A stalled on PKT:145 with RTY:2 in WAIT ACK, confirming that the stop-and-wait retransmission logic is actively attempting to push the same payload packet through a disturbed channel without receiving the required acknowledgement. Figure 8.18 shows Node B in PLAYING state with SEQ:144 OK, confirming that the receiver remains occupied with the prior authenticated command and has not progressed to the subsequent sequence value. Even if Node B were to complete its ongoing execution and resume active listening, the physical and logic-analyzer evidence (Figure 8.15 and Figure 8.16) indicates that the incoming attempts for packet (#145) are being shredded at the waveform level during this interval.

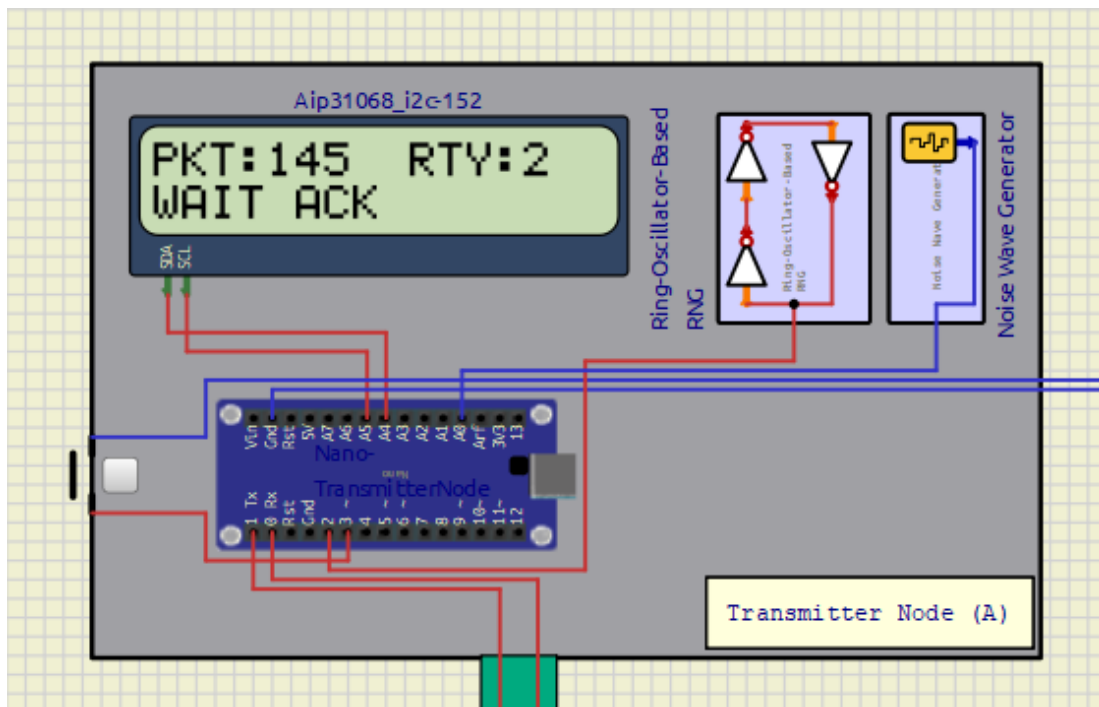


Figure 8.17: Node A status under active jamming. PKT:145 and RTY:2 in WAIT ACK indicate repeated retransmission attempts without acknowledgement.

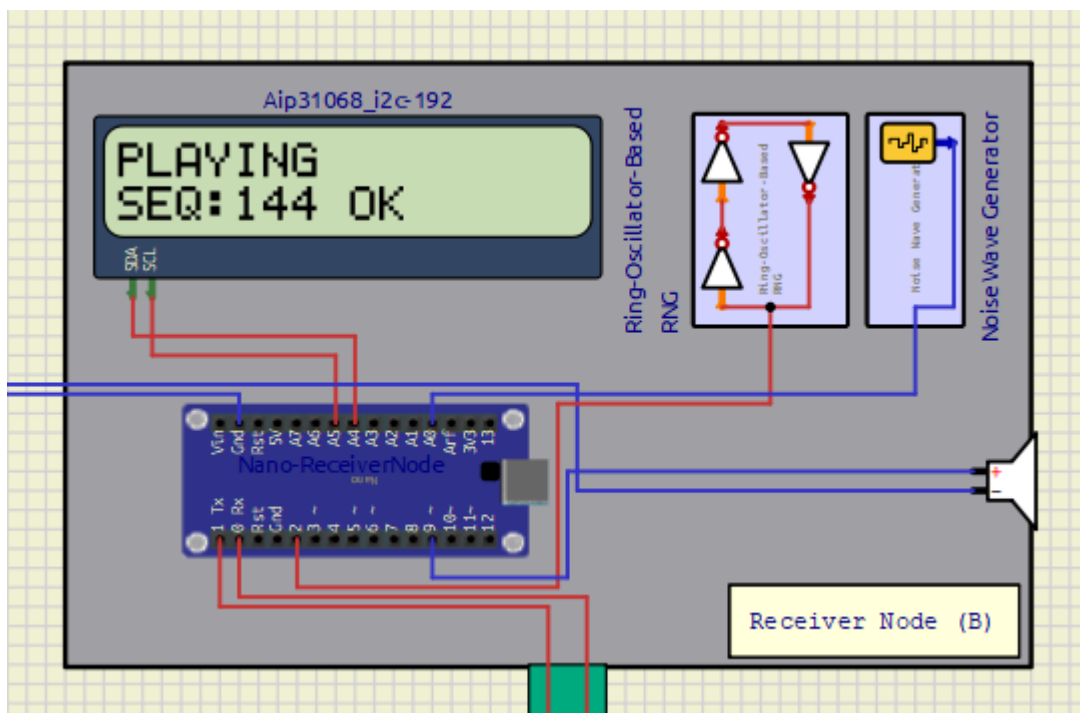


Figure 8.18: Node B status during the same interval. PLAYING SEQ:144 OK indicates ongoing execution of the previous authenticated payload while the transmitter has already advanced to the next packet.

The resulting interaction is summarized in Figure 8.19. While Node B remains in a prolonged execution window for packet (#144), Node A transmits packet (#145) and enters a `WAIT ACK` state. Each jammed transmission attempt fails to yield a valid acknowledgement, leading to sequential retries.

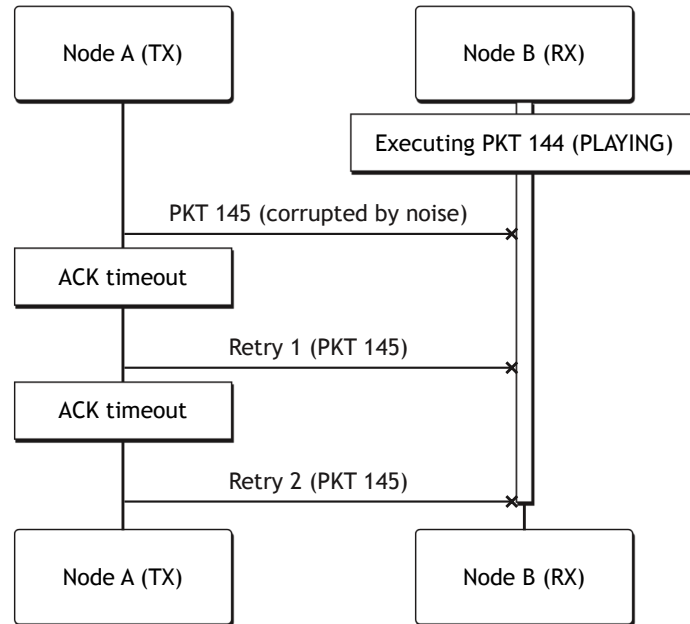


Figure 8.19: Desynchronization under active jamming. Node B remains occupied with PKT 144 while jammed transmissions of PKT 145 fail to produce an ACK, forcing repeated retries from Node A.

### 8.3.5 Scenario 5: Transmitter Timeout and Session Suspension

Active jamming persists without any change in the physical attack mechanism. The distinguishing event in this scenario is therefore not a new waveform signature but a protocol-level safeguard: the transmitter’s ARQ logic reaches its hardcoded retransmission bound for the pending DATA packet (`MAX_RETRIES = 50`). Rather than remaining indefinitely in a waiting state, the transmitter FSM treats this condition as a severe and sustained channel degradation and initiates a controlled session suspension.

On Node A, this transition is observable as an explicit teardown of the current cryptographic session followed by entry into the `RECONNECTING` state. Operationally, this behavior prevents a microcontroller-class transport engine from hanging in an infinite retry loop under a prolonged denial-of-service condition: once the retry budget is exhausted,



the system returns to a reconnection posture and can later re-establish a fresh session when the channel becomes usable again.

On Node B, no corresponding transition is observed at this moment. The receiver remains entirely engaged in executing the physical payload of the previously authenticated packet (#144), sustaining the ongoing delay/melody playback and not yet reaching any local timeout threshold. In this state, it is unaware that the transmitter has already aborted the session and moved to reconnection logic; the node remains locally correct with respect to its last verified command, while globally the session has already been torn down by the transmitter.

Oscilloscope and logic-analyzer figures are intentionally omitted for this scenario because the physical channel conditions remain identical to Scenario 4: the XOR noise injector continues to perturb the forward UART line in the same manner. The focus here is strictly the upper-layer timeout and state-machine response on the transmitter.

The transmitter-side transition into session suspension and reconnection posture is shown in Figure 8.20.

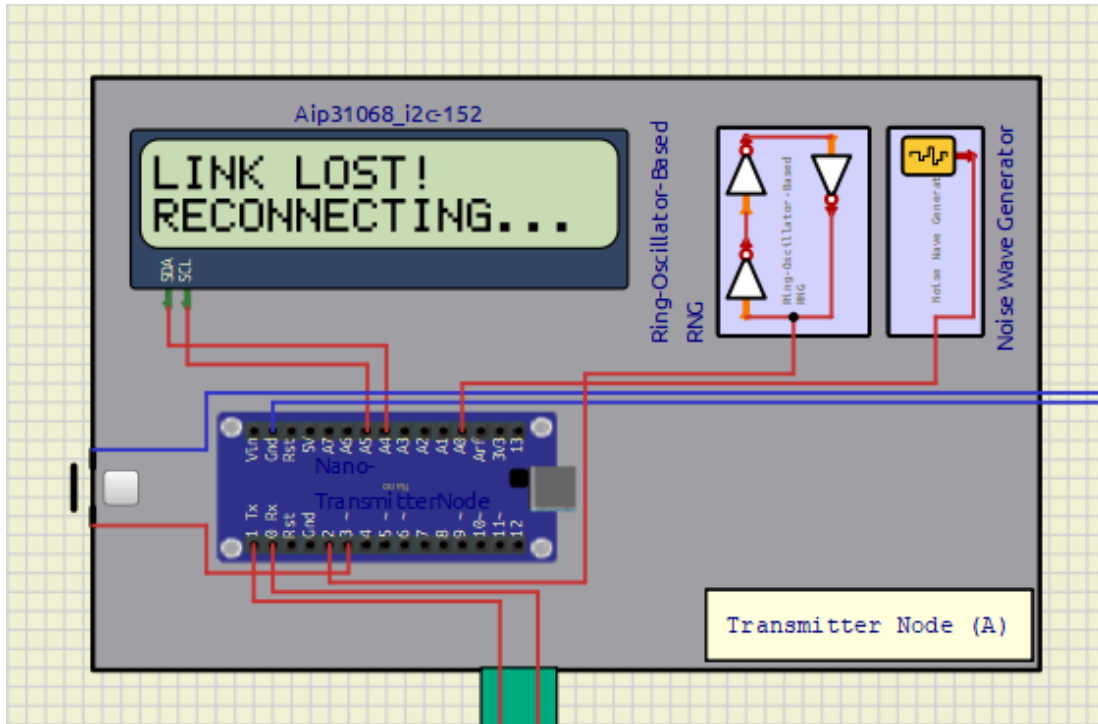


Figure 8.20: Node A enters the session-suspension safeguard after exhausting its retransmission budget. The LCD message LINK LOST! RECONNECTING... captures the moment the transmitter abandons the current session and transitions to reconnection logic under prolonged active jamming.

### 8.3.6 Scenario 6: Mutual Session Teardown and Dead Air

In this scenario, the Digital Noise Generator remains enabled, but the transport-layer interaction has transitioned into an interstitial state in which no valid protocol frames are present on the forward channel. The session has now been torn down at both endpoints: Node A has already suspended its session and remains in `RECONNECTING`, while Node B has returned from its long-running application-layer execution and has subsequently declared the session dead via its watchdog-driven fail-safe.

On the receiver (Node B), the blocking payload associated with packet (#144) has finally completed. When the node resumes network-layer listening, its internal watchdog expires because no authenticated packets have been received for the duration of the execution gap and subsequent dead interval. As a result, Node B drops the session gracefully, resets its cryptographic state, and enters the `TIMEOUT! DROP` status, indicating that session RAM and security context have been released in a controlled manner rather than remaining indefinitely allocated under attacker-induced silence.

On the transmitter (Node A), the state remains `RECONNECTING` as established in Scenario 5. In this posture, Node A is preparing to initiate a new session once the forward channel becomes usable. However, with the attacker still active, the medium remains dominated by disturbance energy, and there is no stable interval in which a valid frame exchange can be observed.

Only an oscilloscope capture is included for this intermediate state. Because neither node is transmitting valid C2P-ARQ frames, a logic analyzer would merely decode a sequence of random toggles that lacks preamble acquisition, header structure, and any meaningful data-link context. The oscilloscope, by contrast, directly exposes the physical condition of the line and can therefore demonstrate that the channel is saturated by adversarial interference even when legitimate endpoints are quiescent.

The Figure 8.21 provides the global topology view of the dead-air condition. The key observation is that the attacker sub-circuit remains enabled while neither endpoint maintains an active cryptographic session: the transmitter is in reconnection posture, and the receiver has reset the session state after concluding that the link is no longer live.

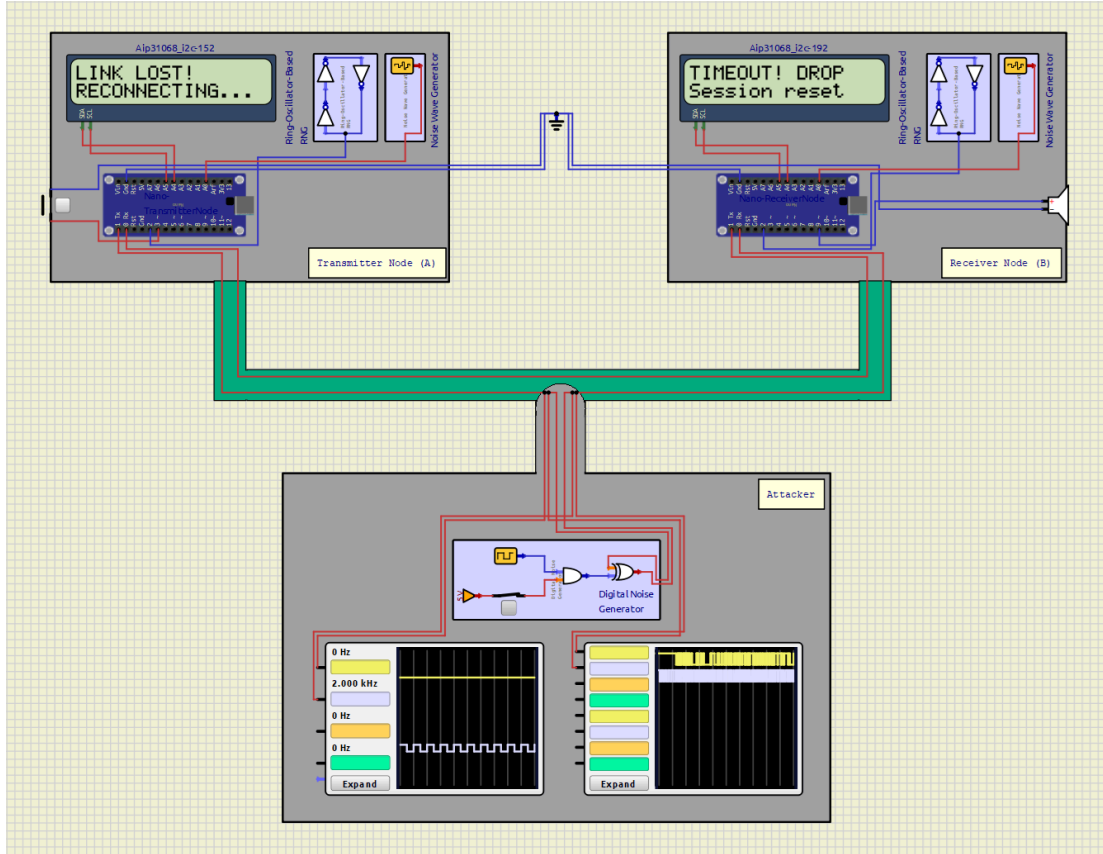


Figure 8.21: Dead-air topology under active jamming after mutual session teardown. The attacker remains enabled while Node A holds RECONNECTING posture and Node B has dropped the session after watchdog expiry.

The Figure 8.22 captures the receiver-side evidence of graceful teardown. The LCD message `TIMEOUT! DROP / Session reset` demonstrates that Node B's fail-safe mechanism has actively discarded the stale session context and cleared its cryptographic state, preventing indefinite retention of a compromised or unreachable session under prolonged jamming.

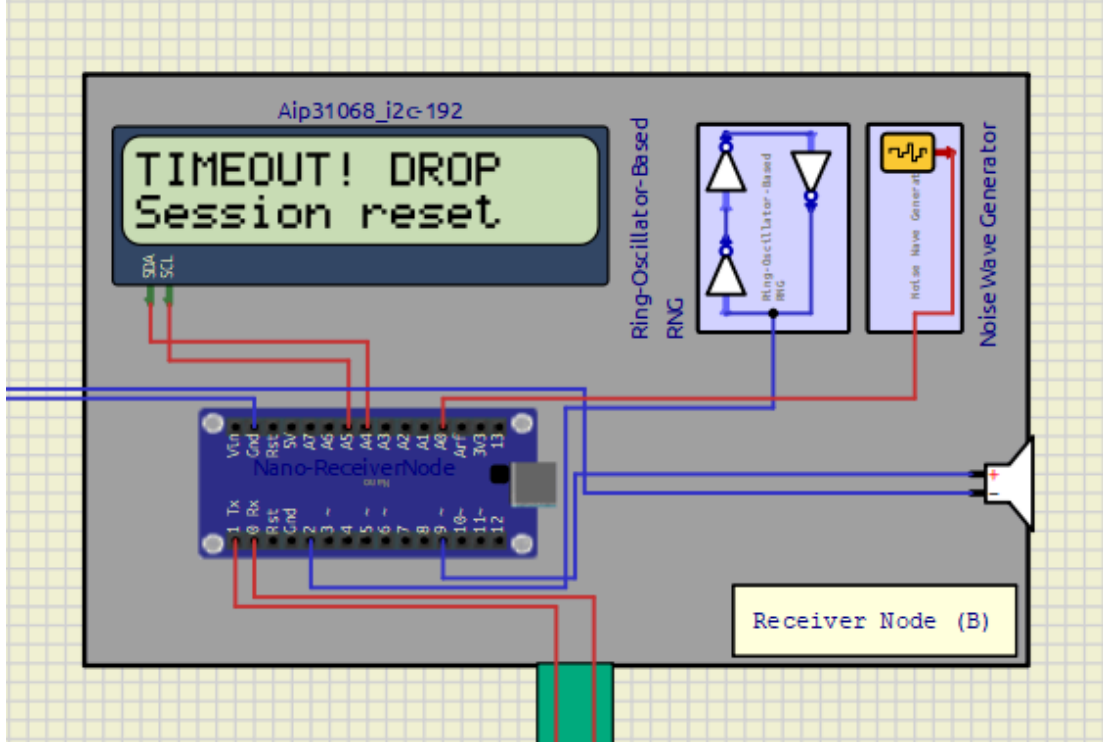


Figure 8.22: Node B indicates watchdog-driven session teardown. The LCD message `TIMEOUT! DROP / Session reset` confirms that the receiver has cleared the abandoned session and returned to a safe listening posture under continued channel denial.

The physics of this dead-air channel is captured by the XOR injection model under an idle transmitter. If the transmitter output is quiescent and held at a constant logic level,

$$S_{tx}(t) = \text{Const}$$

then the receiver-side line becomes

$$S_{rx}(t) = \text{Const} \oplus N(t)$$

where  $N(t)$  is the attacker disturbance process delivered by the Digital Noise Generator. Because *Const* carries no information-bearing transitions, any observed edge activity at the receiver input is attributable solely to  $N(t)$ . The Figure 8.23 confirms this condition visually: the oscilloscope trace is dominated by continuous interference, demonstrating that the physical medium is filled exclusively with adversarial energy during this interstitial state.

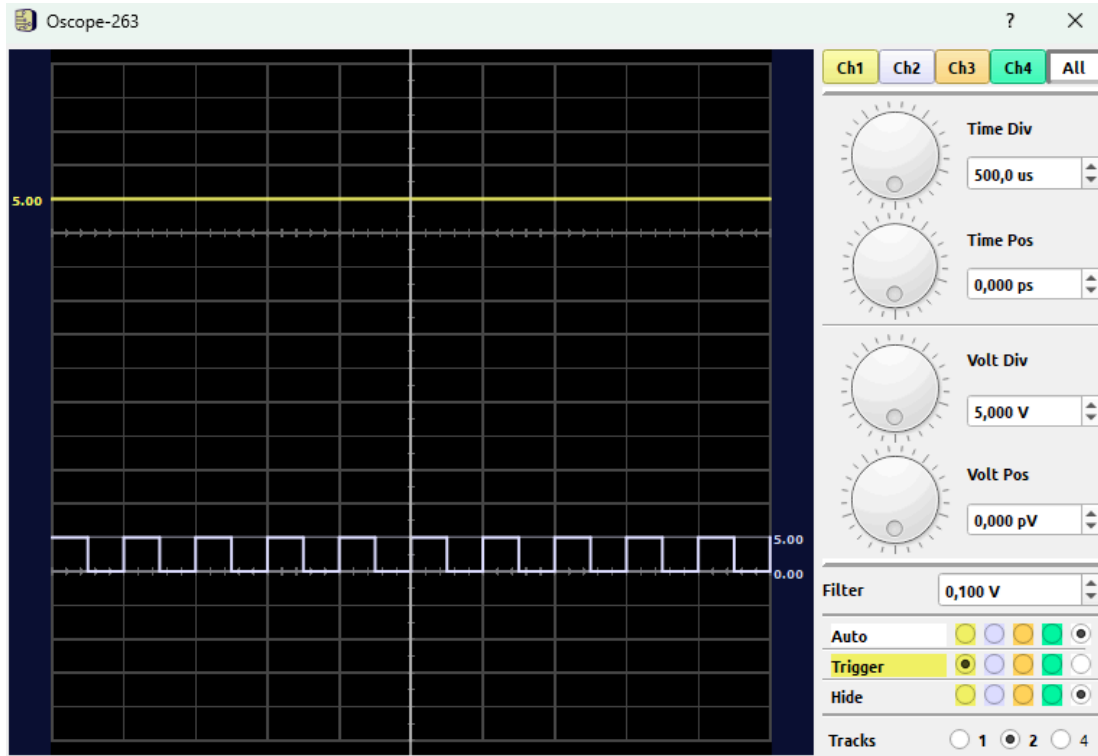


Figure 8.23: Oscilloscope evidence of dead air dominated by the attacker. With the transmitter held at a constant idle level, the XOR injector passes the disturbance process to the receiver input, yielding a channel waveform whose transitions are exclusively noise-driven.

### 8.3.7 Scenario 7: Session Re-initialization Attempts Under Active Jamming

After the mutual session teardown established in Scenario 6, SH2SC-EDT transitions into its self-healing reconnection path while the Digital Noise Generator remains enabled. The forward channel therefore remains subject to continuous XOR-injected disturbance, and the system must attempt session re-initialization without any clean physical-layer interval.

On the transmitter (Node A), the previous session state has been cleared and a new cryptographic session is requested. This transition is evidenced by a reset of the displayed packet counter to zero (PKT:0), and by the transmitter repeatedly emitting the C2P-ARQ `HelloPacket` while waiting for a handshake response (WAIT SYN). The retry counter has already advanced to RTY:21, indicating a persistent failure to obtain a valid acknowledgment under sustained interference.

On the receiver (Node B), the node remains in the disconnected drop state (TIMEOUT! DROP). Because the forward channel is still jammed, the receiver cannot reliably acquire and decode the incoming `HelloPacket` frames and therefore cannot install a new session

nonce or respond with an acknowledgement. The resulting system behavior is a stable, deterministic reconnection loop: session re-initialization is attempted continuously by Node A, while Node B remains safely reset and uncommitted until a valid session request is observed.

The Figure 8.24 provides the topology overview of this reconnection interval. The jammer is explicitly active, and both endpoints are in post-teardown conditions: Node A is attempting a new handshake, while Node B is in a fail-safe state that prevents it from acting on unauthenticated or structurally corrupted traffic.

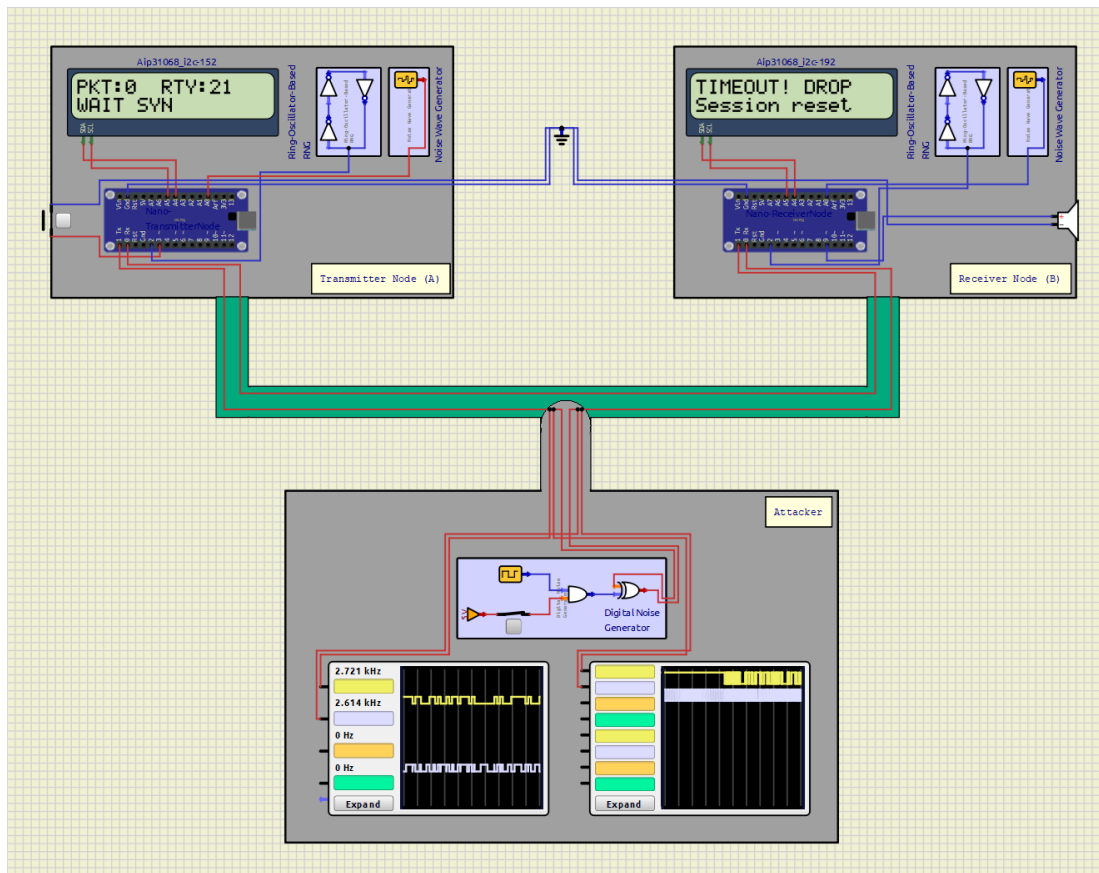


Figure 8.24: Topology during session re-initialization attempts under active jamming. The attacker remains enabled while Node A repeatedly transmits HelloPacket frames and Node B remains in a dropped session state.

The Figure 8.25 ties the reconnection attempt to the explicit FSM-visible state of both nodes. The transmitter display confirms that this is not a continuation of the prior data stream but a new session request (counter reset), and that repeated timeouts have already accumulated. Symmetrically, the receiver display confirms that the watchdog-driven teardown has left the node in a safe disconnected posture until a valid session-open frame can be parsed.

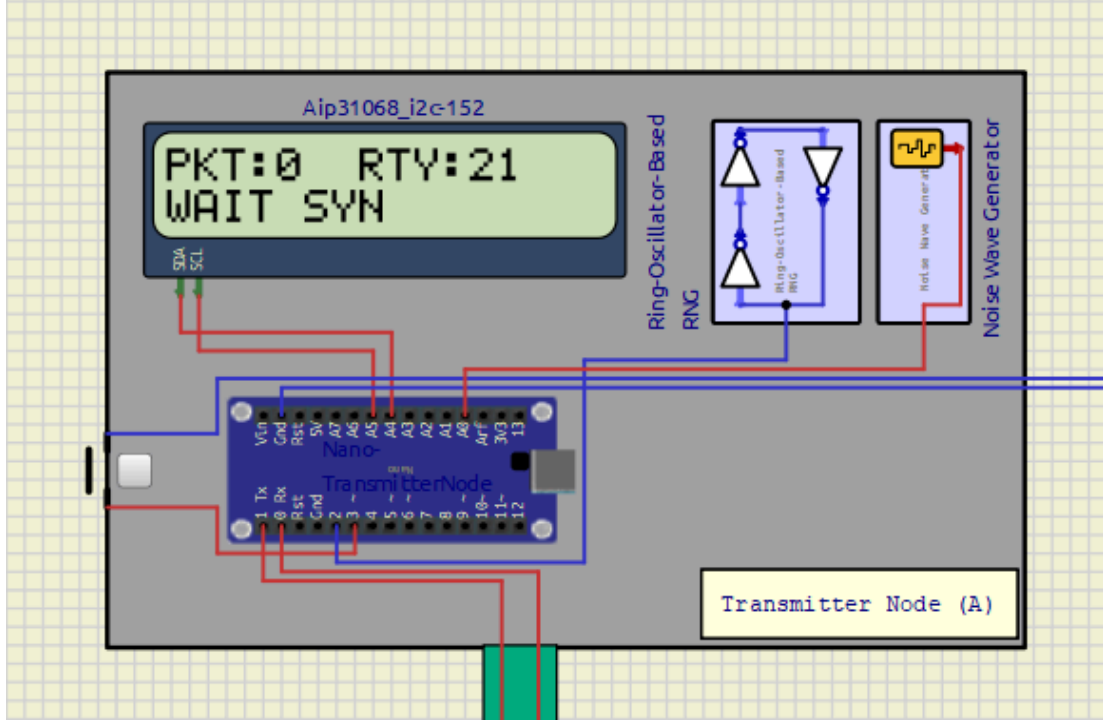


Figure 8.25: Endpoint states under reconnection pressure. Node A displays PKT:0 RTY:21 WAIT SYN, confirming a fresh session attempt and repeated retries, while Node B remains in TIMEOUT! DROP, indicating that no new session has been admitted.

At the physical layer, the transmitter is no longer quiescent as in Scenario 6. Instead, it is actively emitting `HelloPacket` waveforms into a channel that remains dominated by the attacker disturbance process. Under the XOR injection model, the receiver-side signal satisfies

$$S_{rx}(t) = S_{tx_{hello}}(t) \oplus N(t)$$

where  $S_{tx_{hello}}(t)$  is the clean UART waveform corresponding to repeated `HelloPacket` transmissions and  $N(t)$  is the attacker-generated disturbance.

The Figure 8.26 shows the expected consequence of

$$N(t) \neq 0$$

during active transmission: the clean, periodic structure of asynchronous serial bits is repeatedly violated by interference-driven inversions, so the receiver-side waveform no longer preserves the stable per-bit windows required for deterministic decoding.

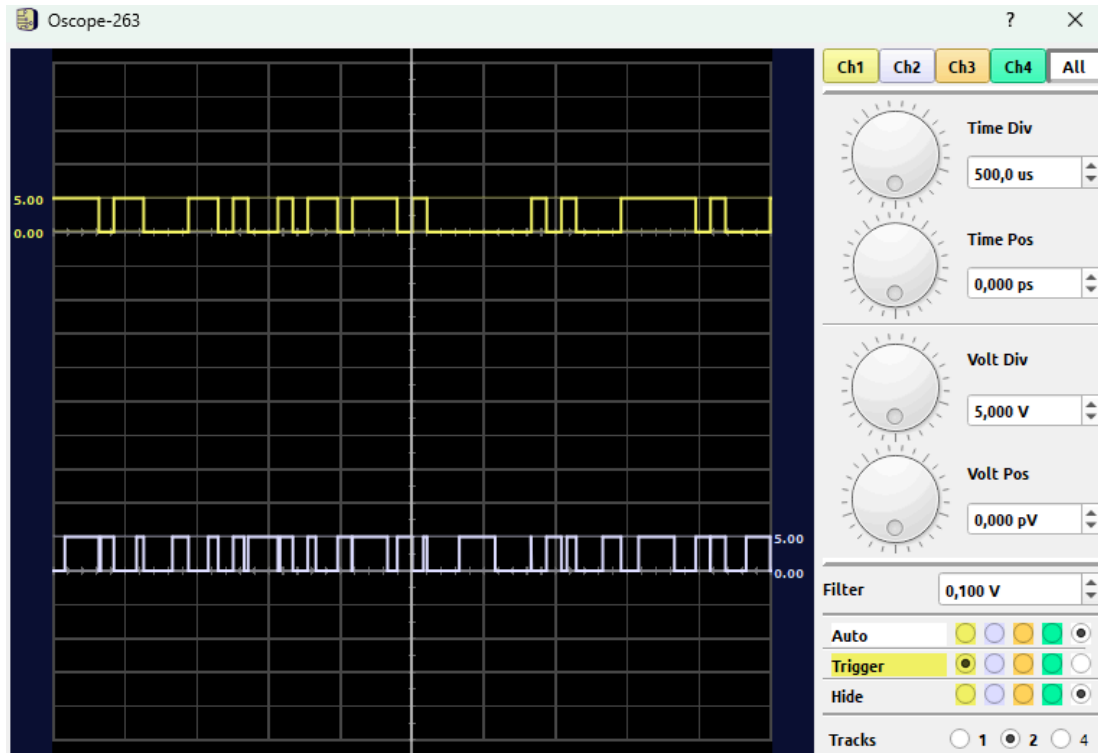


Figure 8.26: Oscilloscope evidence of HelloPacket transmissions under continued jamming. The active handshake waveform is severely distorted by XOR-injected disturbance, preventing stable symbol reconstruction at the receiver input.

The Figure 8.27 provides protocol-structure validation despite the hostile channel. Relative to the encrypted DATA packets captured earlier in Figure 8.10 and Figure 8.13, the observed frame is shorter in duration, consistent with the HelloPacket design (preamble plus a compact session-request body, without the ciphertext-and-MAC tail of a DATA frame). Crucially, the preamble pattern remains visually preserved and invariant across packet types. This invariance is a structural requirement of C2P-ARQ: by fixing the preamble, the receiver parser retains a deterministic synchronization gate even when higher-layer fields are corrupted by noise.



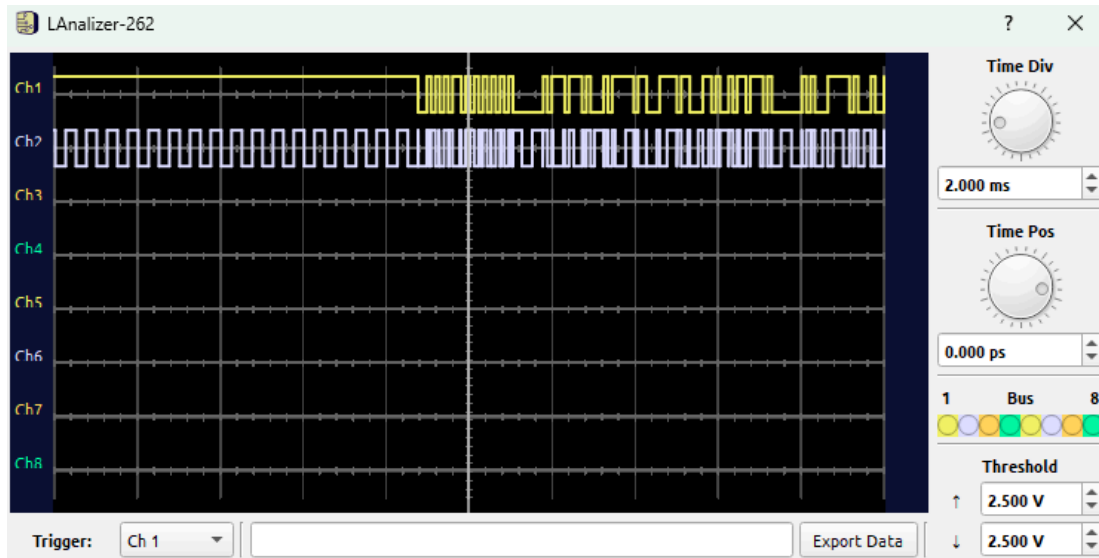


Figure 8.27: Logic-analyzer view of repeated HelloPacket attempts under jamming. The overall frame duration is visibly shorter than the encrypted DATA frames of Scenarios 2 and 3, while the synchronization preamble remains strictly identical, physically validating the C2P-ARQ packet specification.

The reconnection loop is summarized in Figure 8.28. Node A repeatedly transmits HelloPacket session requests; each attempt is physically corrupted by ongoing jamming and fails to elicit an acknowledgement, causing an ARQ timeout and an increment of the retry counter. Node B remains disconnected and therefore emits no response until a valid session-open frame is admitted.

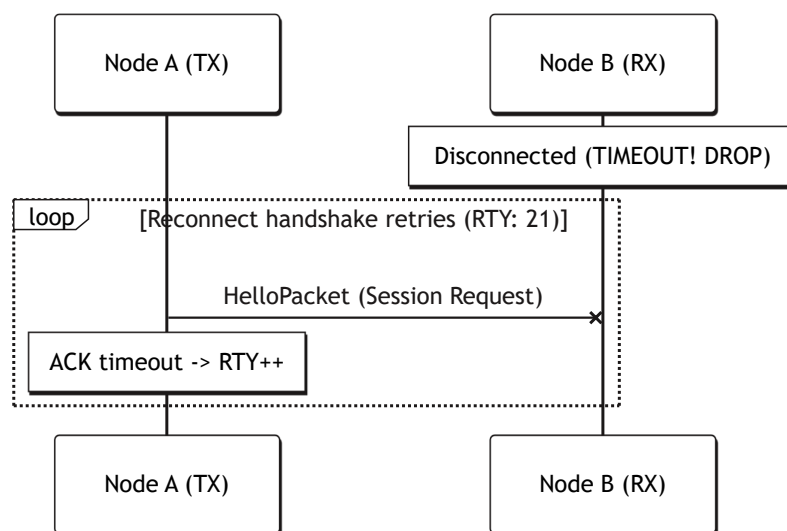


Figure 8.28: Session re-initialization attempts under active jamming. Corrupted HelloPacket transmissions fail to produce an ACK, forcing Node A to loop in WAIT SYN

with an increasing retry counter while Node B remains safely dropped.

### 8.3.8 Scenario 8: Successful Session Recovery and Application State Persistence

In this scenario, the prolonged physical-layer denial finally ceases: the attacker deactivates the Digital Noise Generator, and the disturbance process becomes quiescent,

$$N(t) = 0$$

With the forward channel cleared, the C2P-ARQ self-healing path can complete. The transmitter's `HelloPacket` session request is now delivered with sufficient structural integrity for the receiver to admit it, establishing a fresh cryptographic session. Normal data transmission resumes immediately thereafter.

The topology overview at the moment recovery is achieved is shown in Figure 8.29. The jammer is disabled, and the system returns to normal operation, with both end-points actively participating in the transport-layer exchange.

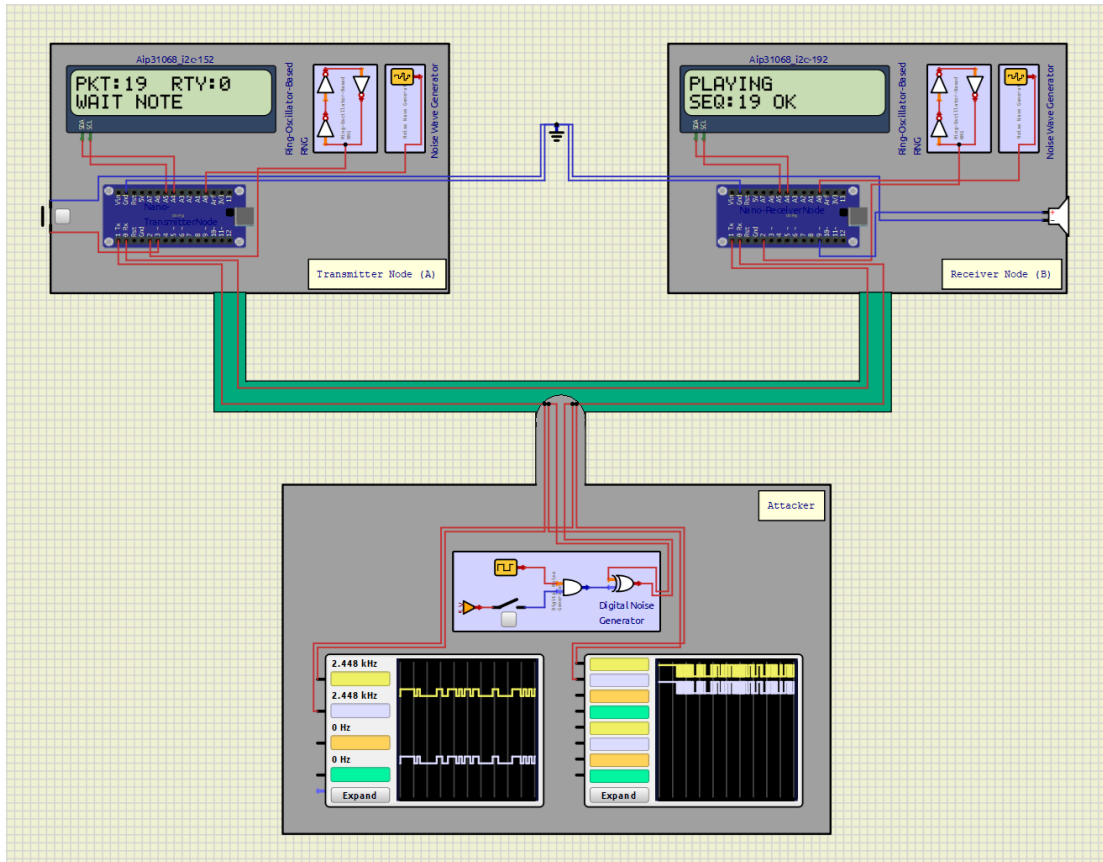


Figure 8.29: Recovery success topology. With the Digital Noise Generator deactivated, the forward channel is clear and SH2SC-EDT resumes normal

## C2P-ARQ operation.

The endpoint LCDs in Figure 8.30 and Figure 8.31 confirm that end-to-end synchronization has been restored under the newly established session. Node A reports that it is processing the 19th packet of the new session (PKT: 19 WAIT NOTE), while Node B confirms authenticated progress to the same point (PLAYING SEQ: 19 OK). This alignment indicates that both the handshake and the subsequent DATA stream are being decoded and acknowledged consistently.

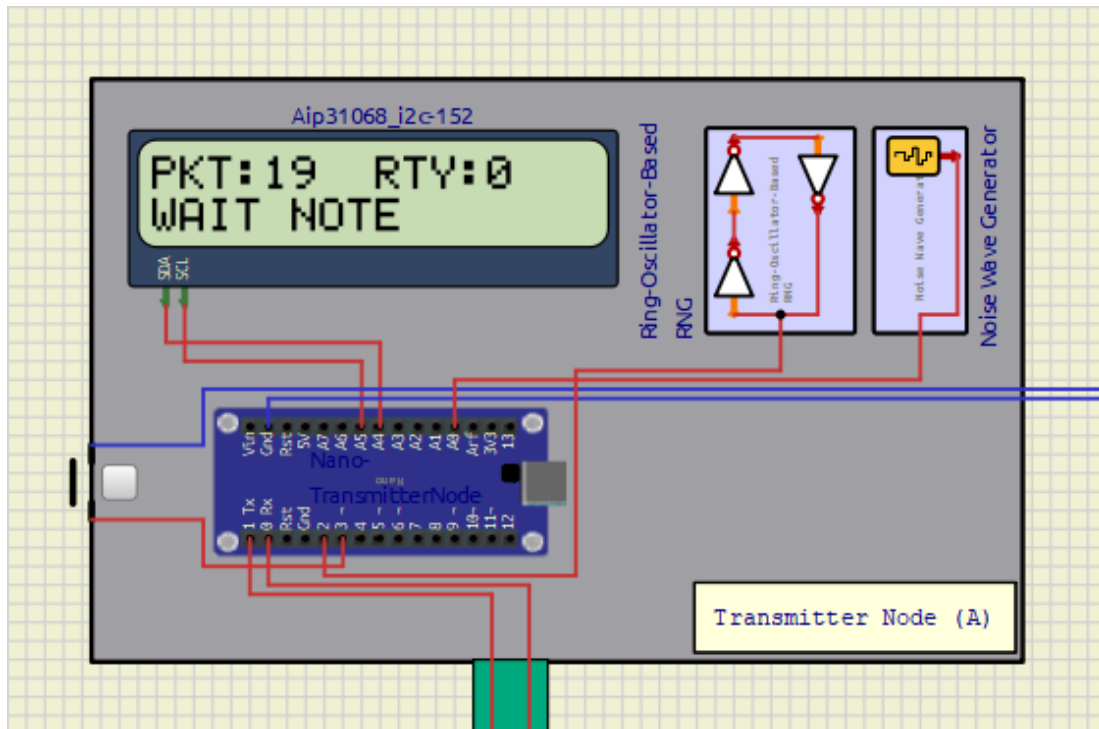


Figure 8.30: Node A status after successful recovery. PKT: 19 indicates forward progress within the new session, and WAIT NOTE indicates normal inter-packet pacing has resumed.

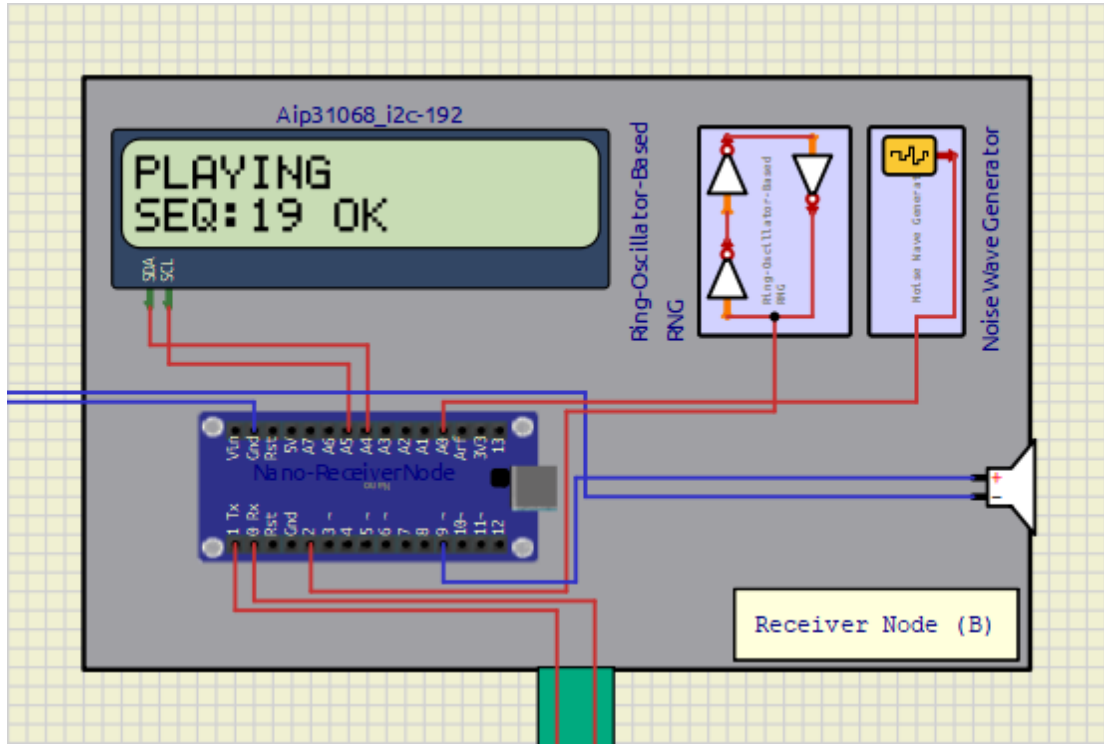


Figure 8.31: Node B status after successful recovery. PLAYING SEQ: 19 OK confirms authenticated reception and application-layer execution at the synchronized sequence position.

This observation captures the central architectural separation in SH2SC-EDT: the Link/Session Layer is deliberately ephemeral and may be re-initialized, while the Application Layer payload stream is stateful and persistent. Because a new cryptographic session is established, the packet sequence numbers are re-started from zero to preserve nonce integrity for ChaCha20-Poly1305 under the fresh session context. However, the application payload state does not reset: the melody execution resumes from the exact note at which the prior session was severed, ensuring zero data loss despite the prolonged outage.

The physical-layer clearance is demonstrated directly by the oscilloscope in Figure 8.32. Under the XOR injection model, the receiver-side waveform becomes

$$S_{rx}(t) = S_{tx}(t) \oplus 0 = S_{tx}(t)$$

so the forward channel is electrically transparent in the absence of attacker disturbance.

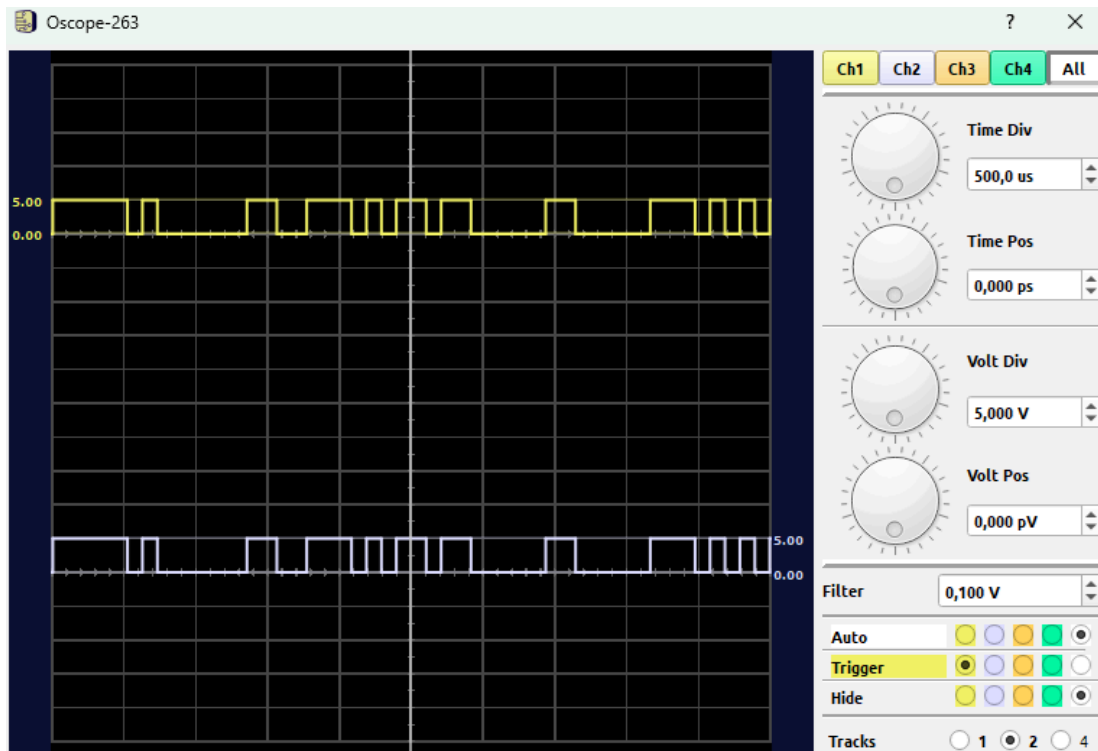


Figure 8.32: Oscilloscope evidence of recovery. With the noise path disabled, the XOR stage passes the transmitter UART waveform without inversion, confirming that the medium is free of adversarial interference.

At the protocol layer, Figure 8.33 shows a clean, uninterrupted data frame consistent with successful framing and decoding. In contrast to the jammed traces of Scenarios 4 and 7, the captured stream preserves stable structure across the entire frame, confirming that recovery is not merely a local FSM transition but an end-to-end restoration of the transport exchange.

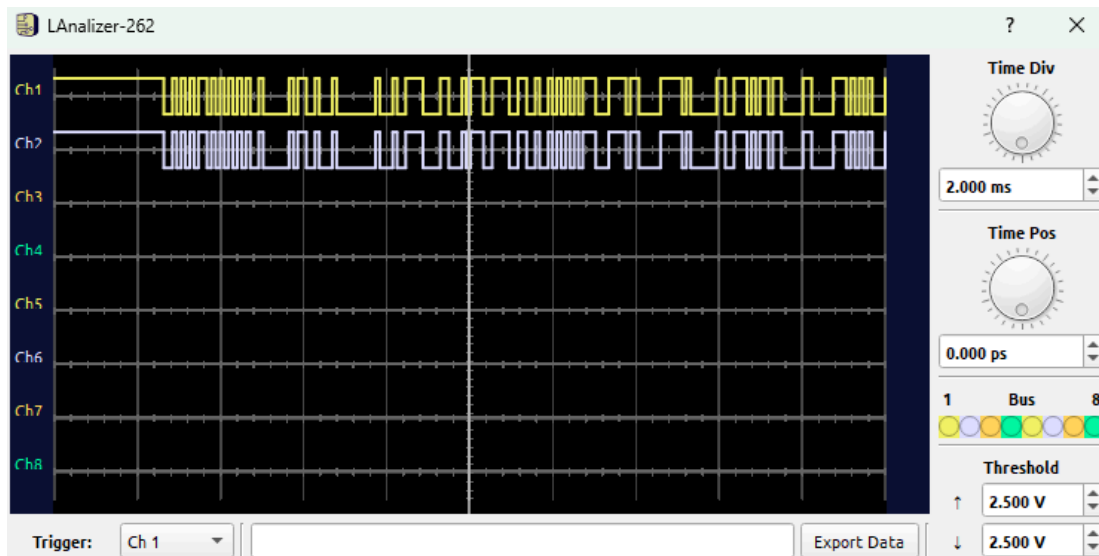


Figure 8.33: Logic-analyzer confirmation of successful protocol-level recovery. The data frame is captured without interruption, consistent with a clear channel and correct C2P-ARQ decoding.

The successful self-healing recovery after jamming cessation is summarized in Figure 8.34. A new session is established, and payload execution resumes from the preserved application state.

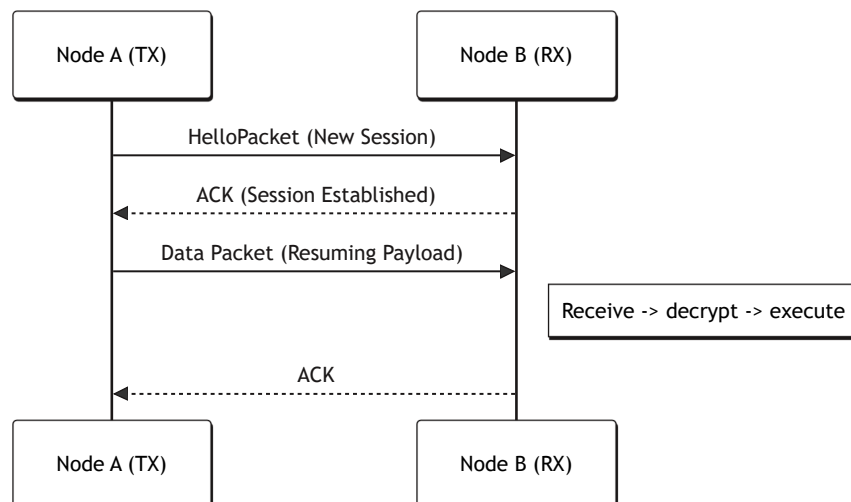


Figure 8.34: Successful self-healing recovery after jamming cessation. A new session is established, and payload execution resumes from the preserved application state.

### 8.3.9 Scenario 9: Legitimate Session Termination and Traffic Analysis Evasion

In this final scenario, the application-layer payload transmission has completed in full. The Digital Noise Generator remains disabled, and the disturbance process remains quiescent,

$$N(t) = 0$$

The system therefore transitions from steady-state encrypted data delivery into an explicit, authenticated session-termination path.

Node A (TX) sends a final full-sized encrypted frame carrying the termination command (the **FINAL** flag) within the protected payload. After receiving the receiver's acknowledgement, Node A exits the active transport lifecycle and enters its terminal **IDLE** state. Node B (RX) receives the final frame, performs authenticated decryption, identifies the termination condition, and closes the cryptographic session. In doing so, Node B also records the total number of executed payload chunks, thereby providing an explicit end-of-session audit point.

The Figure 8.35 provides the topology overview at the moment the session is closed. Unlike the jammed intervals earlier in this subsection, the system is at rest by design rather than by denial: the channel is quiet, and both endpoints have transitioned into terminal states consistent with a completed transmission.

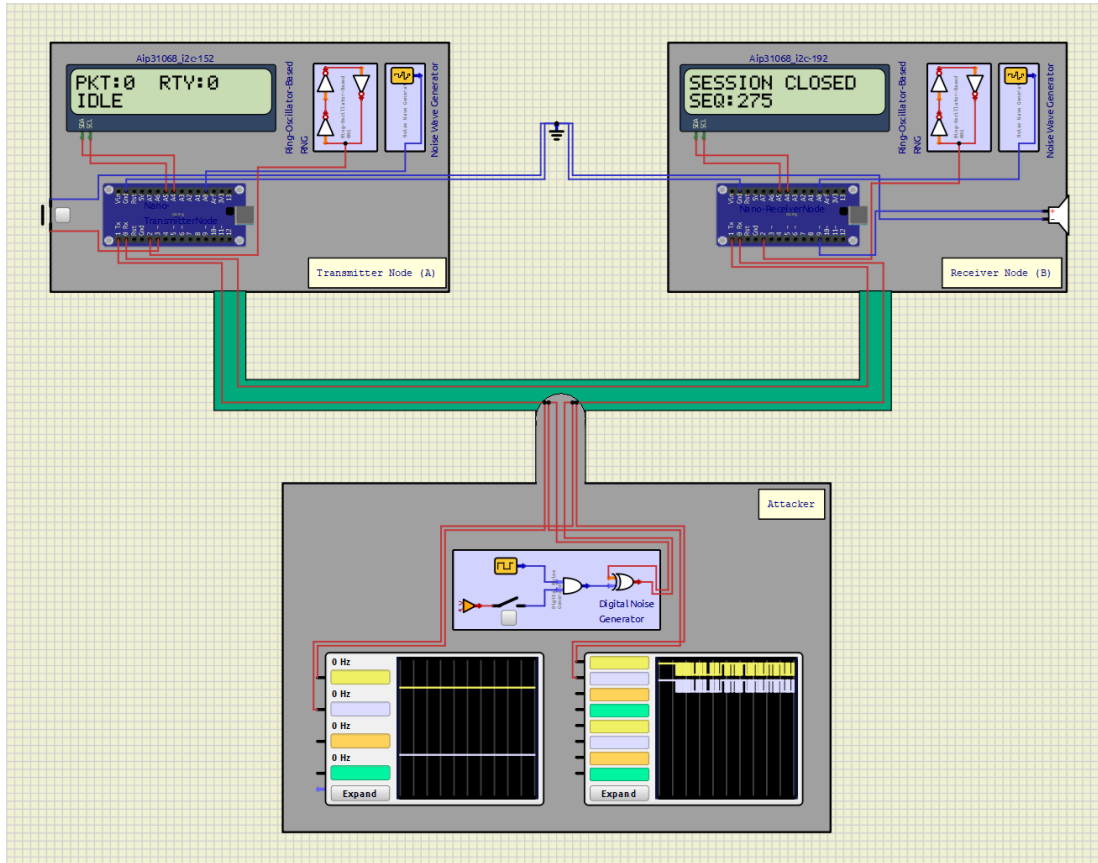


Figure 8.35: Final topology after legitimate session termination. The attacker remains disabled and both endpoints have transitioned into their post-session rest states.

Figure 8.36 and Figure 8.37 provide the endpoint-level termination evidence. Node A's PKT: 0 IDLE indicates that the transmitter has ceased all session activity and is no longer emitting protocol frames. Node B's SESSION CLOSED SEQ: 275 confirms that the receiver recognized the termination command and closed the session in a controlled manner, while also exposing the total authenticated workload processed during the completed transmission.

The transmitter post-session LCD state is shown in Figure 8.36.



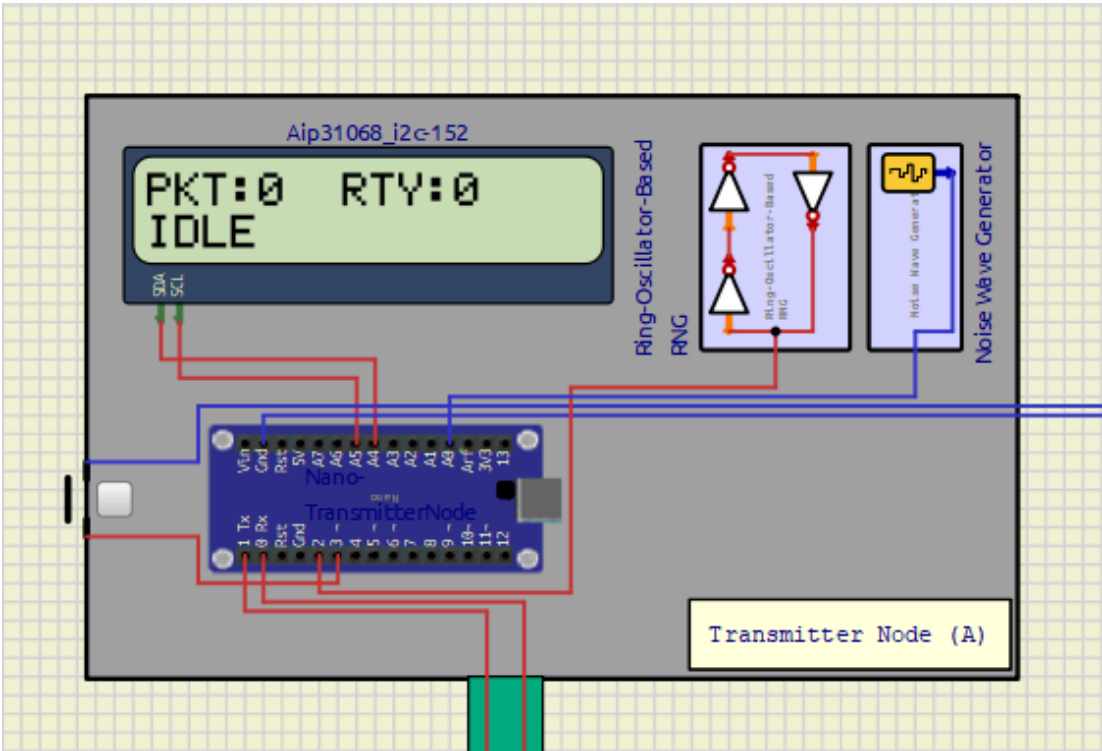


Figure 8.36: Node A post-session state. PKT: 0 IDLE indicates that the transmitter has completed the final authenticated exchange and returned to its terminal idle posture.

The receiver post-session LCD state is shown in Figure 8.37.

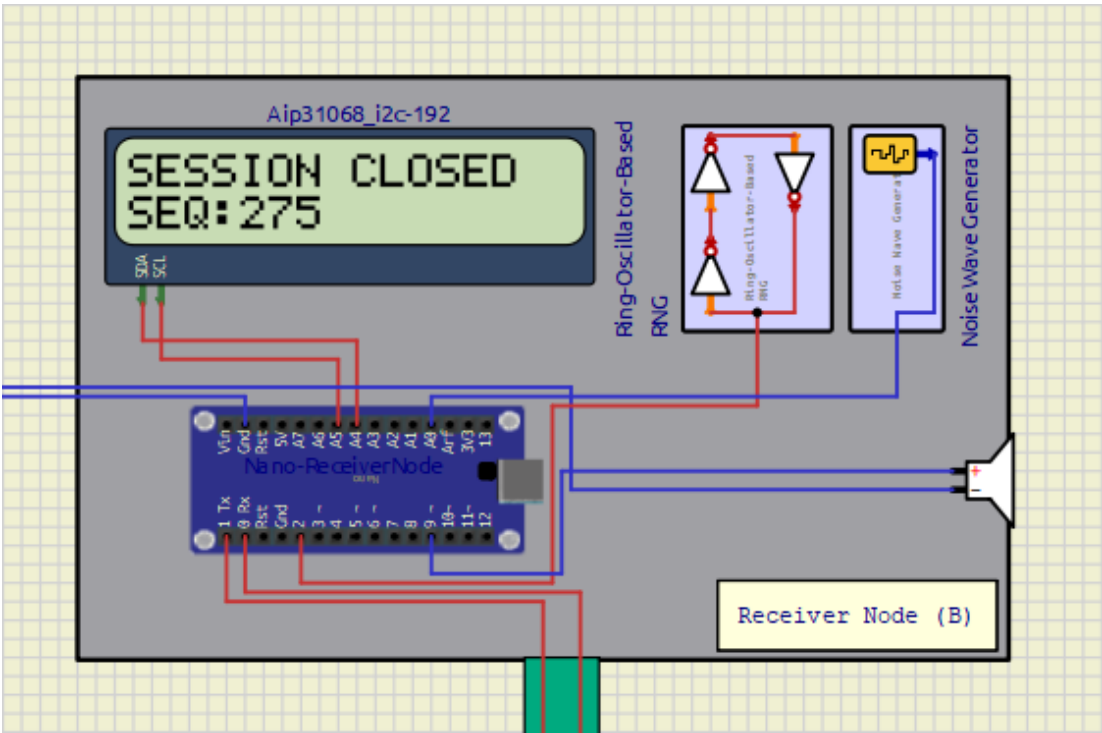


Figure 8.37: Node B post-session state. SESSION CLOSED SEQ: 275 confirms legitimate termination and reports the total number of authenticated payload chunks executed during the session.

The physical-layer quiescence of the post-session channel is captured by the oscilloscope in Figure 8.38. With no transmission activity and no attacker disturbance, the line satisfies

$$S_{tx}(t) = 0, \quad N(t) = 0, \quad S_{rx}(t) = S_{tx}(t) \oplus N(t) = 0$$

so the receiver-side input collapses to a flatline consistent with a legitimate end of communication.

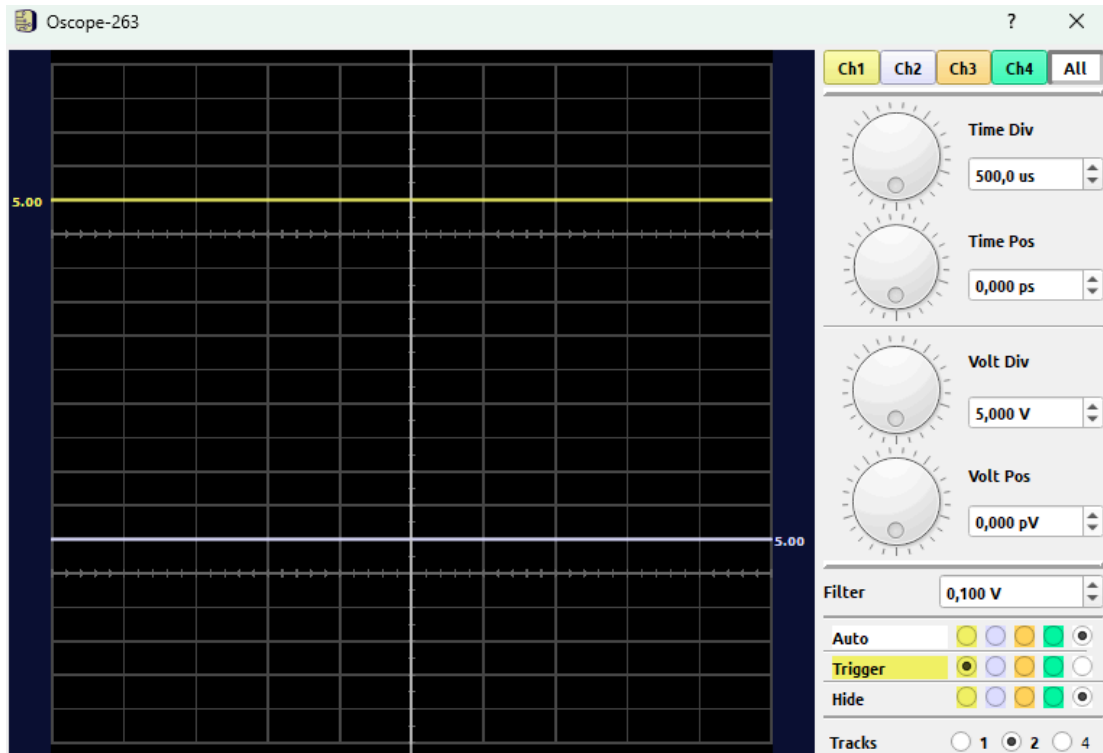


Figure 8.38: Oscilloscope evidence of a dead channel after legitimate teardown. The flatline is consistent with a zero-level receive line under the no-transmission, no-noise condition.

The protocol-side security property of this termination mechanism is exposed most clearly by the logic analyzer in Figure 8.39. The final packet is intentionally emitted as a full-sized frame that structurally mimics a standard encrypted data packet: it begins with the identical synchronization preamble, and its on-wire duration matches the normal data-frame footprint. This design eliminates a trivial traffic-analysis oracle based on

packet-size heuristics: an eavesdropper observing only frame lengths cannot infer that a session has ended merely because a shorter terminating frame appears. Instead, the termination command is expressed inside the encrypted payload and is therefore visible only to an authenticated endpoint.

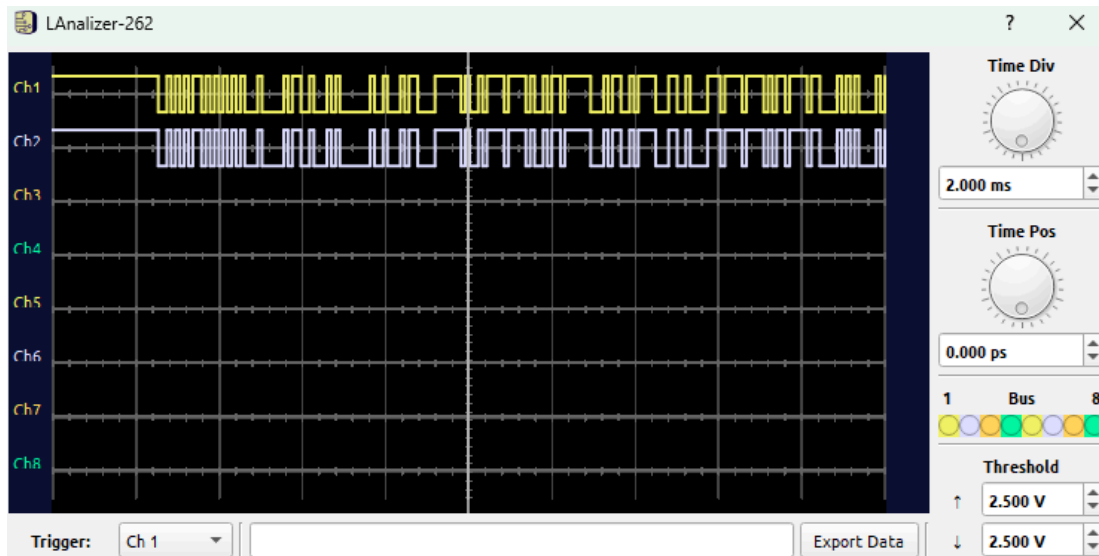


Figure 8.39: Logic-analyzer capture of the final terminating frame. The frame is full-sized and preamble-identical to standard encrypted data traffic, supporting traffic-analysis evasion by suppressing termination inference from packet length.

The teardown exchange is summarized in Figure 8.40. A terminating packet is transmitted and acknowledged as in any normal stop-and-wait data delivery, after which the transmitter enters IDLE and the receiver enters SESSION CLOSED.

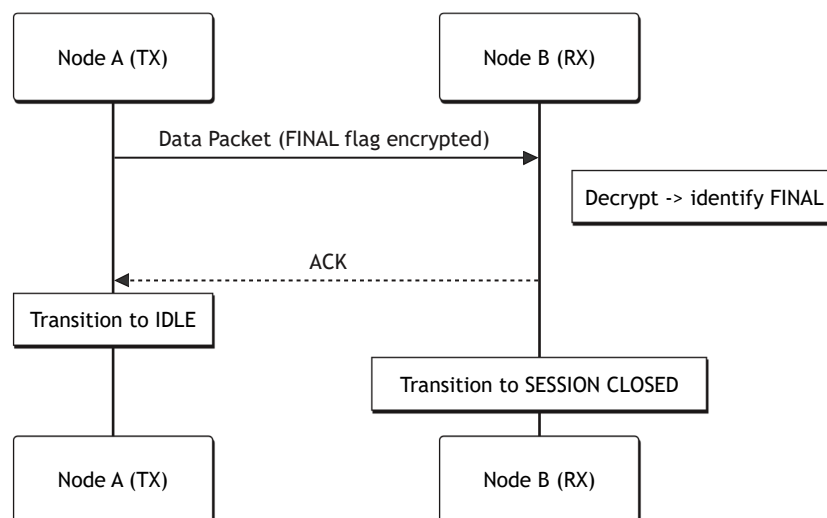


Figure 8.40: Legitimate session termination. The FINAL command is conveyed inside a full-sized encrypted frame and acknowledged before both endpoints transition into their terminal post-session states.

# 9 Conclusion and Future Work

This paper has presented the end-to-end design, implementation, and empirical validation of the SH2SC-EDT hardware-software complex and its C2P-ARQ transport protocol, integrating non-blocking finite-state control, ChaCha20-Poly1305 authenticated encryption, and adversarially grounded jamming experiments in SimulIDE on ATmega328P-class constraints. The following subsections synthesize the protocol’s principal demonstrated efficacies, namely resilience under sustained channel disruption, autonomous self-healing session recovery, and cryptographic security guarantees, and then define the next research trajectories focused on actuator-oriented IoT command migration and advanced power optimization through deep-sleep operating modes.

## 9.1 Summary of Protocol Efficacy

The reported results establish that SH2SC-EDT and its C2P-ARQ transport engine achieve a coherent synthesis of cryptographic assurance and deterministic embedded execution. Across the implemented protocol lifecycle, ChaCha20-Poly1305 AEAD was successfully integrated into a strictly non-blocking Stop-and-Wait ARQ finite-state architecture, thereby providing confidentiality for payload fields and strict frame authentication at the data-link layer through authenticated binding of control metadata and ciphertext. The empirical traces further confirm that protocol progression remains governed by bounded `millis()`-based transitions rather than blocking control paths, preserving timing determinism under AVR-class resource constraints.

The study also verifies the central Self-Healing property under sustained physical-layer degradation. [Section 8](#) demonstrates that, during prolonged jamming, the system autonomously detects unrecoverable channel conditions through bounded retry and timeout logic, performs safe cryptographic session teardown with nonce erasure, and enters controlled reconnection behavior without operator intervention. Once channel integrity is restored, session re-establishment proceeds securely via freshly generated TRNG-seeded session material, validating that recovery is both autonomous and cryptographically sound.

Finally, the experiments confirm that SH2SC-EDT decouples session continuity from application progress, which is the key condition for fault-tolerant delivery without semantic loss. While transport context is intentionally reinitialized after disruption, application

state is preserved, enabling exact Auto-Resume from the interrupted payload position and thereby demonstrating zero data loss across outages. In addition, the full-sized authenticated termination frame preserves on-wire structural indistinguishability from regular encrypted traffic, mitigating trivial traffic-analysis inference of session closure from packet-length signatures.

## 9.2 Migration to Actuator-Based IoT Commands

Although the present empirical campaign used an audio payload (a sequenced melody) as a proof-of-concept for stateful, time-dependent execution, the immediate research trajectory is to migrate the application layer toward direct control of physical IoT actuators. In practical terms, this includes command profiles for servomotors, industrial relays, smart locks, robotic kinematics chains, and SCADA-connected endpoints, where each authenticated packet corresponds to a materially consequential control action rather than an acoustic event. Such migration preserves the protocol core while elevating the validation domain from demonstrative signal playback to safety-critical cyber-physical actuation.

This transition is technically justified by the execution semantics already demonstrated by C2P-ARQ. Physical actuator workflows are inherently sequential and often irreversible at intermediate steps; therefore, they require deterministic command delivery with strict ordering and no silent omission. The protocol’s bounded ARQ behavior, zero-loss delivery objective, and Auto-Resume capability collectively provide the continuity guarantees needed for multi-step mechanical procedures to complete exactly as specified, even when severe channel jamming interrupts communication mid-operation. As a result, session disruption is transformed from a process-terminating failure into a recoverable transport event without loss of application-level progression.

From a cyber-physical security perspective, extending the platform to actuator control further amplifies the importance of authenticated encryption. ChaCha20-Poly1305 in the C2P-ARQ pipeline provides cryptographic rejection of replayed frames and malicious payload injection attempts that, in actuator-centric deployments, could otherwise induce unsafe state transitions, equipment stress, or catastrophic physical outcomes. Accordingly, the planned migration is not a change in protocol assumptions but a direct application of the validated SH2SC-EDT security and resilience properties to higher-impact CPS operational contexts.

### 9.3 Deep Sleep and Power Optimization

A remaining systems-level limitation, typical of deterministic polling architectures, is the sustained energy cost of continuous channel observation and uninterrupted ARQ finite-state evaluation. In the current SH2SC-EDT realization, this design choice is methodologically appropriate for timing transparency and fault-tolerance verification, yet it directly constrains deployment in battery-operated and energy-harvesting IoT environments, where long-term autonomy is dominated by idle-power budget rather than peak compute demand.

An immediate research trajectory is therefore the protocol-aware integration of hardware Deep Sleep modes into the C2P-ARQ lifecycle. In particular, the nodes should transition to ultra-low-power states during IDLE intervals and during prolonged application-layer waiting phases, including inter-payload execution delays that do not require active CPU participation. Such integration must preserve the established non-blocking state semantics by treating sleep entry and wake-up as explicit FSM transitions, thereby maintaining deterministic protocol behavior while minimizing duty cycle in quiescent periods.

To reconcile aggressive power reduction with reliable session continuity, wake-up should be delegated to hardware-driven asynchronous interrupts, for example pin-change triggers generated by the first observable edge of an incoming frame preamble. This interrupt-first reactivation model would allow the microcontroller to remain electrically dormant until genuine channel activity occurs, then resume authenticated C2P-ARQ processing with bounded latency. Combining the demonstrated cryptographic robustness of C2P-ARQ with such aggressive duty-cycling is expected to produce a fully autonomous, highly secure, and exceptionally energy-efficient SH2SC-EDT complex suitable for long-duration field deployment.

# References

1. Andrea, I., Chrysostomou, C., Hadjichristofi, G.: Internet of Things: Security Vulnerabilities and Challenges. In: IEEE Symposium on Computers and Communication (ISCC) (2015).
2. Al-Zubaidie, M., others: A Comprehensive Analysis of Code Injection Attacks in IoT. IEEE Access. (2020).
3. Smith, R.H., others: Vulnerabilities of COTS RF Modules: Replay Attacks on Accessibility Technology. In: Research Symposium (2022).
4. Deogirikar, J., Vidoushi, A.: Security Attacks in IoT: A Survey. In: International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (2017).
5. Neely, M., others: Security Analysis of Sub-GHz Radio Communications and Replay Attack Methodology. In: Australian Information Security Management Conference (2013).
6. Shen, Y., others: Task-Specific Vulnerability Analysis with Case-Driven Insights in RF Sensing. arXiv preprint. (2024).
7. Trend Micro ICS-CERT: A Security Analysis of Radio Remote Controllers: Command Injection and Replay Vulnerabilities. Trend Micro (2018).
8. Dolev, D., Yao, A.C.: On the Security of Public Key Protocols. IEEE Transactions on Information Theory. 29, 198–208 (1983).
9. Backes, M., Pfizmann, B., Waidner, M.: Soundness Limits of Dolev-Yao Models. In: Workshop on Formal and Computational Cryptography (FCC) (2006).
10. Vigano, L., others: Formalization of the Dolev-Yao adversary in Protocol Analysis. In: AAMAS (2006).
11. Smyth, B.: Formal Verification of Cryptographic Protocols with Automated Tools, (2011).
12. Cremers, C.: Formal Verification of Cryptographic Protocols: ProVerif and Scyther. IJCN. (2010).
13. Dupressoir, F., others: Formal Verification of Protocols using General-Purpose Tools, (2014).
14. Lewandowski, S.M.: Frameworks for Component-Based Client/Server Computing (Fat vs Thin Clients). ACM Computing Surveys. 30, (1998).



15. Negris, T., others: Thin Client Technology and User Acceptance. TUDublin School of Computing (2009).
16. IMM DTU: Client-Server Architectural Patterns: Thin or Thick? DTU Compute (2017).
17. Dash, A., Dasgupta, S.: Event-Driven Programming for Embedded Systems using FSM. Semantic Scholar. (2015).
18. Lee, E.A.: CFSM and Process Networks in Embedded Systems, (2003).
19. Billington, J., others: Formal Verification of Stop-and-Wait ARQ Using Petri Nets. In: ACM CCS (2005).
20. Berthomieu, B., others: Modeling and Verification of Embedded Real-Time Systems using Constraint Time Petri Nets. Mathematics / MDPI. (2020).
21. Shannon, C.: Entropy Harvesting in Silicon: TRNGs and PUFs. Information Security. (1948).
22. Fischer, V., others: Thermal Jitter from Free-Running Ring Oscillator in TRNGs. IEEE Transactions on Computers. (2021).
23. Tuncer, T., others: True Random Number Generator based on Ring Oscillators. IJRASET. (2023).
24. Wang, Y., others: Design of True Random Number Generator Based on Multi-stage Feedback Ring Oscillator. ResearchGate. (2024).
25. Kim, J., others: SRAM Data Remanence based TRNG/PUF stability for Key Generation. In: ISLPED ACM/IEEE (2017).
26. Turan, M.S., others: NIST SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation. NIST Special Publication (2018).
27. Barker, E., Kelsey, J.: NIST SP 800-90A Rev 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators. NIST (2015).
28. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the Indifferentiability of the Sponge Construction. In: EUROCRYPT 2008 / Springer LNCS (2008).
29. Keccak Team: The Sponge and Duplex Constructions, (2011).
30. Melo, V.: Sponge construction in lightweight cryptography (SymFrog-512). IACR ePrint. (2024).

31. Dobraunig, C., others: Ascon Specification (Sponge-based Lightweight Cryptography). NIST LWC Finalist (2019).
32. Aumasson, J.-P., others: ChaCha20-based CSPRNG for Embedded Systems. Cryptology ePrint Archive. (2020).
33. OpenBSD / Wikipedia: Cryptographically Secure Pseudorandom Number Generator: arc4random and ChaCha20, (2013).
34. Rogaway, P.: Authenticated-Encryption with Associated-Data. In: ACM Conference on Computer and Communications Security (CCS) (2002).
35. Bozhko, V.: RFC 9771: Properties of Authenticated Encryption with Associated Data (AEAD) Algorithms. IRTF/IETF (2025).
36. Bernstein, D.J.: ChaCha, a variant of Salsa20. In: SASC 2008 (2008).
37. Bernstein, D.J.: The Poly1305-AES Message-Authentication Code. In: Fast Software Encryption (FSE) (2005).
38. Nir, Y., Langley, A.: RFC 7539: ChaCha20 and Poly1305 for IETF Protocols. IRTF (2015).
39. Nir, Y., Langley, A.: RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. IETF (2018).
40. Housley, R.: RFC 8103: Using AEAD\_CHACHA20\_POLY1305 with CMS. IETF (2017).
41. Procter, G.: A Security Analysis of the Composition of ChaCha20 and Poly1305. Cryptology ePrint Archive, Paper 2014/613. (2014).
42. Coutinho, M., others: Differential and linear cryptanalysis of ChaCha. In: SBSEg / SBC (2020).
43. Joux, A.: Authentication Failures in NIST GCM (Nonce reuse in Stream Ciphers). In: NIST Block Cipher Techniques (2006).
44. Gueron, S., others: IETF length-encoding block and nonce reuse hazard (AES-GCM-SIV). RFC 8452 (2018).
45. Langley, A., others: POLYVAL and Length Block Encoding in AEAD. IETF Draft (2015).
46. Planck Collaboration: Planck 2018 results. VI. Cosmological parameters. *Astronomy & Astrophysics (A&A)*. 641, A6 (2020).

47. NASA WMAP Science Team: Universe Age Estimates based on WMAP, (2012).
48. ESA/NASA: Planck reveals an almost perfect Universe: Age estimates, (2013).
49. Chen, S.: Simplex Stop and Wait with ARQ. *Computer Networks*. (2003).
50. Fairhurst, G., Wood, L.: RFC 3366: Advice to Link Designers on Link ARQ. IETF Best Current Practice (2002).
51. Al-Jubari, A.: Performance of ARQ protocols over unreliable channels. *Semantic Scholar*. (2010).
52. De Vuyst, S., others: Performance Analysis of Stop-and-Wait ARQ for Wireless Channels with Bursty Errors. In: *HET-NETs* (2004).
53. Kumar, A., others: UART module using Verilog HDL (PISO/SIPO). *International Journal of Advanced Research*. (2025).
54. Sharma, P., others: UART Architecture with Built-in FIFO Buffering Feature. *VLSI Design*. (2021).
55. Shao, e.al.: Defending Embedded Systems Against Buffer Overflow. In: *Princeton University / ACM* (2004).
56. Massey, J.L.: Optimum Frame Synchronization. *IEEE Transactions on Communications*. 20, 115–119 (1972).
57. Recayte, E., Munari, A.: Packet detection and sync-word correlation. *Elsevier*. (2023).
58. Halderman, J.A., others: Lest We Remember: Cold Boot Attacks on Encryption Keys. In: *17th USENIX Security Symposium* (2008).
59. Yang, Z., others: Dead Store Elimination (Still) Considered Harmful. In: *26th USENIX Security Symposium* (2017).
60. Huang, Y., Kintala, C.M.R.: Software Rejuvenation: Analysis, Module and Applications. In: *25th International Symposium on Fault-Tolerant Computing (FTCS-25)* (1995).